Handwriting robot

Final Report

D.R. Braune
23539195

Submitted as partial fulfilment of the requirements of Project EPR400
in the Department of Electrical, Electronic and Computer Engineering
University of Pretoria

November 2026

Study leader: Prof. H.C. Myburgh

Part 1. Preamble

This report describes the engineering process followed to complete my final year, which involved creating a handwriting robot to classify and reproduce text in author's handwriting.

1. Project proposal and technical documentation
This main report contains an unaltered copy of the approved Project Proposal (as Part 2 of the report).

All the code that I developed appears as a separate submission on the AMS.

2. Project history
The project was done on my own, with advice from an ex-project student, algorithms and techniques inspired by referenced papers, like classifier structure.

3. Language editing
This document has been language edited by a knowledgeable person. By submitting this document in its present form, I declare that this is the written material that I wish to be examined on.

My language editor was _____.

_____ _____

Language editor signature *Date*

4. Plagiarism declaration

I, _____ understand what plagiarism is and have carefully studied the plagiarism policy of the University. I hereby declare that all the work described in this report is my own, except where explicitly indicated otherwise. Although I may have discussed the design and investigation with my study leader, fellow students or consulted various books, articles or the internet, the design/investigative work is my own. I have mastered the design and I have made all the required calculations in my lab book (and/or they are reflected in this report) to authenticate this. I am not presenting a complete solution of someone else or that I found somewhere.

Wherever I have used information from other sources, I have given credit by proper and complete referencing of the source material so that it can be clearly discerned what is my own work and what was quoted from other sources. I acknowledge that failure to comply with the instructions regarding referencing will regarded as plagiarism. If there is any doubt about the authenticity of my work, I understand that I may have to attend an oral ancillary examination/evaluation about the work.

5. AI usage declaration

The author confirms that no artificial intelligence tools, including language models or code-generation tools, were used in the preparation and writing, or any analysis that appears in this

report. All content, design, explanations, data analysis, and code were developed solely by the author.

6. Software library usage declaration

Unless explicitly stated otherwise in the next paragraph, the author confirms that no software library functions, packages, or toolboxes were used in the design, implementation, or analysis presented in this report. All code and computations were developed entirely by the author

7. Unaltered Project Proposal

I certify that the Project Proposal appearing as the Introduction section of the report is an unaltered copy of the approved Project Proposal.

Signature

With my signature below, I hereby certify that all statements, declarations, and information presented above are complete.

D.R. Braune	Date

TABLE OF CONTENTS

Part 2. Project definition: approved Project Proposal

1. Project description
2. Technical challenges in this project
3. Functional analysis
4. System requirements and specifications
5. Field conditions
6. Student tasks

Part 3. Main Report

1. Literature study 1
    - 1.1. Handwriting recognition and writer identification 1
        - 1.1.1. Offline recognition and available datasets 1
        - 1.1.2. Image conditioning and text separation 1
        - 1.1.3. Neural networks and feature extraction 2
        - 1.1.4. Training and assessment 4
        - 1.1.5. Techniques for writer identification 4
    - 1.2. Handwriting synthesis in a writer's style 4
    - 1.3. Physical reproduction and pen-path mapping 5
    - 1.4. Application of the literature 6

2. Approach 7
    - 2.1. Mechanical Hardware Design 7
    - 2.2. Page condition and line segmentation 7
    - 2.3. Text and writer classification 8
    - 2.4. Style extraction and handwriting reproduction 8
    - 2.5. Integration and testing 8

3. Design and implementation 9
    - 3.1. Design summary 9
    - 3.2. System block diagram 10
    - 3.3. Hardware Design 11
        - 3.3.1. Plotter designs and mechanics 11
        - 3.3.2. Stepper motor and driver 12
        - 3.3.3. Step resolution, microstepping and calibration 15
        - 3.3.4. Pen up/down mechanism 16
        - 3.3.5. End stop switches and calibration 18
        - 3.3.6. Gcode generation and motion mapping 20
    - 3.4. Page conditioning and line segmentation 23
        - 3.4.1. Lighting normalisation and ink extraction 24
        - 3.4.2. Page orientation and skew correction 25
        - 3.4.3. Line detection and separation 25
        - 3.4.4. Assigning ink to the correct line 28
        - 3.4.5. Preparing the classifier crops 29

3.4.6. Complete processing sequence and output 30
3.4.7. Segmentation verification and limitations 31
3.5. Neural network design, training and classification 32
3.5.1. Training data, labels and evaluation separation 32
3.5.2. Shared convolutional feature extractor 33
3.5.3. Text recognition using sequence modelling and CTC 36
3.5.4. Writer identification and stroke normalisation 37
3.5.5. Training procedure and model selection 40
3.5.6. First principles and framework implementations 41
3.5.7. Network verification and design limitations 42
3.6. Style extraction and handwriting reproduction 43
3.6.1. Choice of reproduction representation 43
3.6.2. Character alignment and glyph extraction 44
3.6.3. Skeleton tracing and pen path construction 45
3.6.4. Glyph selection and library quality 47
3.6.5. Writer style measurement and storage 50
3.6.6. Text layout and connected handwriting synthesis 50
3.6.7. Candidate selection for legibility and writer identity 52
3.6.8. Machine coordinate mapping and G-code generation 54
3.6.9. Reproduction verification and design limitations 56
3.7. System integration and embedded operation 57
3.7.1. Classification and reproduction workflows 57
3.7.2. Embedded execution, interfaces and job control 58
3.7.3. Integration verification and remaining constraints 58

4. Results 59
4.1. Summary of results achieved 59
4.2. Qualification tests 61
4.2.1. Test material, configuration and measurement definitions 61
Qualification test 1: Reproduction of digital text in a trained writer's style (R1)
Objectives of the test or experiment 62
Equipment used 62
Test setup and experimental parameters 62
Steps followed in the test or experiment 62
Results or measurements 62
Observations 65
Statistical analysis 65
Qualification test 2: Reading handwritten content and identifying its writer (R2)
Objectives of the test or experiment 65
Equipment used 65
Test setup and experimental parameters 65
Steps followed in the test or experiment 66
Results or measurements 66
Observations 67
Statistical analysis 68
Qualification test 3: Complete reproduction time on the embedded platform (R3)

Objectives of the test or experiment 68
Equipment used 68
Test setup and experimental parameters 68
Steps followed in the test or experiment 68
Results or measurements 68
Observations 69
Statistical analysis 69
Qualification test 4: Time to decode a photographed page and identify its writer (R4)
Objectives of the test or experiment 69
Equipment used 69
Test setup and experimental parameters 69
Steps followed in the test or experiment 70
Results or measurements 70
Observations 71
Statistical analysis 71
Qualification test 5: Gantry dimensional accuracy and geometric consistency (R5)
Objectives of the test or experiment 71
Equipment used 71
Test setup and experimental parameters 71
Steps followed in the test or experiment 71
Results or measurements 72
Observations 72
Statistical analysis 72
Qualification test 6: Storage and use of multiple trained styles (R6)
Objectives of the test or experiment 73
Equipment used 73
Test setup and experimental parameters 73
Steps followed in the test or experiment 73
Results or measurements 73
Observations 73
Statistical analysis 73
4.2.2. Qualification under the proposal's field conditions 73

5. Discussion 74
5.1. Critical evaluation of the design 74
5.1.1. Interpretation of results 74
5.1.2. Critical evaluation 76
5.1.3. Unsolved problems 78
5.1.4. Strong points of the design 79
5.1.5. Expected failure conditions 79
5.2. Considerations in the design 80
5.2.1. Ergonomics 80
5.2.2. Health and safety 80
5.2.3. Environmental impact 81
5.2.4. Social and legal impact 81
5.2.5. Ethics clearance 81

6. Conclusion 82
6.1. Summary of the work completed 82
6.2. Summary of observations and findings 82
6.3. Contribution 82
6.4. Suggestions for future work 82

7. References 83

A. Supporting implementation calculations 87
A.1. Numerical network derivatives 87
A.2. Optimisers and gradient scaling 88
A.3. Morphology, thinning and path simplification 88
A.4. Profile and trajectory quantities 89
A.5. Detailed training procedures 91
A.6. Expanded numerical calculation checks 91
A.7. Implementation and interface reference tables 92
A.8. Motion-file conversion procedure 94
A.9. Controller pin and supply planning 94

B. Subsystem visual evidence and independent software checks 97
B.1. Actual capture variation and input resolution 97
B.2. Independent component and seam references 98
B.3. CTC probability and derivative check 98
B.4. Stored profiles, emitted files and physical page 99
B.5. Candidate-scoring configuration and scope 101
B.6. Post-design seam-coefficient sensitivity 101
B.7. Real-input calculation traces 101
B.8. Visual checks of numerical training operations 103
B.9. Stroke-width quantisation in the validation sweep 103
B.10. Glyph coordinates and measured connectedness 105

C. Supporting qualification evidence 106
C.1. Evaluated checkpoints and populations 106
C.2. Physical reproduction by writer 106
C.3. New geometric style comparison on physical pages 107
C.4. Archived geometric style study 109
C.5. Timing selection and optimisation scope 109
C.6. Cost, consent and final records 110
C.7. Supporting timing and dimensional figures 111

LIST OF ABBREVIATIONS

BiLSTM	bidirectional long short-term memory

Part 2. Project definition: approved Project Proposal

This section contains the problem identification in the form of the complete approved Project Proposal, unaltered from the final approved version that appears on the AMS.

For use by the Project lecturer	Approved	✓	Revision required	

Feedback

For use by the Project module lecturer only

Approved - the project is clearly delineated.
However, a number of things to pay attention to in especially section 4.

Symbol awarded: B

To be completed by the student

PROJECT PROPOSAL 2026

Project no	HCM4	Revision no	0

Title	Surname	Initials	Student no	Study leader (title, initials, surname)
Mr	**Braune**	**D.R**	23539195	Prof. H.C. Myburgh

Project title (this is the title on the project concept note)

Handwriting robot

Language editor details	Language editor signature
M.T. Braune	M.T. Braune
Student declaration I understand what plagiarism is and that I have to complete my project on my own.	**Study leader declaration** This is a clear and unambiguous description of what is required in this project. Approved for submission (Yes/No)
Student signature	Study leader signature and date 20 April 2026

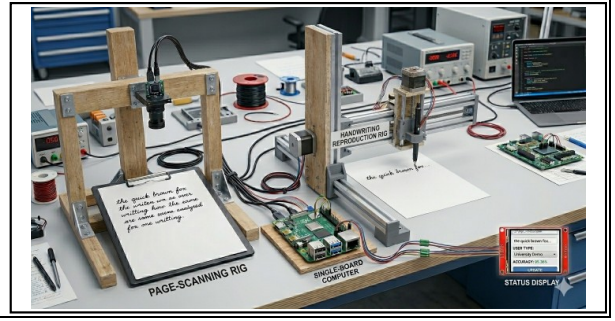
1. Project description.

What is the problem to be solved with your project? Describe in non-technical language what your project is about. What does your system have to do? Refer explictly to Figure 1 that you inserted on the right.

The need for handwriting still exists for authenticity, ease of communication and personal touches. The problem is that many individuals do not possess the time, skills or ability to write neatly and legibly. This leads to the solution presented in this proposal: a handwriting robot capable of recognizing a user's handwriting style and being able to replicate it.

The proposed solution is a handwriting robot capable of training on multiple different user's handwritten text (using a page-scanning rig to capture the handwritten page like the one seen in the image) and being able to produce any desired piece of text in any of these user's writing styles (using a text reproduction rig like the one seen in the image). The system will be able to recognize which user authored a piece of text and identify what text was written (shown to the user through a display as seen in the image). A gantry with 2-dimensional movement will perform actions with a pen to automatically reproduce the desired text.

Figure 1. Insert an image below that captures the idea with your project. Explain in the text on the left.



2. Technical challenges in this project

Describe the technical challenges that are *beyond* those encountered up to the end of third year and in other final year modules.

2.1 Primary *design* challenges

What is the intended core technical challenge in the project? Which aspect or aspects of the design of the system do you expect to be the most challenging?

1. Text image processing will be performed from first principles, including the isolation of letters from an image of various fonts and sizes of handwritten text.
2. A neural network will be designed and trained from first principles to be able to categorize text to different user's styles based on certain style elements, and identify the text written.
3. A gantry rig will be designed to be capable of performing the necessary precise 2-dimensional movements using a pen, whilst holding the pen and paper securely in place.
4. Pen path planning will be implemented, which involves the conversion of written characters to a 2D motion the gantry needs to perform, and then mapping the movement to stepper motor controls.

2.2 Primary *implementation* challenges

Which aspect or aspects of the implementation do you expect to be the most challenging?

1. Image processing and deep learning will be performed on a single board computer (instead of on a personal computer), which has limited memory, storage and power.
2. Smooth motor control will be needed to produce precise and consistent pen movement to create accurate text.
3. Near real-time classification of handwritten text and reproduction of digital text on paper will need to be achieved.

All processing will be done on an embedded platform, and not on a PC platform. Confirmed by having ticked this box. ☑

Library functions will be used for hardware interfacing only. Confirmed by having ticked this box. ☑

3. Functional analysis

3.1 Functional description

Describe the design in terms of system functions as shown on the functional block diagram in section 3.2. This description should be in *narrative format*. **DO NOT** use a bullet list.

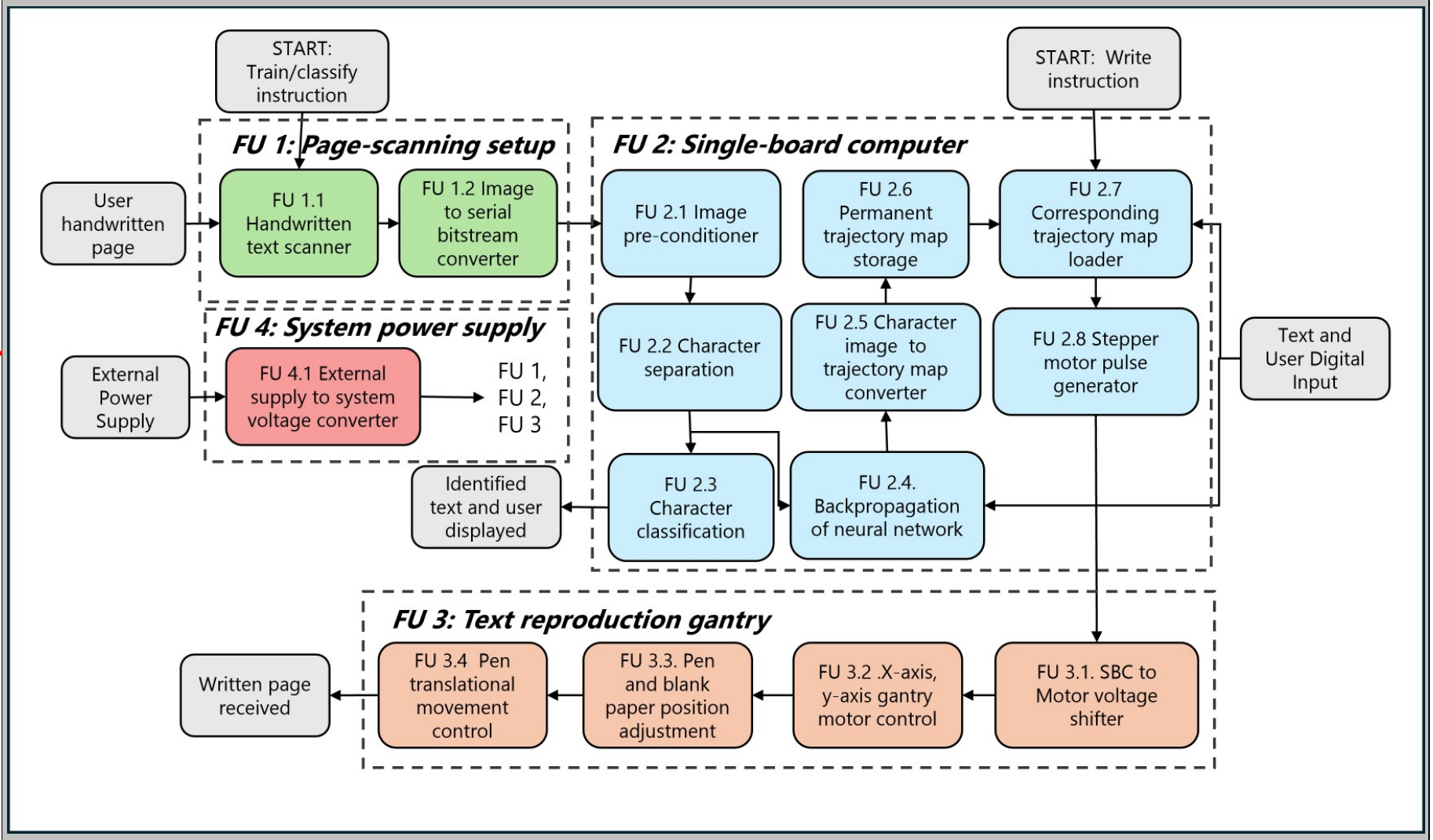
There are three main modes of operation for the handwriting robot: training, classification and reproduction

Training/Classification: First, the user will input their page of handwritten text to the scanner (FU 1.1). The user will select to either train or classify, and will input the intended digital text alongside this through an input interface. Once the page is scanned, a camera USB driver will be used to transfer the image to the single board computer (FU 1.2). The image will be conditioned first to fix warping or noise in the image (FU 2.1), and then the character separation will occur. This will separate the page characters (or groups of characters that are connected) to match the desired neural network input size (FU 2.2). The difference between training and classification occurs here: if the 'training' option is selected, the separated characters will be used to adjust the neural network neuron weightings against the desired digital text that was input (FU 2.4). Each separated character or group of characters will also be converted into a trajectory map (coordinate map of required pen motion) (FU 2.5). This trajectory map will be stored permanently within the system (FU 2.6). If the 'classify' option is selected, the separated characters or group of characters will be passed into the trained neural network classifier (FU 2.3), and the corresponding digital text and detected user will be outputted to the display interface.

Reproduction: If the user sends a 'write' instruction, the digital text and user style desired to be reproduced will be provided. The corresponding trajectory for each character/group will be retrieved from storage (FU 2.7), and will be converted into stepper motor controls required to produce the desired pen movements (FU 2.8). These stepper motor controls will be sent to the corresponding motors via a stepper motor driver to allow for forward and reverse movements (FU 3.1). The x-motor, y-motor and pen lift mechanism will be controlled through the stepper motor driver (FU 3.2/3.4). After the pen has been adjusted and is held securely by the pen holder in the robot (FU 3.3), precise handwritten imitations of each character will be written.

Remove power supply (FBD should shown system functions).
Remove the "start" blocks - this is not a flow diagram, but rather an FBD.

3.2 Functional block diagram (this should not be a flow diagram)



4. System requirements and specifications

These are the core requirements of the system or product (the mission-critical requirements) in table format **IN ORDER OF IMPORTANCE**. Requirement 1 is the most fundamental requirement.

	Requirement 1: the fundamental functional and performance requirement of your project	**Requirement 2** (Second in order of importance)	**Requirement 3** (Third in order of importance)
1. Core mission requirements of the system or product. Focus on requirements that are **core** to solving the engineering problem. These will reflect the solution to the problem.	The system must be able to accurately reproduce any desired digital text on paper in the writing style of any writer from the training set.	The system must be able to accurately decode the content of the handwritten text, and identify which writer from the training set wrote the handwritten text.	The system must reproduce text on paper at a near real time speed, at a rate that can be considered practical and useful.
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?	The system should reproduce desired text with a minimum character accuracy of 85% while having a high stylistic correlation of 85% to the chosen writer's style.	The system must be able to identify the writer of the handwritten text from the training set and the content of the written text with an overall accuracy of 95%.	The system must be capable of reproducing the desired text in the writing style of any writer in the training set at a rate of no slower than 4 seconds per character.
3. Explain where this specification originated : i.e., **explain** the value that you selected. I.e., *how* will meeting the specification given in point 2 above *solve the problem*?	An accuracy of 85% (averaged from Babushkin et al. results [1, p8]) is high enough to represent the input text, ~~but practical to account for the build up of precision loss in the system.~~ see class discussion on what specs should be based on.	An overall classification accuracy of 95% (similar to existing classifiers mentioned in Acta Informatica Pragensia [2, p6]) ensures practical usability and consistency.	Averaging the time required to produce text between 0% and 1% error (Ossi Teikari [3, p35]) gives approximately 4 seconds per character, allowing the system to reproduce text at a practical rate with minimal error.
4. How will you demonstrate at the examination that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will ***prove*** these were met.	A sentence of text will be reproduced in a specific user style, and the amount of characters that can be visibly seen to match the input text (in the style of the chosen user) must be >= to 85% of the total characters produced. This is not a proper way to measure write in text	A predefined paragraph of text must be handwritten by a writer in the training set and classified by the system. The classified text and writer must match the input text and writer with an overall accuracy of >= 95% .	A sentence of x characters long will be sent to the system to be reproduced, the system must be able to reproduce it within 4*x seconds. this means "convolution"
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*	Accurate conversion from character image to pen trajectory map, precise motor control from pen trajectory map, and a structurally sound gantry design for accurate drawing will be designed and implemented by the student.	A rig to capture an image of written text, image conditioning and noise removal, character separation and processing, classification with a neural network, and communicating necessary data to a display will be designed by the student.	Techniques to optimize pen movements for quicker drawing, smooth motor control and robust design for efficient gantry movements will be designed and implemented by the student.
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks **library functions** will be used (if relevant to the project).	Physical motors, pen, single board computer, stepper motor driver, and separate components for gantry design e.g. screws or belts will be taken off the shelf.	A camera, camera driver (library function), single board computer, separate components for image-capturing rig, and a physical device where data is displayed will be taken off the shelf.	Efficient physical motors, single board computer, stepper motor driver, and separate components for gantry design e.g. smooth ball bearings will be taken off the shelf.

the NUMBER of characters, NOT amount.

System requirements and specifications page 2

	Requirement 4 (Fourth in order of importance)	**Requirement 5**	**Requirement 6**
1. Core mission requirements of the system or product. Focus on requirements that are core to solving the engineering problem. These will reflect the solution to the problem.	The system must be able to identify the writer of the handwritten text, and decode the content of the handwritten text in a short, practical amount of time.	The gantry of the system must have precise pen movements and accurately draw the desired characters with minimal positional error.	The system should be able to be trained on multiple different writing styles and be capable of storing all the writers' necessary character trajectory maps within the system permanently.
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?	The system must identify the writer of the written text from the training set and decode the content of the text at a rate of no slower than 25ms per character with an extra 2 seconds for image capture/conditioning.	The gantry of the system must be capable of drawing the desired character trajectory map while maintaining an absolute positional error that does not exceed 0.5mm. 0.5 mm relative to text that is how large?	The system should be capable of training on a minimum of 10 different users' writing styles, and be capable of storing the necessary character mappings for each of these 10 user's permanently.
3. Explain where this specification originated: i.e., explain the value that you selected. I.e., *how* will meeting the specification given in point 2 above solve the problem?	2 seconds was given to image capturing and conditioning due to transmission times and complex conditioning and noise removal algorithms, and 25ms per character classification due to simple forward propagation calculations. same comment as before. Specs are selected to best solve the problem, not because "processing will take long".	According to research by T. Huang and R. Xiong [4] an absolute positional error of 0.3mm with a gantry is achievable and indicates sufficient accuracy, therefore a maximum absolute error of 0.5mm was chosen for slightly more leeway. imcorrect motivation	Having 10 trained user styles stored at the same time proves multi-user neural network recognition and multi-user storage. More than 10 is not required due to neural network complexity and storage limitations. same comment again
4. How will you demonstrate at the examination that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will *prove* these were met.	A page with a sentence of x handwritten characters will be given to the system. The time from the initial instruction to the received classified text and user should be within 2 + x*0.025 seconds.	The system should be instructed to recreate the same word twice without varying the dataset characters, and the two words should appear identical on-top of each other, and not exceed an absolute positional error of 0.5mm.	The system should display the current amount of users in the system, and any random user should be selected out of the 10 and have text reproduced in their style to prove the system can be trained with 10 different users.
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*	A rig to capture an image of written text, image conditioning,optimized and efficient character separation, neural network character classifier, and calculation and display of time taken will be designed and implemented by the student.	Accurate conversion from pen trajectory map to precise stepper motor control, robust gantry system to write characters, secure pen adjuster and holder, and a secure paper holder will be designed and implemented by the student.	A rig with a camera to capture new text, sufficiently complex neural network to be able to classify minimum 10 user's styles, and a storage strategy to store 10 user's character trajectory mappings will be designed by the student.
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks **library functions** will be used (if relevant to the project).	A camera, camera driver, single board computer, separate components for image-capturing rig, and a physical device where time is displayed will be taken off the shelf.	Precise stepper motors, separate components needed for gantry, pen and paper holders, and a pen will be taken off the shelf. This show repeatability, not accuracy	A camera, camera driver, single board computer, components for image-capturing rig, and permanent storage device will be taken off the shelf. NUMBER of users. Not amount.

System requirements and specifications page 3

	Requirement 7 (Seventh in the order of importance)	**Requirement 8**	**Requirement 9**
1. Core mission requirements of the system or product. Focus on requirements that are core to solving the engineering problem. These will reflect the solution to the problem.			
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?			
3. **Explain where this specification originated**: i.e., explain the value that you selected. I.e., *how* will meeting the specification given in point 2 above solve the problem?			
4. **How will you demonstrate at the examination** that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will *prove* these were met.			
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*			
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks l**ibrary functions** will be used (if relevant to the project).			

5. Real world field conditions

These are the ACTUAL, REAL WORLD CONDITIONS under which your project has to work *and will be demonstrated.*

	Real world field condition 1 (required to include)	**Real world field condition 2** (optional to include)	**Real world field condition 3** (optional to include)
Field condition requirement. In which real world conditions does the system have to operate?E.g., "lab conditions" or "outside in uncontrolled conditions". Describe the one, two or three most important field conditions.	The system is expected to perform in well-lit areas like a lab or office environment, at an operating range of 400-1000 lux, to ensure minimal noise in image capturing.	The system is expected to run on a stable, level and vibration free surface, to ensure the page scanning is accurate and the pen in the gantry can write precisely without fluctuations.	

6. Student tasks

6.1 Design and implementation tasks of the student

List your primary design and implementation tasks in bullet list format (5-10 bullets). These are *NOT* product requirements, but *YOUR* tasks.

- The student will research types of neural networks, their structure, the effect of neurons and layers, and how to extract features of text from different layers.
- The student will research image conditioning, noise removal, character separation strategies, and pen trajectory mapping.
- The student will design and implement the convolution neural network, image conditioning, and character separation algorithms.
- The student will simulate the classifier on existing handwritten text datasets with adjusted character separation strategies and neuron/layer numbers to meet requirements.
- The student will construct a rig to capture images of handwritten text on an A4 page, and implement the neural network and algorithms on a chosen single board computer.
- The student will test the new classifier implementations on the single board computer with the captured images to ensure requirements are met.
- The student will design the gantry system in CAD software and construct it with the necessary store-bought individual components.
- The student will design and implement the pen trajectory mapping algorithm and the stepper motor control.
- The student will test the constructed gantry with the implemented classifier to ensure the remainder of the requirements are met.

6.2 New knowledge to be acquired by the student

Describe what the theoretical foundation to the project is, and which *new* knowledge you will acquire (*beyond* that covered in any other undergraduate modules).

- The student will acquire new knowledge of training and classification of character images using a neural network.
- The student will acquire new knowledge involving the extraction of text features from neural network layers.
- The student will acquire new knowledge about the process of image processing with character isolation.
- The student will acquire new knowledge about pen trajectory map creation.
- The student will acquire new knowledge on 2D gantry design and implementation with precise motor control.
- The student will acquire new knowledge about the implementation of neural networks on a single board computer.

7. References

Format your reference list either in IEEE reference format or in APA reference format. Check what your study leader prefers and use consistently.

[1] V. Babushkin, H. Alsuradi, M. O. Al-Khalil, and M. Eid, “Analyzing handwriting legibility through hand kinematics,” Frontiers in Artificial Intelligence, vol. 8, Mar. 2025, doi: https://doi.org/10.3389/frai.2025.1426455.

[2] Sidar Agduk and Emrah Aydemir, “Classification of Handwritten Text Signatures by Person and Gender: A Comparative Study of Transfer Learning Methods,” Acta Informatica Pragensia, vol. 11, no. 3, pp. 324–347, Dec. 2022, doi: https://doi.org/10.18267/j.aip.197.

[3] O. Teikari, "Reproducing Handwriting with Position-Controlled Robots," Master's thesis, Dept. Pervasive Systems, Tampere University of Technology, Tampere, Finland, 2016.

[4] T. Huang and R. Xiong, "Cost-Effective Robotic Handwriting System with AI Integration," in Proc. 2024 IEEE Long Island Systems, Applications and Technology Conference (LISAT), 2024.

Part 3. Main Report

1. Literature study

1.1 Handwriting recognition and writer identification

1.1.1 Offline recognition and available datasets

The ability of a computer or system to identify written text after being trained on large amounts of data is known as HWR (Handwriting Recognition) or HTR (Handwritten Text Recognition) [1, p. 39]. One type of handwritten text recognition is offline recognition, which means the data being identified by the system is static and captured post-completion of the page of handwritten text. This is ideal for identifying the text from a fixed image of handwritten content by utilising image processing techniques and algorithms to identify the text that was written [2, p. 268]. The other type is online recognition, which is a more dynamic process that uses information recorded during the creation of the handwriting. The addition of a time variable includes data about the order of strokes as well as pen-up or pen-down movements [3, p. 18].

This project uses offline recognition because its inputs are photographed or scanned pages. The text and writer must therefore be identified from visible ink geometry. The absence of recorded stroke order also influences how the handwriting can later be reproduced: a path recovered from an image is a geometric estimate, rather than a recording of the writer's movements.

Multiple datasets have been used for training handwritten text recognition modules. The EMNIST (Extended MNIST) By_Class dataset consists of 814 255 handwritten characters and numbers, each being 28 by 28 pixels in size [1, p. 40]. There is only a single character per image, which suits isolated-character classification but does not itself supply examples of connected handwritten words. The IAM handwriting dataset provides complete handwritten English pages, lines and words with their transcriptions and writer metadata [4]. Hemanth et al. report the expanded database as 5 685 labelled sentences and 115 320 labelled words from 657 writers [5, p. 2460].

The dataset influences which variations the classifier encounters and which labels are available. For this project, line images and their transcripts support recognition without manually labelling every character boundary. Writer-labelled pages also support author classification and extraction of individual style profiles. IAM supplies public handwriting variation, while the personal dataset represents the project's photographed input conditions. A small isolated-character dataset alone would not address both requirements.

1.1.2 Image conditioning and text separation

The image conditioning of handwriting pages includes techniques performed on the handwritten text before it is passed into the classification and feature-recognition algorithms. This accounts for dimensional adjustments, removal of inconsistencies and normalisation of the data, to make the images more appropriate for text separation and classification [5, pp. 2458–2460]. Normalisation reduces the influence of unwanted variation; examples include min–max scaling and z-score normalisation [1, pp. 40–41].

Binarisation differentiates the background page from the ink of the written text, computed by pixel. A global threshold taken from the grey-level histogram uses one decision level for the image [6]. A local threshold instead uses statistics from a window around each pixel and can accommodate uneven illumination [7, pp. 2–3]. Deskewing corrects page rotation, while deslanting corrects the lean of individual characters by applying a horizontal shear. These are different transformations: aligning text lines does not require removing the writer's characteristic slant [8, 9].

For the camera input in this project, illumination correction and deskewing are needed before the ink is separated into lines. Slant must remain available as a writer-specific measurement; it can be removed for glyph storage and reapplied when the text is synthesised. The purpose is to suppress capture variation while retaining the information needed for reproduction.

Text separation is the process of breaking up a page of handwritten text into smaller segments or blocks that can be used in feature extraction and neural-network classifiers. Existing methods separate text into lines, words or individual characters. Hemanth et al. obtain lines and words through filtering, dilation and bounding boxes [5, pp. 2459–2460]. When lines are mostly straight and separated by blank rows, boundaries can be found from minima in the row ink profile. Where ascenders and descenders overlap, a boundary that bends between lines is more suitable. Minimum-cost seam methods provide a way to trace such curved boundaries [10, p. 3].

The project therefore separates whole lines before recognition. This avoids requiring a reliable cut between every pair of connected letters. Character boundaries are still needed to construct a glyph library, but there the known transcript and a trained line reader can constrain the extraction. Line segmentation and glyph extraction consequently solve different problems, even though both divide an image.

1.1.3 Neural networks and feature extraction

Many different types of neural-network models have been implemented for handwriting recognition, including artificial neural networks (ANNs), convolutional neural networks (CNNs), recurrent neural networks (RNNs) and combinations of them. ANNs consist of interconnected neurons, weights, hidden layers and biases that learn a mapping from the input to an output. CNNs use local connections and shared kernels to extract spatial patterns with fewer independent parameters than an otherwise comparable fully connected image layer [11, pp. 6–8]. RNNs maintain a state across a sequence, allowing a prediction to depend on neighbouring evidence [2, p. 269].

The convolutional layers of a CNN perform feature extraction by sliding a convolutional kernel over the image. Each local input value is multiplied by a corresponding kernel value, the products are summed, and a bias and activation function produce an output feature value. Reusing the kernel allows the same pattern to be detected at different locations [11, p. 6]. For unit stride and no padding, an N by N input with an f by f kernel returns an $(N-f+1)$ by $(N-f+1)$ feature map. Pooling then reduces spatial resolution and retains selected responses [11, pp. 6–7]. Figure 1 illustrates these operations.

The LeNet-5 model is an established example of this structure. It uses a 32×32 input and seven trainable layers, including convolution, subsampling and fully connected stages; its original output uses radial-basis units [11, pp. 7–8]. A deeper alternative is a residual network, whose

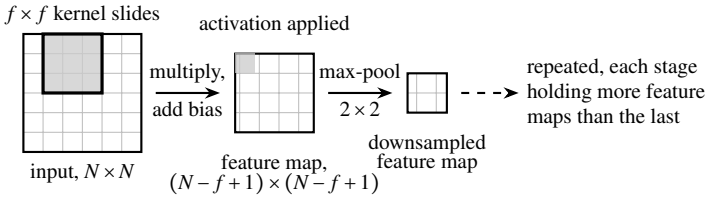


Figure 1. One convolutional layer and one max-pooling layer. The kernel slides over every position of the input, so an $N \times N$ input gives an $(N-f+1) \times (N-f+1)$ feature map, which pooling then downsamples [11, pp. 6–8]. This size expression assumes unit stride and no padding.

skip connections provide additional routes for information and gradients through the network [2, p. 270]. These examples show the choice between compact feature extraction and a more complex model; increasing depth does not guarantee an improvement for every dataset or processor.

A combination of CNNs and recurrent layers allows an entire line to be read without word or character separation. The convolutional layers convert the cropped line into feature columns, and a bidirectional recurrent layer reads the sequence in both directions, providing context from before and after each position [12, pp. 4–5]. In this offline setting, the sequence axis is the horizontal position in the image, rather than the measured time at which a pen moved. The full line crop is available before classification. Related convolutional–recurrent recognition systems use the same distinction between image features and sequence prediction [13, pp. 2–3].

Long sequences make simple recurrent networks difficult to train because gradients can diminish or grow through repeated updates. Long short-term memory (LSTM) uses an additive memory state and gates to control what is retained, replaced and exposed [3, pp. 4–5]. The convolutional feature columns usually outnumber the transcript characters, so their alignment is initially unknown. Connectionist temporal classification (CTC) adds a blank symbol and sums the probabilities of the paths consistent with the transcript. Repeated adjacent symbols are collapsed before blanks are removed [14, pp. 2–4]. Figure 2 shows the resulting recognition path.

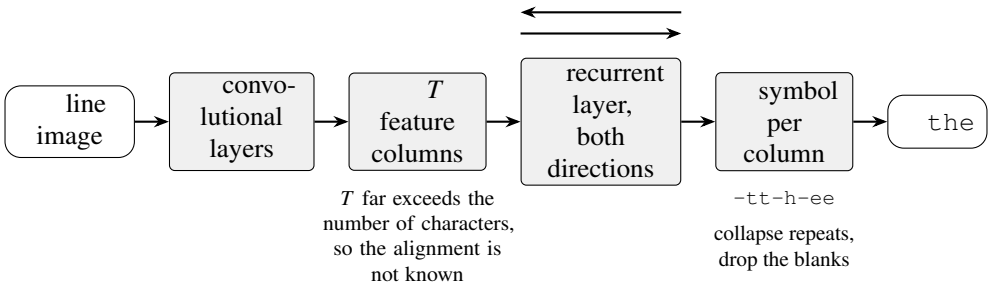


Figure 2. Reading a whole text line without cutting out characters, after the arrangement of Kizilirmak and Yanikoglu [12, pp. 4–5]. The convolutional layers produce T feature columns, a recurrent layer reads them in both directions, and a symbol including a blank is emitted per column; collapsing repeats and removing the blanks gives the transcription.

This combination is used for the project's reader because the training labels describe complete lines. The CNN extracts stroke patterns, the bidirectional LSTM retains their horizontal context, and CTC trains the network without supplied character positions. For author identification, the same convolutional architecture is useful, but its features must describe the writer across a line

rather than decode an ordered transcript. The two tasks therefore require different output heads and separately trained weights.

1.1.4 Training and assessment

The way networks are trained corresponds to the architecture they are built from. Batch normalisation subtracts the batch mean and divides by the standard deviation, with a stabilising constant and learned scale and offset [15, p. 4]. Dropout reduces overfitting by removing a random subset of a layer's outputs during training, limiting dependence on individual features [16, pp. 1931, 1934]. Data augmentation introduces small rotations, shears or smooth distortions while retaining the transcription [17]. These methods address different sources of variation and do not replace separate validation data.

Recognition is commonly assessed through character error rate (CER) or word error rate (WER), using insertions, deletions and substitutions relative to a reference transcript [14, p. 2]. An error in one character can spoil a whole word, but word edit error is not simply a count of words at matching positions. The metric and its reference population must remain explicit when comparing systems.

For this project, numerical forward passes, backward passes and optimiser updates make the learning calculations inspectable. A corresponding library implementation supports training and comparison, with stored arrays used for NumPy inference on the ODROID. The report distinguishes the selected library-trained weights from the independently trained numerical models. Character edits, writer decisions and physical drawing measurements are evaluated separately because they measure different functions.

1.1.5 Techniques for writer identification

There are multiple approaches to identifying the author of a written piece of text. One uses measured, text-independent features such as slant, height, stroke width, spacing and connectivity. Another describes the distribution of edge directions and recurring letter fragments, giving a texture and shape description of the writing [18, p. 701]. Alternatively, a neural network learns discriminative features from examples labelled by writer [19]. Such a classifier identifies a writer among its trained classes; it does not automatically identify an unfamiliar person.

This project needs an author classifier to select and assess the stored writer styles, while explicit geometry measurements are needed to reconstruct their appearance. Ink-trace width can itself contain writer information [20, p. 162], but the gantry uses its own pen. Stroke normalisation is therefore applied to reduce reliance on that cue and direct the classifier towards shape. Writer identity and stylistic fidelity remain separate measures: recognising an author does not establish that every letter or spacing feature has been reproduced accurately.

1.2 Handwriting synthesis in a writer's style

Before handwritten text can be reproduced through hardware, the desired text must be created in the selected author's style. Four approaches illustrate the trade-offs between source data, learned variation and the form of the output.

Recurrent stroke-level synthesis. A recurrent neural network predicts the next pen movement through small coordinate offsets and an end-of-stroke indicator. It models a distribution of possible movements because no letter is written identically every time [3, pp. 18–20]. Its output is naturally suited to a plotter, but training uses online recordings. This project's offline pages contain no measured stroke order, so this approach would require a different source dataset and an additional generative model.

Adversarial image synthesis. A generative adversarial network trains a generator and discriminator together: one generates handwriting, while the other helps distinguish generated samples from real examples. GANwriting also constrains content and writer style to produce text images [21, pp. 3–7]. Once trained, the generator produces an image in a forward pass. This is a relevant alternative for learning contextual letterforms, but a raster output still needs conversion into the pen paths required by this project.

Diffusion-based synthesis. This method learns to reverse a noise-corruption process and generates handwriting through successive denoising steps. One-DM uses one reference style image at generation time, while its training uses a much larger handwriting corpus [22, pp. 3–5, 9]. A small reference requirement must therefore be distinguished from the data and computation required to train the model. For this project, diffusion would add both a separate training task and image-to-path recovery, rather than directly supplying the motion representation.

Glyph-library synthesis. This method assembles words from shapes or glyphs extracted from existing handwritten pages. Haines et al. store labelled forms and select and join them using information about their shape, context and spacing [23, pp. 4–8]. Their standard data collection uses approximately four A4 pages per writer [23, p. 4]. This suits the project's small personal collections and allows explicit control of size, slant and spacing. The main limitation is that a finite collection of character shapes and joining rules only approximates the way connected handwriting changes with its neighbours.

Generated handwriting can be assessed through geometric comparisons, recognition and human judgement. Haines et al. include human evaluation, while image-generation work also uses text and writer recognition to assess the output [23, pp. 12–14], [21, pp. 8–9]. For the present robot, model scores help choose a candidate before drawing, but final physical output also needs its own read-back and geometric assessment. A classifier used to select the candidate is not an independent human judgement of its style.

1.3 Physical reproduction and pen-path mapping

There are three relevant families of mechanism for reproducing text on paper: Cartesian systems, serial-chain robots and parallel-chain robots. A Cartesian system consists of a gantry with independent x–y movement. Serial-chain robots consist of linkages connected in a chain, like a typical robot arm. Parallel mechanisms use several linked chains to support the moving platform [24, pp. 5–8]. These arrangements differ in their workspace, mechanical structure and coordinate transformations.

A Cartesian gantry is suitable here because the page is flat and the main motion requirement is two-dimensional. Each axis can be commanded and calibrated separately, with a further pen-lift mechanism for travel. This makes the conversion from an image-derived path to physical

coordinates direct, while leaving frame alignment, transmission compliance and missed steps as practical accuracy concerns.

A pen path is stored as a list of points, and a recovered stroke usually has more points than a plotter needs. Polyline simplification reduces this list while retaining the stroke's shape. The Douglas–Peucker method joins the endpoints, finds the point furthest from that line and retains it when its distance exceeds a chosen tolerance, then repeats on the resulting parts [25]. A small tolerance follows the stroke closely; a large tolerance removes more points but can cut corners. When handwriting exists only as an image, the path must first be recovered, for example by thinning the ink to a centreline [26, p. 237].

These operations are needed to turn the project's stored glyphs into manageable G-code trajectories. Simplification controls geometric detail, while the thinning and tracing determine which strokes are available to the synthesiser. Neither recovers the original writing speed or pressure from a photograph, so the controller must define its own drawing feed and pen transitions.

There are multiple methods of linear actuation that could be considered for a Cartesian plotting system, including belt drives, rack-and-pinion drives and screw drives [27]. The project uses timing belts and pulleys to convert motor rotation into movement along supported rails. This choice prioritises a straightforward, affordable page-sized mechanism with sufficient travel speed; its nominal step resolution must still be distinguished from measured positioning accuracy.

1.4 Application of the literature

The literature supports a system built around offline pages, complete-line recognition and explicit glyph geometry. Image conditioning and curved line separation prepare photographed input; convolutional features support separate text and writer outputs; transcript-guided extraction turns observed handwriting into reusable paths. A glyph library then supplies controllable geometry without training an additional handwriting generator. The Cartesian gantry provides the physical output stage. Section 2 explains how these choices were combined and developed, while Section 3 gives their calculations, implementation and verification.

2. Approach

The project was approached as two interacting systems: one interprets photographed handwriting and constructs writer-specific paths, while the other converts those paths into physical pen movement. The proposal's functional analysis provides their responsibilities; the G-code interface allows their implementation and verification to proceed separately.

2.1 Mechanical Hardware Design

Before any computer vision or neural networks were worked on, the hardware to perform the pen plotting was created. This is because the hardware does not rely on any of the coding elements to be complete to be able to be tested, and hardware can experience major delays in production due to order delays, stock shortages or component failure, adding to the reason for it to be constructed first. A Cartesian gantry with two independent axes was chosen over some sort of robotic writing arm, as system requiring only flat, cartesian movement does not need joint-dependent kinematics, and each axis can be calibrated separately and work indepedently from one another. To achieve the linear motion required, GT2 timing belts and pulleys were chosen over a lead-screws due to their lower cost, faster travel and ease of customizstion.

Two NEMA 17 stepper motors were chosen for the X and Y axes, driven by A4988 driver chips. Because a stepper motor always rotates by precisely the number of commanded steps at exact intervals, accuracy could be assured without an encoder. To allow for even more increased accuracy and minimal friction, linear bearings were used within the mechanics of the system to allow parts to move with ease and precision, instead of just having the printed supports sliding over the steel rails of the gantry. The use of microswitches on the axes allowed the system to be able to calibrate to reproduce text more accurately. The gantry was designed to be able to reproduce any G-code file that was provided, allowing for easy integration with the other subsystems in future.

2.2 Page condition and line segmentation

In parallel while working on the gantry and waiting for hardware to be delivered, work was done on the first processing stage of the software pipeline, which was performing image conditioning and segmentation of those pages into a format the future classifiers would accept. A photograph produces a wide range of inconsistencies that need to be dealt with, including uneven lighting, page rotation, printed rules, and strokes from neighbouring lines merging into each other. The image conditioning part of the code was designed to remove these undesired page inconsistencies without ruining the quality of the original writing, using methods like lighting normalization, ink separation, and skew correction.

Segmentation of the page of text was chosen to be done lines at a time instead of per word or character, as it was found that dynamic word separation that worked in all cases was difficult, and line separation has the added advantage of more context to train and classify text. The design chosen to perform the segmentation involves locating line centres first, finds curved boundaries between the lines and its ascenders and descenders, and only then uses the geometry of each ink piece to decide which line actually owns it. Deskewing, removal of image background, and artefact rejection were deliberately ordered to run before line centres are estimated, while

recovery of faint ink was left until after ownership of each piece had already been decided, since doing it earlier risked reattaching noise to the wrong line. The system fomatted each page into the ideal crop sizes to be passed into the text and author classifiers.

2.3 Text and writer classification

Once clean line crops were available, the next stage had two jobs: reading the text and identifying who wrote it. Both depend on the same stroke features, so a single convolutional backbone was built, but then the writer and author classifiers varied after that point. The text classifier uses a CNN–BiLSTM architecture with decoded CTC output, which was heavily based off existing research, and chosen because it reads a whole line without needing individual characters pre-cut, as mentioned in the segmentation section, and cursive text has no reliable character boundary to cut at in the first place.

Training data came from a mix of public handwriting (IAM) and the project's own photographed pages, and both author and text models were trained using hold-out pages or lines to allow the system to be validated against, to prove over-fitting did not occur. The writer network's input was stroke-normalised rather than using varying ink thickness from real pages, specifically because the gantry's own pen produces constant stroke thicknesses, regardless of the author.

2.4 Style extraction and handwriting reproduction

Once the previous subsystems were created, the actual text synthesis and reproduction was made. Four reproduction methods were considered, and the method chosen instead was a stored set of normalised glyph trajectories. The other were not chosen because an online dataset of pen strokes was not available, and the chosen IAM dataset was not extensive enough to train a rpeproduction model, neither was the necessary GPU processing power available for it. Using the glyph trajectory method simply resused data and models the project already had, but at the cost of weaker handling of how letters join and vary with context compared to a true generative model especially for heavy cursive styles. The new text to be created is then assembled from the stored glyphs rather than generated from scratch, giving direct, inspectable control over scale and spacing before anything is sent to the gantry. Extracted features like slant, spacing etc. are also applied to the synthesised text.

2.5 Integration and testing

The integration stage formed a complete pipeline of the system through a web server, with G-code remaining the constant fixed interface between the software reproduction output and the gantry hardware. The web interface was designed to be able to preview and inspect generated G-cocdes before any gantry operation was called. By previewing the gantry before reproduction, the precision of the gantry could be measured, and any errors with gantry movement isolated. Safety features were also included as part of the integration, which included a Emergency abort feature on the web server to manually stop any gantry command, as well as end-stop detection with switches to ensure the motors did not stall if the end of the rail was ever reached.

3. Design and implementation

The Cartesian plotter constrains the workspace, pen state and motion required from the software. The design therefore starts with hardware, then follows page segmentation, text/writer classification and handwriting reproduction, before describing their integration.

Numerical substitutions use the configured mechanism and recorded processing of stored handwriting, with inputs and scope collected in Appendix B.7. Borrowed methods are referenced where introduced; project-specific transforms and heuristics are identified as design choices. These traces show how the calculations operate; physical qualification remains in Results.

3.1 Design summary

This section summarises the project design tasks and how they were implemented (see Table 1). The tasks follow the deliverables and student design work in the approved Project Proposal. Completion refers to the implementation of a task; compliance with a numerical system specification is established separately by the qualification tests in the Results section.

Deliverable or task	**Implementation**	**Completion and report location**
Investigation of methods	Compared recognition, conditioning and reproduction approaches.	Completed; Literature Review; Sections 3.5, 3.6.
Page capture and image conditioning	Camera arrangement, illumination normalisation, ink extraction and orientation correction.	Completed; Sections 3.3.1 and 3.4.
Separation of handwritten text	Line detection, curved boundaries and ink ownership; later transcript-constrained glyph localisation.	Completed; Sections 3.4, 3.6.2.
Text and writer classifiers	Convolutional features, sequence recognition and writer identification.	Completed; Section 3.5.
Network training implementations	NumPy calculations and framework training; selected weights are framework-trained.	Completed; Sections 3.5.5–3.5.7.
Writer-specific trajectory maps and storage	Aligned glyph extraction, skeleton tracing, variant filtering and persistent style profiles.	Completed; Sections 3.6.2–3.6.5.
Handwriting synthesis and G-code	Text layout, glyph assembly, candidate selection and conversion to machine coordinates.	Completed; Sections 3.6.6–3.6.8.
Gantry, pen mechanism and motion control	Custom mechanical assembly, two-axis drive, pen switching, end stops and calibration.	Completed; Section 3.3.
Processing and motion integration	Server connects capture, processing, stored profiles and execution.	Completed; Section 3.7.
Subsystem verification	Numerical checks and offline image/path experiments; formal integrated qualification is reported in Results.	Completed; Sections 3.5.7, 3.4.7; Appendix B.

Table 1. Design summary.

3.2 System block diagram

The system is split into a main hardware and software block diagram. Figure 3 shows the hardware connections for page capture, handwriting interpretation and physical reproduction. The system is not required to have an internal power supply, so the ODROID N2+ and the stepper motor are connected to an external supply. A USB webcam held with a page clamp is connected to the ODROID N2+ USB port, to capture page scans. An external device is used as a display interface and to connect through SSH to the ODROID N2+. Two NEMA 17 stepper motors control the x and y axis movement, driver by A4988 stepper motor drivers. GT2 timing belts and pulleys convert the stepper motor rotations to Cartesian movement, moving the pen across the page. Lastly a BC337 transistor circuit is used to drive a N20 gear motor to lift the pen up/down on the page.

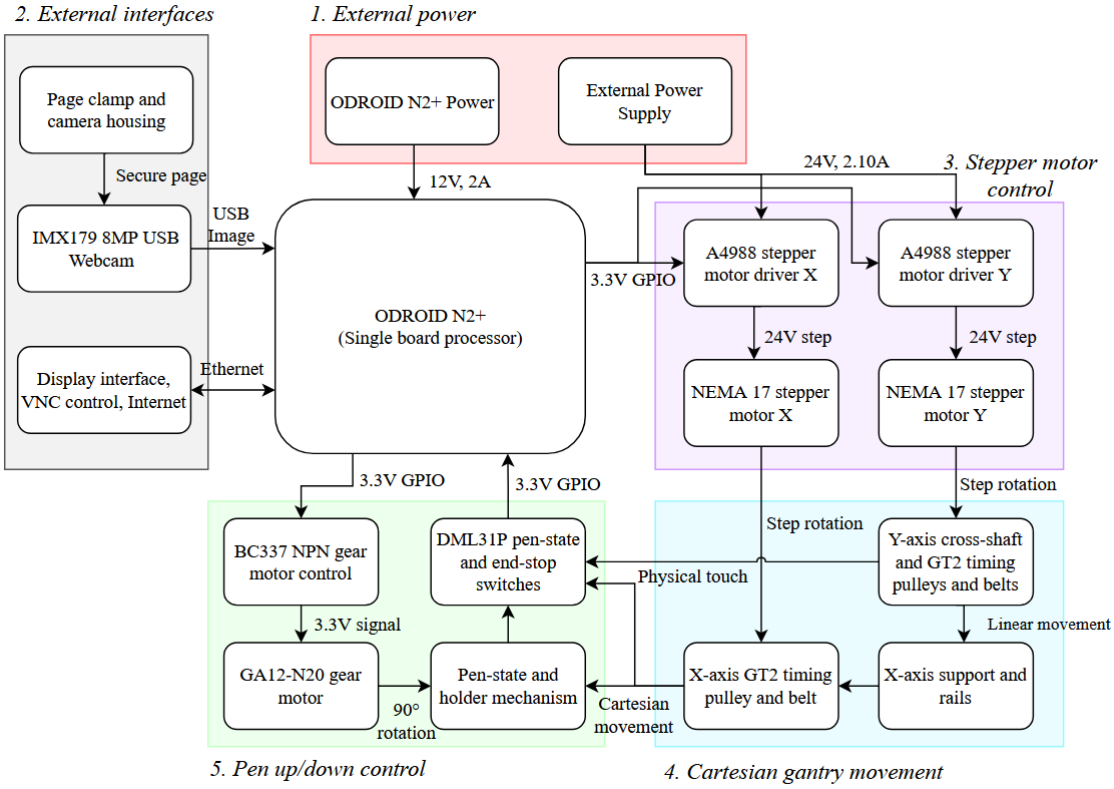


Figure 3. Hardware system block diagram supplied by the author. The detailed controller pins, signal levels and power connections are listed in Table 19.

Figure 4 shows the corresponding software pipeline. When classifying or training with data, the page scan must be conditioned first, through normalization, ink separation etc. and the segmenter splits the page into clean line crops, to be passed into the training or classification models. If the system is prompted to train its models, the cropped lines will either be run with the text model trainer, or gets stroke-normalized to train the author model, and glyph extraction occurs. If classification is occuring, the cropped line is simply passed into the text classifier straight or gets stroke-normalized and passed into the author classifier. Lastly if the system is prompted to reproduced text, the system must first perform the sentence synthesis and G-code generation so the gantry has something to write. The pen feedback loop and emergency stop sequence are

triggered depending on the state of the micro-switches used. The end-stop switches also control the calibration sequence of the gantry.

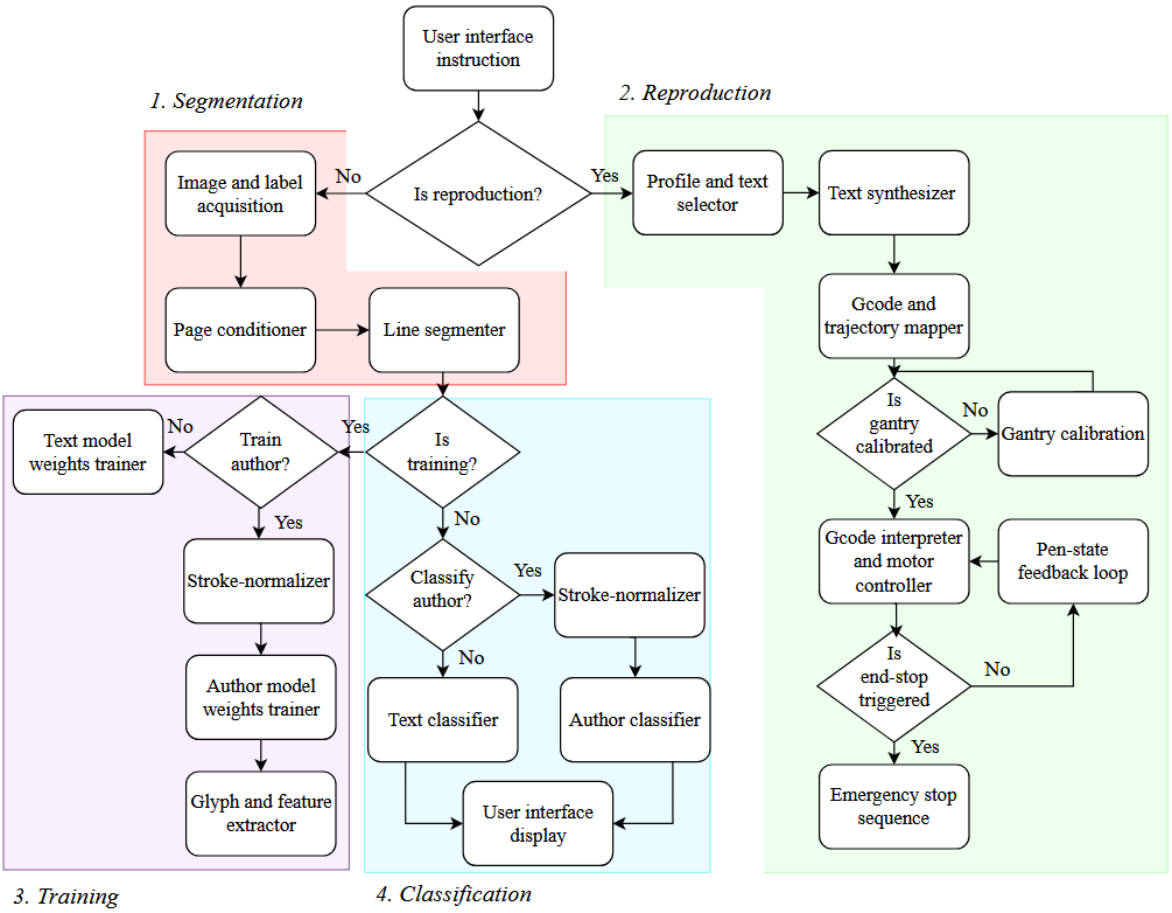


Figure 4. Software system block diagram supplied by the author: (1) segmentation, (2) reproduction, (3) training and (4) classification.

3.3 Hardware Design

3.3.1 Plotter designs and mechanics

The chosen method of text plotting is a Cartesian gantry system with 2 independent axes of movement with a pen up/down mechanicsm. Complex kinematics equations and error dependencies are avoiding by using 2 isolated x and y movement methods for reproduction [24]. Linear GT2 timing belts and pulleys are used to convert rotary motion from stepper motors and axles to the desired cartesian movements. Timing belts are used for their reliability, cost effectiveness, and higher speed travel [27].

A Cartesian gantry consists of a stationary rail that controls movement in one axis, and a movable rail that controls the other axis of movement. For this system and future explanations, the stationary rails control y-axis movement, while the x-axis rail moves along the y-axis. The following diagram explains axes of the gantry:

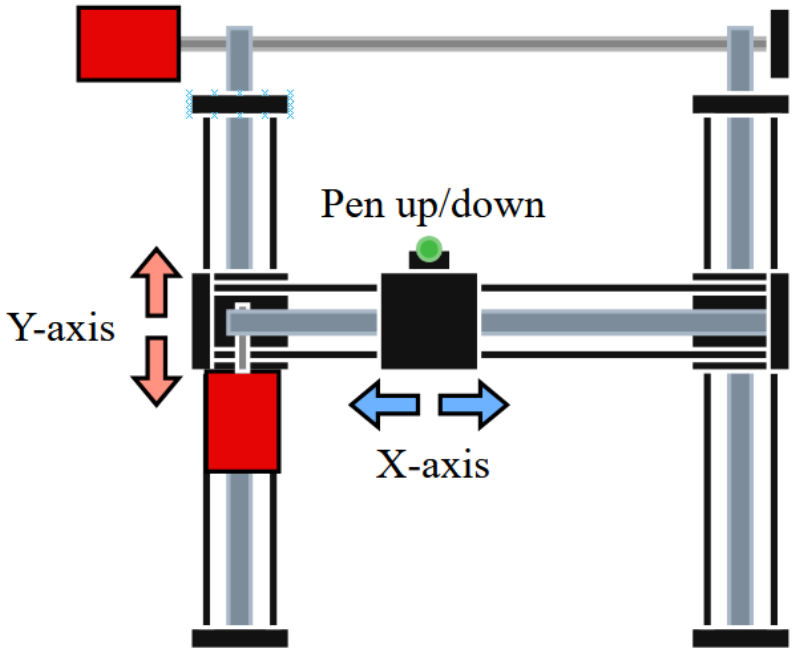


Figure 5. Cartesian gantry axes and pen up/down mechanism.

The x-axis rail needs to move with stability and consistency, which is why it is supported on both ends by dual-rails on the y-axis. A singular stepper motor was chosen to drive both pulleys on both y-axis rails to ensure synchronicity and save costs. 2 independent stepper motors driving one axis creates synchronicity issues because any difference in timing, missed steps or load torques could offset the movement between the two motors. The challenge with using one motor to drive two timing pulleys, is finding a method to extend the shaft of the motor to connect to both timing pulleys without overloading the motor and applying too much tortion or twist to the new axle. A rigid coupler was used to extend the axle of the stepper motor to a chromed steel rod.

To reduce strain on motors, and allow for increased accuracy and precision, linear bearing were fitted around the chromed rod rails to allow all parts to slide with very little friction. To verify the stepper motor could handle the weight of the x-axis rail further, the torque required to move the rail at the desired speed was calculated. It is described in Section 3.3.2 with the descriptions of the stepper motor type chosen and its product specifications.

The camera support holds the USB camera over the restrained page so that the image-processing stage receives a consistent view of the writing area. The IMX179 autofocus webcam and page restraint connect to the hardware arrangement shown in the system block diagram.

3.3.2 Stepper motor and driver

A stepper motor does not continuously turn with a PWM input, but rather turns at specific fixed angles each time there is a switch in the windings (the coils of the stepper motor). The A4988 regulates the winding current through a process called chopping, which switches the current supply on the winding until the current rises to a certain point, and then switches off at the target

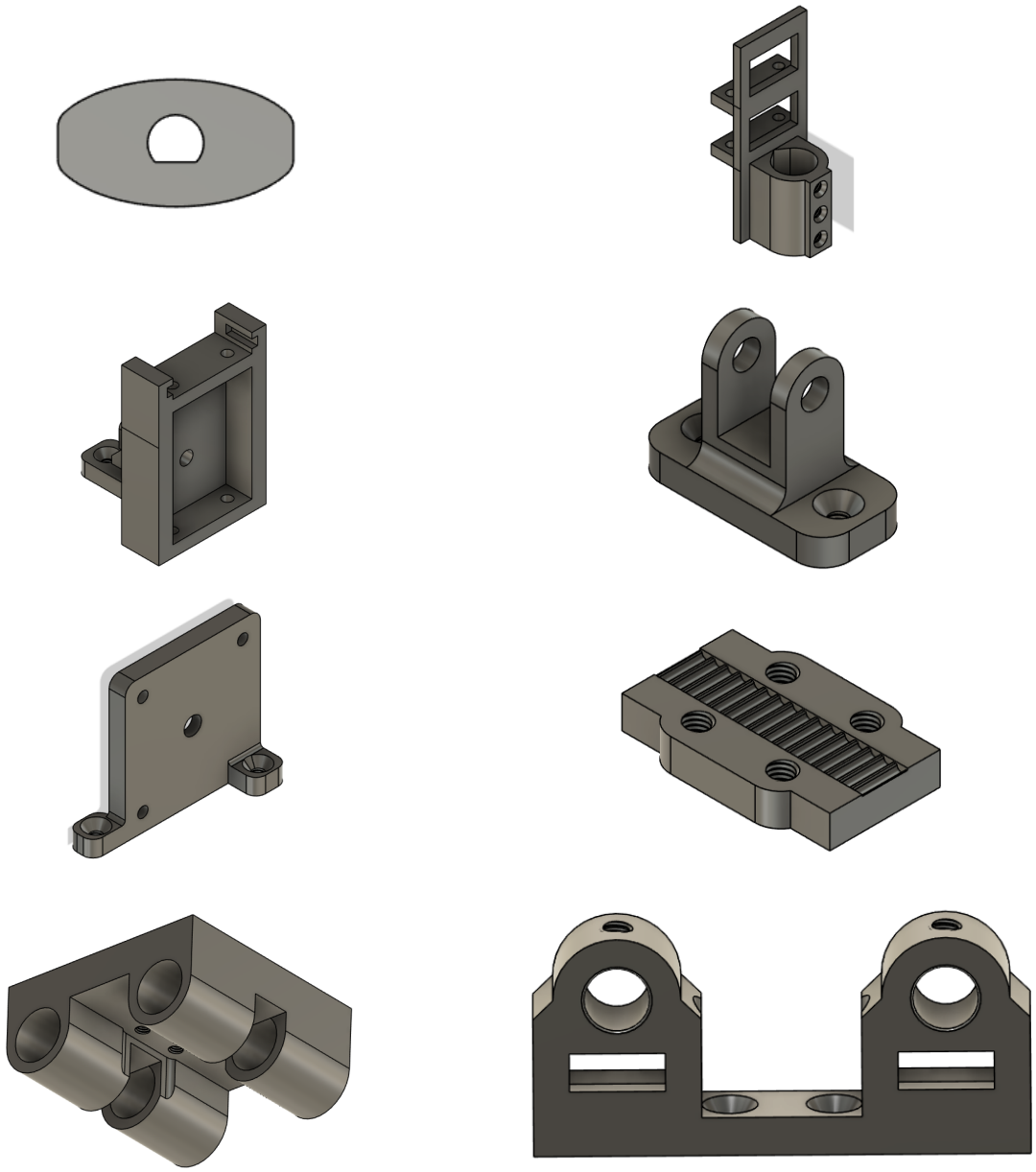


Figure 6. CAD models of the gantry components.

current. This occurs at high frequencies and effectively keeps the current going to the coils within a specific range so as to not overshoot, even when the motor is not actively stepping. A step is created by slighty adjusting the current value between 2 coils just enough to allow the rotors teeth to make a small jump or 'step'. The stepper motor is perfect for the precise movements required for handwriting reproduction with the gantry, as it produces steps at exact, known intervals, with position tracked from commanded steps rather than an encoder. This assumes that the motors do not miss steps.

Two stepper motors are used in the gantry plotter, one for x and one for the y-axis direction. The type of stepper motor chosen was the 42BYGH40-01 Nema 17 bipolar stepper motor, with a resolution of 200 full steps per revolution and a current rating of 1.68A per phase [**?**]. The following calculations were done to determine if the Nema 17 stepper motor would be strong enough for the use case of the gantry plotter, and if a singular motor could control both y-axis belt drives safely.

Dziedzic et al. examined samples written at 10–60 mm/s [**?**]. The original sizing check uses the upper value, rounded from 3600 to 4000 mm/min for headroom; this is a design benchmark, not the selected operating feed. Current generated jobs use 900 mm/min for drawing and 2400 mm/min for travel.

For the installed 20-tooth GT2 pulleys with 2 mm belt pitch, the transmission geometry gives [28]

$$\ell_{\text{rev}} = N_t p = 20(2) = 40 \text{ mm/rev}, \qquad r_p = \frac{N_t p}{2\pi} = 6.37 \text{ mm}. \tag{1}$$

Speed-to-torque margin check. The design linear feed is converted to a motor speed and compared with the torque required to move the assembly. The earlier curve readings are retained as a conditional calculation: their applicability to the installed motor and driver settings has not yet been verified.

$$n = \frac{v}{\ell_{\text{rev}}} = \frac{4000 \text{ mm/min}}{40 \text{ mm/rev}} = 100 \text{ RPM} \tag{2}$$

where n is the motor speed in revolutions per minute, $v = 4000$ mm/min is the sizing feed (60 mm/s rounded up for margin), and $\ell_{\text{rev}} = 40$ mm is the belt travel per revolution already established in Equation 1 from the GT2 pulley's pitch ($p = 2$ mm, $N_t = 20$ teeth).

$$T_{\text{po}}(n) = T_1 + \frac{n - n_1}{n_2 - n_1}\left(T_2 - T_1\right) = 7 + \frac{100 - 60}{120 - 60}(21 - 7) \approx 16.3 \text{ Ncm} \tag{3}$$

where $T_{\text{po}}(n)$ is the conditional pull-out torque estimate at speed n, found by linear interpolation between the original two curve readings: $(n_1, T_1) = (60 \text{ RPM}, 7 \text{ Ncm})$ and $(n_2, T_2) = (120 \text{ RPM}, 21 \text{ Ncm})$.

$$T_{\text{req}} = r F_f = r \mu m g = 0.006366 \times 0.004 \times 0.950 \times 9.81 \approx 0.0237 \text{ Ncm} \tag{4}$$

where T_{req} is the torque needed at constant velocity, $r = 6.37$ mm is the pulley pitch radius from Equation 1, $\mu = 0.004$ is the original assumed bearing-friction coefficient, $m = 0.950$ kg is the complete moving assembly and $g = 9.81$ m/s^2. The bearings support the weight, so the motor overcomes horizontal resistance rather than lifting the assembly. The relations $F_f = \mu m g$ and $T = r F_f$ give this steady-speed estimate [28].

$$\frac{T_{\text{po}}}{T_{\text{req}}} = \frac{16.33}{0.02373} \approx 688 \tag{5}$$

Under the assumed friction and original curve values, the estimated torque ratio is about 688. This retains the sizing argument for one motor driving both sides through a shared shaft. It is not a measured operating margin and does not include acceleration, preload or misalignment.

Figure 7. NEMA 17 stepper motor (left) and A4988 stepper motor driver (right).

To power the two stepper motors, two A4988 stepper motor drivers are used.

3.3.3 Step resolution, microstepping and calibration

Three properties of a stepper drive fix the resolution of this machine. The motors used here take 200 *full steps* per revolution, so one full step is 1.8°; because position is simply the number of steps commanded, no encoder reports where the carriage actually is, which is what *open-loop* control means, and the design simply trusts the step count. That trust is the assumption on which requirement R5 rests, and it fails silently if a step is ever missed. The driver subdivides each full step by *microstepping*: rather than switching the two windings fully on or off, it holds them at intermediate current ratios, placing the rotor at intermediate angles. At the 1/16 setting used here, one full step becomes 16 microsteps, so a revolution is 3200 microsteps; microstepping buys resolution and smoother motion but not proportionally more accuracy, because the intermediate positions are held by the ratio of two winding currents and are therefore more sensitive to load torque than a full step is.

Equation 6 gives the step resolution from 200 full steps [**?**], 16 microsteps [29, p. 6] and $\ell_{\text{rev}} = 40\,\text{mm}$,

$$\kappa = \frac{200 \cdot 16}{\ell_{\text{rev}}} = \frac{3200}{40} = 80\ \text{steps/mm}, \qquad \Delta = \frac{1}{\kappa} = 12.5\ \mu\text{m}, \tag{6}$$

which calibration replaces by each axis's measured value. Targets are *absolute* step counts, $\text{round}\big((x+5)\kappa\big)$ with a 5 mm offset, differenced against the tracked position, so rounding cannot accumulate. Quantisation error is at most $\Delta/2 = 6.25\ \mu$m per axis, 8.8 μm combined, 57 times finer than the 0.5 mm of R5 — but that is the *digital* resolution only, and belt stretch, backlash, missed steps, switch repeatability and pen compliance were not measured.

Coil current and its derating. The datasheet relation [29, p. 9] between the trimmer reference, the carrier's two sense resistors and the regulated current gives

$$I_{\text{coil}} = \frac{V_{\text{ref}}}{8\,R_S} \quad \Longrightarrow \quad V_{\text{ref}} = 8\,R_S\,I_{\text{coil}} = 8 \cdot 0.1 \cdot 1.68 = 1.34\ \text{V} \tag{7}$$

for the 1.68 A motor rating [**?**], conditional on the reported R10 sense-resistor marking meaning 0.1 Ω. This gives a rating-based ceiling, rather than the measured adjustment. For comparison, reducing the phase-current limit to 1 A would give $V_{\text{ref}} = 8(0.1)(1) = 0.8$ V. Carrier cooling can require a lower current, but the actual fitted V_{ref} and thermal margin were not recorded.

3.3.4 Pen up/down mechanism

The pen-lift mechanism itself is a design choice justified against the alternative of a third stepper or a servo motor. A servo motor expects a continuous pulse-width-modulated position command and typically needs its own timer peripheral or a precisely timed software loop running concurrently with the gantry's own step timing, competing for the same processor cycles that generate STEP pulses. An N20 geared d.c. motor run in one direction only, with a cam that toggles the pen up or down every 90° of rotation, needs only an on/off drive pulse of roughly known duration and a microswitch to confirm the result; it is cheap, widely available, has high torque at low speed through its internal gearbox for its size, and has a smaller profile than a comparable servo, at the cost of giving no absolute position, only a toggle.

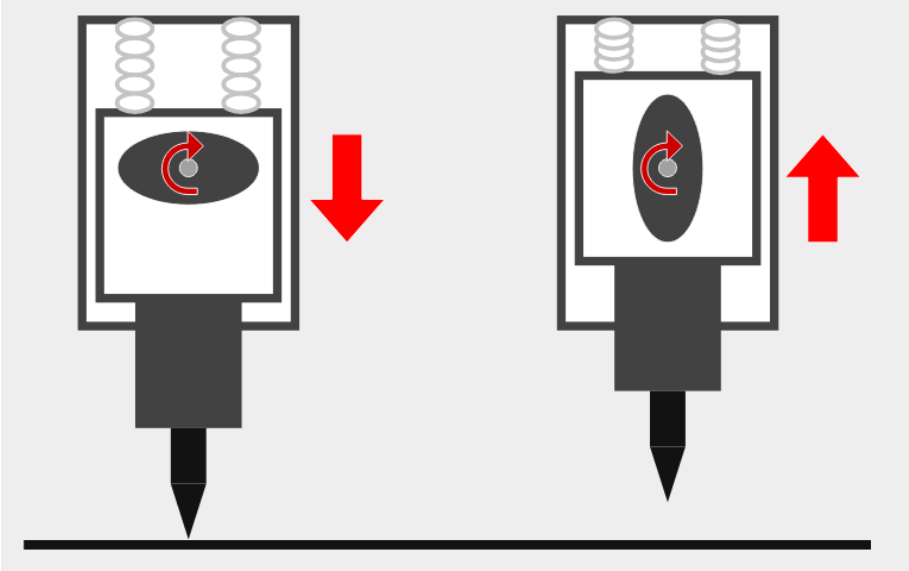


Figure 8. Cam-operated pen-lift mechanism in the down (left) and up (right) positions.

The implemented lift, planned in the proposal as a stepper, gives only pulses, so the pen state must be tracked in software, and the pen switch is feedback only (high = up). The pen-drive routine of Algorithm 1 must therefore test the switch *before* energising the motor, since the cam turns one way only and an unnecessary pulse would toggle the pen to the wrong state rather than leave it alone. The controller runs the motor until the requested switch state is confirmed or a timeout occurs, with a 40 ms overrun after pen-up confirmation. The generator's separate settling dwell is 350 ms; this is not a measured fixed motor-rotation time.

Pen motor driver and base-resistor selection. The GA12-N20 geared motor was selected to turn the pen cam with a compact, low-speed geared output. The listed 3 V, 150 rpm variant has a specified voltage range of 1.5–5 V, a no-load current of 10 mA, a rated current of 30 mA, and a stall current of 230 mA; these are supplier figures for that variant, rather than measured currents under the installed cam load [**?**]. The supplied circuit uses the Odroid's 3.3 V GPIO signal to drive the base of a BC337 NPN transistor. The transistor is a low-side switch: motor current flows from the 3.3 V DC motor rail, through the motor and collector-emitter path to the shared ground, while the GPIO supplies only base current [30]. The source symbol in the supplied diagram should therefore be read as a DC supply, not an AC source.

Assuming a base-emitter voltage of approximately 0.7 V, the 1 kΩ base resistor limits the nominal GPIO current to

$$I_B = \frac{V_{\text{GPIO}} - V_{BE}}{R_B} = \frac{3.3 - 0.7}{1000} \approx 2.6\,\text{mA}. \tag{8}$$

This keeps the GPIO current far below the motor current, but the GPIO must still be rated to source at least 2.6 mA. The BC337 datasheet's saturation test uses 50 mA of base current for 500 mA of collector current, a conservative forced-current-gain ratio of 10 [31]. Using that ratio as a design check, 2.6 mA of base current supports about 26 mA of saturated collector current. That is slightly below the listed 30 mA rated motor current. To provide the same conservative saturation drive at 30 mA would require $R_B \leq (3.3 - 0.7)/(0.030/10) \approx 870\,\Omega$; at the listed 230 mA stall current it would require $R_B \leq (3.3 - 0.7)/(0.230/10) \approx 110\,\Omega$. The latter would draw about 23 mA from the GPIO, so the 1 kΩ circuit is not guaranteed to saturate the transistor at rated or stall load. The 1 kΩ resistor is the prototype's GPIO-current-limiting choice. A suitable logic-level MOSFET or motor driver would avoid relying on this base-current margin if the installed motor approaches those loads.

The supplied circuit image does not show a flyback diode. A diode should be connected across the motor, reverse-biased during normal operation, with its cathode toward the positive motor rail and anode at the transistor collector, to provide a path for the inductive current when the transistor switches off [32].

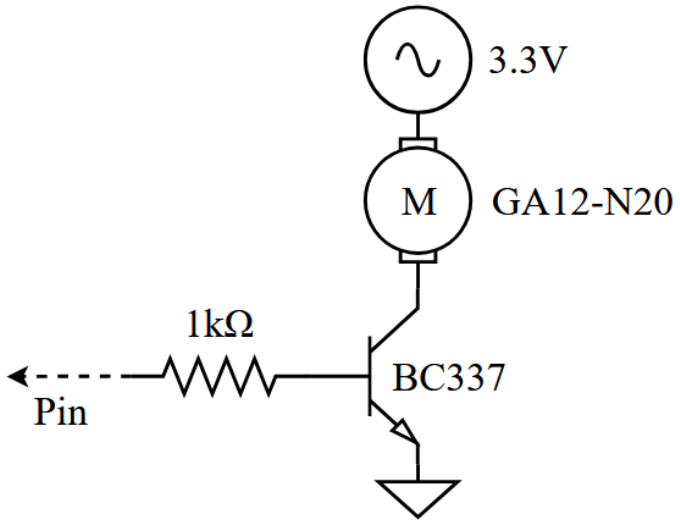


Figure 9. Supplied low-side BC337 switch circuit for the GA12-N20 pen-lift motor.

Algorithm 1 Pen toggle

Require: desired state wantUp
if any end-stop is currently triggered **then**
  refuse the request
else if the pen switch already reads wantUp **then**
  return {the motor must *not* run; the cam turns one way only}
else
  energise the pen motor
  while before the deadline and the pen switch does not yet read wantUp **do**
    wait
  end while
  if the switch now reads up **then**
    wait 40 ms more to seat the cam
  end if
  de-energise the pen motor
end if

During tuning the clicker sometimes over-rotated and the pen did not push down fully before the switch read as triggered, requiring the switch seating and timing to be adjusted experimentally. Differing pen pressure on a book or an exam pad also introduced inconsistent ink, so a rigid clipboard with a single page clamped flat was adopted. The 0.35 s settling dwell contributes to the predicted writing time, but is separate from switch-confirmed actuation. Missed steps cannot be detected from the step counter, so physical accuracy must be assessed from the drawn output rather than nominal resolution alone.

3.3.5 End stop switches and calibration

All four axis end-stop switches are checked before every major-axis step and every G-code line. A potentially triggered switch is accepted only after three consecutive high reads 6 ms apart (Algorithm 2). The pen switch is read separately by the pen-control routine. The confirmation interval is

$$t_{\text{confirm}} = (3-1)(6\,\text{ms}) = 12\,\text{ms}. \tag{9}$$

The stepping loop does not continue pulsing during this check. The interval is therefore not a measured trip-to-stop latency and cannot simply be multiplied by the requested feed to infer stopping distance.

The DML31P was selected as a compact, momentary lever microswitch for detecting gantry end-stop contact; the product listing identifies it as an SPDT switch rated for 1 A at 125 V AC [**?**]. In this circuit it is used only as a low-current logic contact, not to switch a motor. The diagram uses the common and normally-open contacts so that actuating the switch connects the Odroid N2+'s 3.3 V logic rail to the GPIO input; the source symbol represents DC. The 10 kΩ resistor is a pull-down: it holds the input low while the switch is open, preventing a floating input, and draws a small current when the switch closes. The N2+ GPIO uses 3.3 V logic [30].

With the switch closed, the pull-down current and resistor dissipation are

$$I_{\text{closed}} = \frac{V_{\text{rail}}}{R_{\text{PD}}} = \frac{3.3\,\text{V}}{10\,\text{k}\Omega} = 0.33\,\text{mA}, \qquad P_{\text{PD}} = \frac{V_{\text{rail}}^2}{R_{\text{PD}}} = \frac{(3.3\,\text{V})^2}{10\,\text{k}\Omega} \approx 1.1\,\text{mW}. \tag{10}$$

The 0.33 mA flows from the 3.3 V rail through the switch and resistor to ground; only the GPIO input leakage flows into the pin. When the switch opens, the resistor pulls the input to ground; when it closes, the input rises to approximately 3.3 V. This provides defined logic levels while keeping the closed-contact current low. The GPIO must be configured as an input before this circuit is connected and kept in input mode during operation. If the pin were configured as an output-low, closing the switch would connect the 3.3 V rail directly to the GPIO output driver; the $10 k\Omega$ pull-down does not limit that fault current.

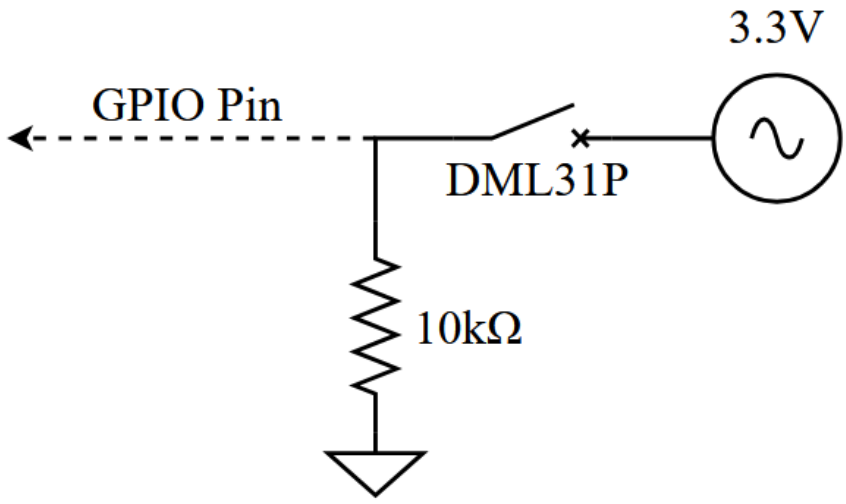


Figure 10. DML31P end-stop switch with a 10 kΩ pull-down connected to an Odroid N2+ GPIO input.

Algorithm 2 Debounced end-stop read, called before every major-axis pulse

Require: set of end-stops to ignore (already triggered on this axis)
1: **for** each end-stop not in the ignore set **do**
2: **if** it does not read closed **then**
3: **continue** to the next end-stop
4: **end if**
5: re-read it twice more, 6 ms apart {three reads in total, debounce of Equation 9}
6: **if** it read closed on all three reads **then**
7: **return** that end-stop as triggered
8: **end if**
9: **end for**
10: **return** none triggered

On a trip the program latches a stop, sets both STEP pins low, retracts off every triggered switch and aborts the job; targets are clamped to the calibrated area and the latch clears only by explicit command, refused while a switch is still triggered. Protection is *software only*: no hardware stop that cuts motor power is documented, the switches are polled only during a job, the web stop request can interrupt a running job, but it is not an independent hardware power cut.

Equation 6 rests on assumed transmission values and the position after power-up is unknown, so each session begins with a direction-agnostic calibration, written so that it never needs to know

which direction leads to which switch. It drives one way at 1.6 ms per step until *either* switch of that axis triggers, reverses, and counts the steps N to the *other*. With the hand-measured travel L, the resolution, the initial step counter and the position in millimetres are

$$\kappa_{\text{axis}} = \frac{N}{L}, \qquad n_0 = N \text{ if the second switch was MAX else } 0, \qquad x_{\text{mm}} = \frac{n}{\kappa} - 5, \tag{11}$$

so the zero of Equation 11 lies 5 mm inside the minimum switch, the usable area is 190.6×251.3 mm, and each axis retracts 5 mm off its last switch. Using the nominal 80 microsteps/mm, a 1.6 ms homing-step period corresponds to 7.8125 mm/s. The counted switch-to-switch traverses alone would then need $(200.6 + 261.3)/7.8125 \approx 59.1$ s; this excludes the initial searches, retractions and parking, and is not a measured calibration duration. The result is deliberately not stored between sessions, since a bump between sessions would silently invalidate it, and a failed calibration leaves movement unavailable until calibration succeeds. Algorithm 3 states the procedure.

Algorithm 3 Direction-agnostic calibration of one axis

1: drive one way at 1.6 ms per step until *either* switch of this axis triggers
2: reverse; count steps N until the *other* switch triggers
3: $\kappa_{\text{axis}} \leftarrow N/L$ {L the hand-measured travel, Equation 11}
4: **if** the second switch to trigger was the maximum-side one **then**
5: $\quad n_0 \leftarrow N$
6: **else**
7: $\quad n_0 \leftarrow 0$
8: **end if**
9: retract 5 mm off the switch that is still closed
10: **if** an already-triggered switch, an unexpected switch on the other axis, or 200 000 steps pass with no trigger **then**
11: $\quad$ abort calibration; refuse all jobs until it is re-run
12: **end if**

The hand measurement of travel also sets the scale of every subsequent movement. An error δL scales every commanded position by $x\,\delta L/L$, so to keep that scale error below half of the 0.5 mm allowed by R5 over the 190.6 mm of usable X travel,

$$\delta L \le \frac{0.25 \text{ mm}}{190.6 \text{ mm}} L = \frac{0.25}{190.6} \cdot 200.6 \text{ mm} = 0.26 \text{ mm} \quad (0.13\ \%), \tag{12}$$

and switch-trigger repeatability adds directly to the zero error on top of that. This is an allocation of half the positional-error allowance to scale error, not a measured uncertainty budget. Switch repeatability and mechanical compliance use part of the remaining allowance [33, pp. 18–19].

A mechanical switch does not change state cleanly: its contacts strike, separate and strike again over a few milliseconds, so a single read taken during that interval may return either level. Treating one high read as a trip would stop the machine at random, and treating the first read as final would miss a genuine trip. The standard remedy is to *debounce* the input, requiring the same level on several reads spaced far enough apart to outlast the bouncing.

3.3.6 Gcode generation and motion mapping

G-code stores movement and pen-state instructions in a text file. The hardware controller reads each command and its parameters, then converts the target coordinates to motor pulses. The in-

structions used by this plotter are explained below before the square-and-triangle commissioning example.

Interpolating a straight line. Stepping the two axes independently would draw an L-shaped path, so the program interpolates with Bresenham's integer line algorithm [34, p. 28]: an error term e is carried along the major axis of $a_x = |\Delta x|$ and $a_y = |\Delta y|$ steps, the major axis is pulsed every iteration and the minor axis whenever e would go negative, and the end-stops are read before every major step. Keeping e in $[0, a_x)$ gives the minor-axis count after k major steps as

$$m_k = \left\lceil \frac{k\,a_y - \lfloor a_x/2 \rfloor}{a_x} \right\rceil, \tag{13}$$

which is within half a microstep of the ideal $k\,a_y/a_x$, at most 6.25 µm at the nominal 80 microsteps/mm, using integer arithmetic only. Equation 13 is the reason no floating-point division appears in the inner loop: the deviation from the true line is bounded by construction rather than by rounding behaviour. Algorithm 4 states the loop.

Algorithm 4 Coordinated line movement with an integer error accumulator

Require: signed target changes in microsteps
1: set axis directions; choose major count N_M and minor count N_m
2: $e \leftarrow \lfloor N_M/2 \rfloor$
3: **for** each of the N_M major-axis steps **do**
4: **if** abort requested or an end-stop is confirmed **then**
5: **return** interrupted
6: **end if**
7: pulse major axis; update its count; $e \leftarrow e - N_m$
8: **if** $e < 0$ **then**
9: pulse minor axis; update its count; $e \leftarrow e + N_M$
10: **end if**
11: **end for**

Figure 11 illustrates the commanded microsteps for a separate generated move of $(+6, -28)$ microsteps. Its maximum deviation from the ideal line is 0.5 microsteps, or $0.5/80 = 0.00625$ mm at the nominal scale. This checks command-space interpolation; it does not measure the pen's physical path.

Equation 14 gives the delay of every pulse for a move of length d at feed v_f over $N_{\text{maj}} = \max(|n_x|, |n_y|, 1)$ major-axis steps,

$$\tau = \max\!\left(\frac{d/v_f}{N_{\text{maj}}},\, 2\,\mu\text{s}\right), \qquad v_f = \max(F/60,\, 0.1), \tag{14}$$

with F in mm/min: 900 gives $\tau = 833$ µs (1200 steps/s), 2400 gives 312.5 µs (3200 steps/s, 60 rpm), homing 625 steps/s or 7.8 mm/s. Because *every* pulse of both axes sleeps for τ, a segment takes $(n_{\text{maj}} + n_{\text{min}})\tau$ and Equation 15 gives the effective feed,

$$v_{\text{eff}} = v_f \frac{n_{\text{maj}}}{n_{\text{maj}} + n_{\text{min}}} \in \left[\frac{v_f}{2}, v_f\right], \tag{15}$$

unity axis-parallel, $\frac{1}{2}$ at 45°. There are no controlled acceleration ramps; the steady-speed torque estimate alone does not establish a safe starting rate [28].

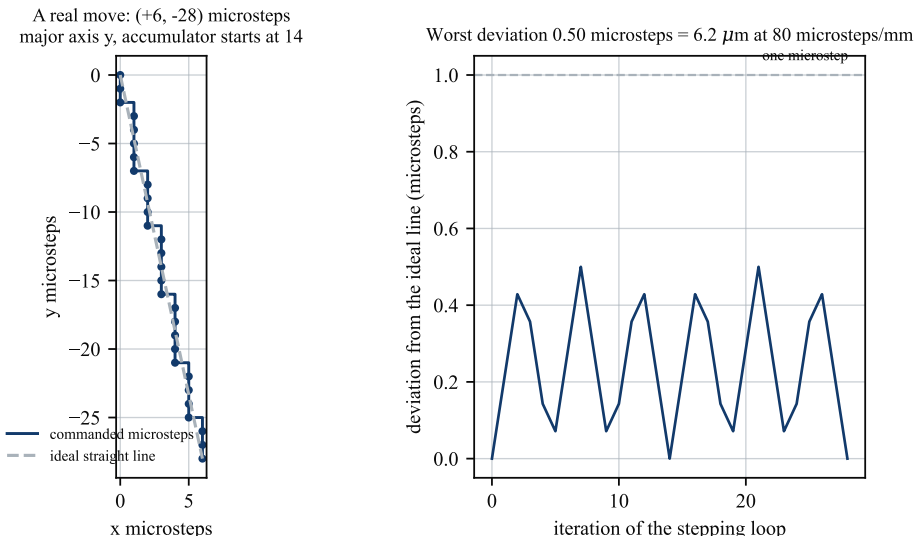
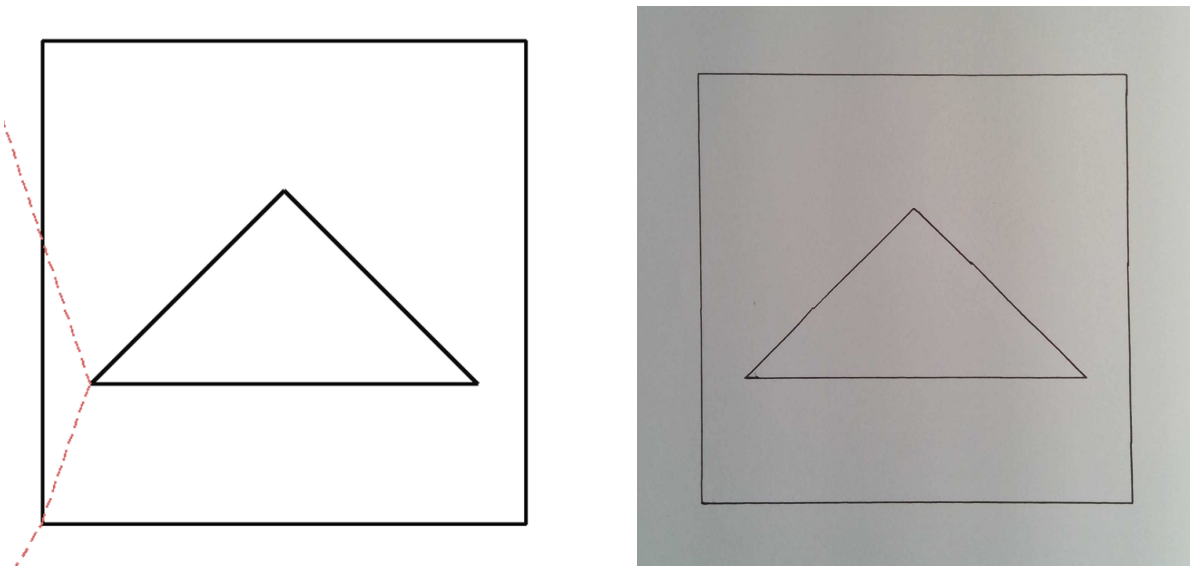


Figure 11. Integer axis stepping and command-space deviation for a generated move.

Figure 12 compares the generated path preview with the corresponding physical plot. The test file commands a 100 mm square and an 80 mm-base triangle, providing both horizontal and diagonal movements for visual comparison.



(a) G-code path preview (b) Reproduced drawing

Figure 12. Comparison of the G-code preview and the physical square-and-triangle plot.

The following extract shows the commands used to draw the square; the full file then continues with the triangle before its final `M2`. Coordinates are in millimetres and use absolute positioning; the pen is lowered before the drawing moves and raised after the square is complete.

```
G21
G90
G0 F2400
G0 X45.000 Y76.000
```

```
M3
G4 P0.350
G1 F900
G1 X145.000 Y76.000
G1 X145.000 Y176.000
G1 X45.000 Y176.000
G1 X45.000 Y76.000
M5
G4 P0.350
```

Table 2 describes the commands and parameters used in this file. Travel commands do not themselves raise the pen, and the travel feed is restated after drawing because `F` remains active until changed. The meanings of `M3` and `M5` are specific to this plotter firmware, where they operate the pen-lift mechanism rather than a spindle.

Command or parameter	Meaning in this plotter
`G21`	Select millimetres as the units for coordinates and feed rates.
`G90`	Use absolute positioning, so each X/Y coordinate is measured from the machine origin.
`G0`	Move rapidly without drawing, used for positioning and travel with the pen raised.
`G1`	Make a straight, feed-controlled move; with the pen down, this draws a line.
`G4 P`	Dwell for the duration specified by P in seconds; used here to let the pen-lift motor finish moving.
`M3`	Lower the pen using the plotter's one-way pen-lift motor.
`M5`	Raise the pen using the plotter's one-way pen-lift motor.
`M2`	End the G-code program.
`X`, `Y`	Target coordinates in millimetres when used with the active positioning mode.
`F`	Feed rate for movement, in millimetres per minute with `G21` active.
`P`	Dwell duration in seconds when supplied to `G4`.

Table 2. G-code commands and parameters used in the square-and-triangle test file.

3.4 Page conditioning and line segmentation

The recogniser requires an image containing one line of text. A photographed page, however, can contain uneven lighting, rotation, page edges and strokes extending between neighbouring lines. This stage prepares the page and separates it into ordered line crops while preserving the shape and relative spacing of the handwriting. The following subsections explain each operation before bringing them together in the complete processing sequence.

3.4.1 Lighting normalisation and ink extraction

The first part of the process is to tell apart ink from paper. A single threshold applied directly to brightness can classify a shadowed area as ink, therefore causing a misclassification. Comparing each pixel with its local background reduces this problem, as in adaptive thresholding [7, pp. 2–3]. This design uses a local brightness ratio, followed by a threshold chosen from the ratio histogram. A local brightness and threshold means the level is calculated from a certain area around each pixel to determine its local brightness,

The page is represented as grey values from 0 to 255. The analysis uses a half-resolution image which is obtained by averaging each 2x2 block, which reduces the pixel count by a factor of four. Averaging the pixels lets a thin stroke contribute to the smaller image even if it would fall between alternate sampling positions, so if it is any of the four blocks, it will have an effect.

Estimating the local background. For any grey image G, the local background B_r is the average of a square window that has a radius of r. The implemented radius is $r = \max(8, \min(40, \text{round}(H/41)))$, SOURCE where H is the working image height. This makes the window follow image size within bounded limits, so it averages over strokes rather than treating their darkness as the background.

A summed-area table provides the window sum from four corner values [7, p. 2]. With exclusive prefix sum S, the sum over rows $[y_0, y_1)$ and columns $[x_0, x_1)$ is

$$\Sigma_{[y_0,y_1)\times[x_0,x_1)} = S(y_1,x_1) - S(y_0,x_1) - S(y_1,x_0) + S(y_0,x_0). \tag{16}$$

Dividing by the window area gives B_r. Once the prefix image has been constructed, each window needs four accesses regardless of its radius. Edge replication prevents artificial zero padding from darkening the estimated background at the image boundary.

Separating ink from paper. Each pixel is divided by its local background estimate. Dark ink therefore has a lower ratio than the surrounding paper. The binary mask M marks pixels whose ratio is below the selected threshold t^*:

$$R(y,x) = \frac{G(y,x)}{\max[B_r(y,x), 1]}, \qquad M(y,x) = \mathbf{1}[R(y,x) < t^*]. \tag{17}$$

Here R is normalised brightness, and the denominator floor prevents division by zero. This ratio formulation is the project's adaptation of local-background comparison; it brings paper brightness towards a common level before threshold selection.

Otsu's criterion selects t^* by maximising separation between the two histogram classes [6]. For a candidate threshold t, ω_0 and ω_1 are the class masses, and μ_0 and μ_1 are their mean bin indices:

$$t^* = \arg\max_t \ \omega_0(t)\omega_1(t)\big[\mu_0(t) - \mu_1(t)\big]^2. \tag{18}$$

The implementation applies this criterion to 256 bins over the clipped ratio range 0–1.2, rather than to the original grey histogram.

Calculation on an actual page. The recorded Owen01 trace uses a stored photograph reduced before analysis, producing an 800×600 pre-deskew working image. At pixel $(317, 343)$, $G = 100$. Its 41×41 background window has prefix values 20696850, 18404173, 18359673 and 16338366. Equation 16 gives $\Sigma = 271370$, hence $B_r = 271370/1681 = 161.43367$ and $R = 100/161.43367 = 0.619449$.

For that image, Otsu's criterion selects bin 171. Its class masses are 0.07069375 and 0.92930625, with mean bin indices 124.98356 and 217.71913, giving between-class variance approximately 564.97942. The corresponding ratio threshold is $(171+0.5)(1.2/256) = 0.80390625$. Since the example pixel's ratio is lower, it is retained as ink. These values describe the reduced calculation trace; the native-page examples below are separate runs.

3.4.2 Page orientation and skew correction

Once an ink mask is available, the page must be aligned so that text lines run approximately horizontally. A quarter turn changes the line axis, while a small rotation changes the slope of lines on that axis. These are handled separately.

For the quarter-turn decision, the code compares the fraction of nearly empty rows in the current image with that in its 90° rotation. The rotation is accepted only when its score exceeds the original by a factor of 1.35. This prevents two similarly weak scores from causing an unnecessary turn, although sparse text can leave the correct axis ambiguous.

After choosing the line axis, fine deskewing tests seventeen angles from −4° to +4° in 0.5° steps. At each angle, the row profile $p_\theta(y)$ counts the ink pixels projected onto row y. The chosen angle maximises the squared row counts, following the projection-profile principle [8]:

$$\theta^* = \arg\max_\theta \sum_y p_\theta(y)^2, \qquad p_\theta(y) = \sum_x M_\theta(y,x). \tag{19}$$

Squaring favours ink concentrated into distinct text rows rather than spread across neighbouring rows. A further reduced mask is used for this search because it needs the arrangement of whole lines. Ink coordinates are mapped onto candidate rows instead of rotating the full image at every trial. The selected rotation is then applied to the full-resolution and working grey images, and the ink mask is rebuilt.

In the Owen01 trace, the scores at +0.5°, +1.0° and +1.5° are 442973, 464105 and 439403. The largest score across all seventeen angles is 464105, so the selected rotation is +1.0°.

3.4.3 Line detection and separation

The next task is to locate the text lines and separate neighbouring lines without cutting their strokes. A straight horizontal cut works only when there is a clear blank row. Ascenders and descenders can overlap vertically even when they belong to different lines, so the design uses line centres followed by curved boundaries.

Removing marks and locating line centres. The line pitch P is the estimated vertical spacing between text lines. A preliminary estimate sets the scale for removing printed structure: the bright page region separates the paper from dark surrounds, and long nearly horizontal or vertical runs are masked using limits of 1.25 and 1.4 pitches. Artifact components are also removed before the final line centres are selected. These are geometric rules; a long handwritten underline can resemble a printed rule and may also be removed.

After this cleaning, pitch is estimated again from a smoothed row profile over the central writing region. Candidate peaks must exceed 6% of the maximum profile value. The preferred pitch is the median spacing between peaks, subject to a lower bound based on component height. This

reduces the chance of treating the ascender, body and descender bands of one line as separate text lines. Autocorrelation and component-height estimates provide fallbacks when peak spacing is unsuitable.

Line centres are selected in order of peak strength, with a minimum separation of $0.6P$ to suppress multiple centres within one line. A further option merges peaks separated by a shallow valley, but its default threshold disables that merging. It is therefore not assumed to operate on every photographed page.

Finding a boundary between lines. Between neighbouring centres, the boundary follows a low-cost path from the left edge to the right edge. The cost should make crossing ink expensive while keeping the path near the middle of the gap. For midpoint y_m and half-gap height h_g, the implemented local cost is

$$C(y,x) = 4\widetilde{M}(y,x) + 0.012\left(\frac{y - y_m}{h_g}\right)^2. \tag{20}$$

Here $\widetilde{M}$ is the ink mask after morphological closing: dilation followed by erosion [35, p. 541]. The closing radius is $\max(2, \lfloor 0.07P \rfloor)$ working pixels. Closing fills narrow breaks that could otherwise let the boundary pass through a stroke. The ink penalty is deliberately stronger than the midpoint attraction. These weights are project design choices, rather than values prescribed by seam carving.

The minimum-cost path is found by dynamic programming, following the seam-carving principle [10, p. 3]. The accumulated cost D includes the local cost and the cheapest predecessor in the previous column:

$$D(y,x) = C(y,x) + \min_{|d| \le s}\{D(y+d, x-1) + 0.02|d|\}. \tag{21}$$

The allowed row change is $s = 7$ per column, and $0.02|d|$ penalises unnecessary vertical movement. Backtracking from the lowest cost in the final column gives the boundary.

In the recorded boundary between centres 38 and 61, $y_m = 49.5$ and $h_g = 11.5$. At row 49 the closed mask is zero, giving $C = 0.012[(49 - 49.5)/11.5]^2 = 0.0000226843$. At column 283, the selected predecessor is row 50 with stored cost 0.00641966. Its one-row change adds 0.02, so the accumulated cost is $0.00641966 + 0.02 + 0.0000226843 \simeq 0.02644234$. This connects the local penalty to an actual step of the path search.

Figure 13 shows two selected gaps from Abhinav01 processed at native page resolution. The first was selected for the largest reduction in crossed ink: a straight midpoint cut intersects 64 working-image ink pixels, while the seam intersects one. The second was selected for the largest departure from the midpoint. Its seam moves by up to 22 rows and crosses no ink, compared with 44 pixels for the straight cut. These examples explain the boundary choice; they are not an average segmentation-accuracy measurement.

A separate sensitivity check varies the midpoint and step penalties over six fixed gaps from two pages. At the implemented 0.012/0.02 setting, mean midpoint offset is 0.363 working pixels; removing midpoint attraction raises it to 4.055 pixels. Every tested setting still crosses two raw-ink pixels across those seams. The check shows how the weights affect the path without establishing a unique optimum. The complete settings and outcomes are retained in Appendix B.6.

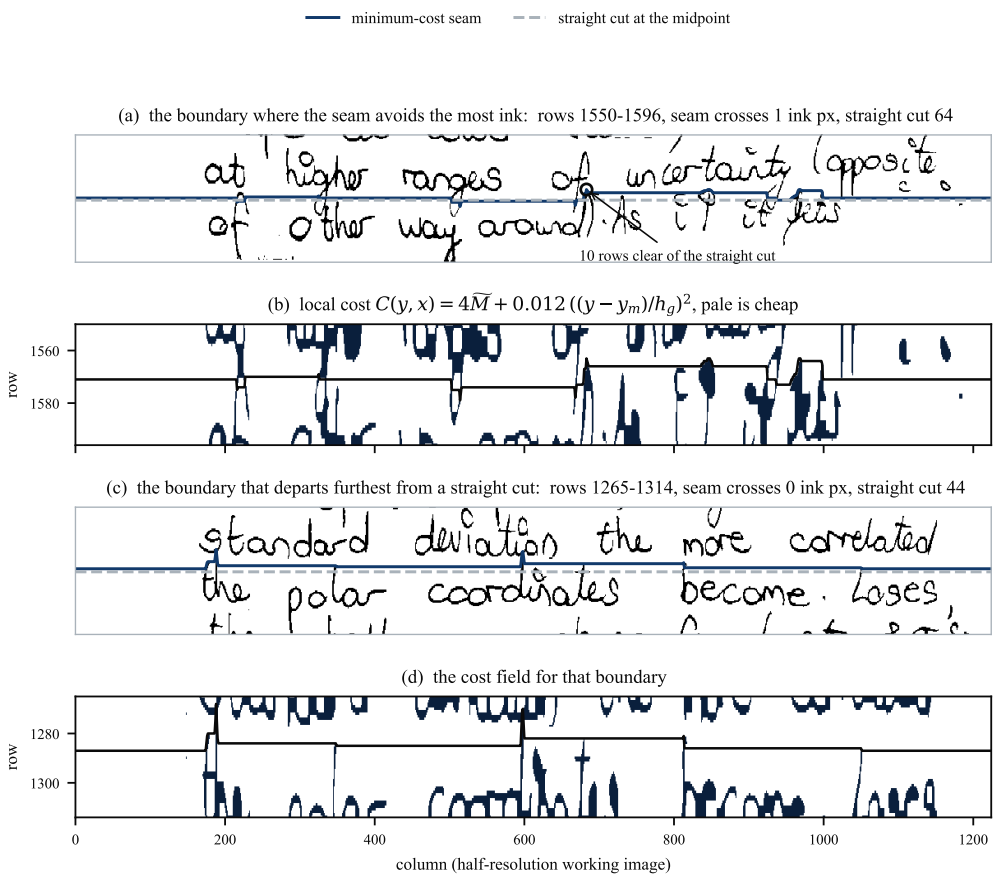


Figure 13. Seam paths and local cost fields on two explicitly selected gaps from Abhinav01.

3.4.4 Assigning ink to the correct line

A seam initially divides the page into bands, but the band alone does not always identify which line owns an ink piece. A dot may fall above the boundary of its stem, or a descender may extend into the band below. Connected-component analysis refines these assignments. The implementation uses row runs and union operations with diagonal contact, giving eight-connected components [36, pp. 476–479].

For a component spanning several bands, a band containing at least 15% of its in-band ink is treated as substantial. The uppermost substantial band provides the reference owner for small-piece corrections; if none qualifies, the largest overlap is used. Large components are not moved wholesale: their pixels keep the seam-band assignment except where the narrow-tail and small-piece rules apply.

An `i` dot between two lines illustrates the reason for this correction. Its position may place it in the upper band, while a stem below supports assigning it to the lower line. Closing can also join the dot to an unrelated descender, so the raw mask is revisited rather than accepting the closed component without further checks. An enclosed-ring test distinguishes a detached loop from an open hook. These decisions use geometry, not knowledge of the intended character.

In Algorithm 1, $\mathcal{C}$ is the set of ink components, $a_{C,n}$ is component C's overlap with band n, and O stores the final line index of each retained ink pixel.

Algorithm 1 Refining seam bands into ink ownership

Require: Ink mask M, components $\mathcal{C}$, seams, centres and pitch P.

for each component $C \in \mathcal{C}$ **do**
  Count overlap $a_{C,n}$ with each seam band n
  $S \leftarrow \{n : a_{C,n} \geq 0.15 \sum_j a_{C,j}\}$
  Choose topmost S as reference owner, or maximum overlap if $S = \varnothing$
  Correct eligible narrow tails and small pieces; reject remote specks
  Associate compact dots with plausible stems using pitch-scaled guards
end for
$O \leftarrow$ corrected ownership, preserving large-piece seam assignments
for each raw component near a neighbouring line **do**
  if detached-dot or enclosed-ring guards pass **then**
    Transfer eligible pixels to the supported stem's line; protect correction
  end if
end for
Reassign residual fragments lacking their own body-band support
return O; excluded pixels retain no line index

The three largest correction cases on Abhinav01 are shown in Figure 14. Colour identifies the assigned line, and the boxes locate the affected ink. Across the page, 462 pixels change assignment; the three boxed components contain 115, 78 and 73 pixels. This makes the correction rules visible, although the reassignment count is not an accuracy score because no manually labelled pixel-ownership reference was supplied.

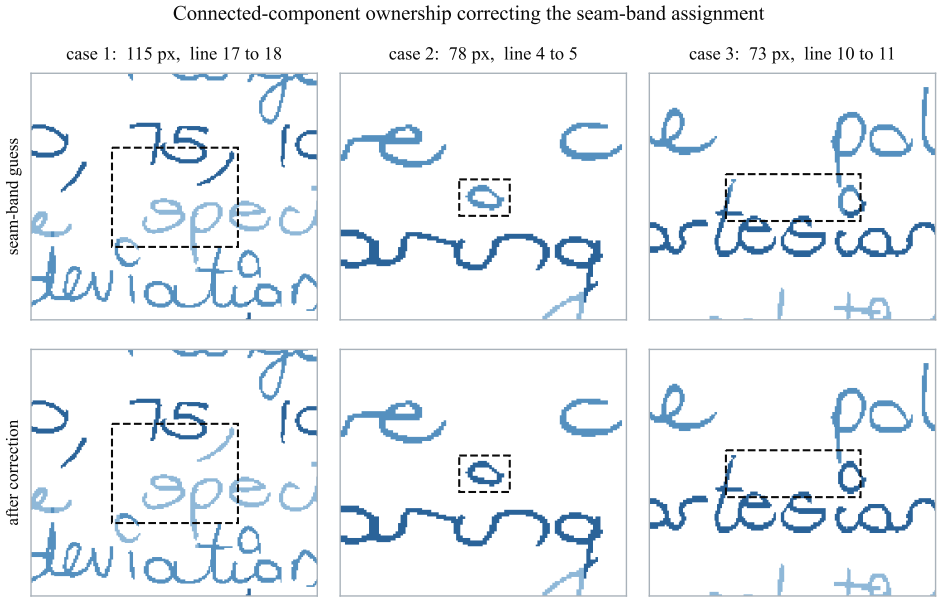


Figure 14. Initial seam-band ownership and corrected component ownership on the same page.

3.4.5 Preparing the classifier crops

Once ownership has been established, each line is cropped from the full-resolution image. The ownership map excludes neighbouring ink, while local contrast adjustment strengthens faint strokes within the retained line. This order matters: contrast recovery should not bring a neighbour's stroke back into the crop.

The crop uses a soft interpolation of its own ink mask to avoid jagged edges. The separate ownership exclusion uses nearest-neighbour interpolation so that a yes/no assignment is not blurred back into partial ownership. Contrast is stretched between a robust ink level and the paper level, followed by a darkness exponent of 0.55. This strengthens faint writing without treating the whole crop as a solid band.

A final guard rejects a crop only when its dark-pixel fraction exceeds both 15% and three times the page's median crop fraction. Requiring both conditions helps retain a genuine dense line when the other lines are sparse. If every crop would be rejected, the input crop list is retained. This is a guard against solid shadows, rather than a guarantee that every retained crop contains valid text.

The classifier input must then fit a 64×640 canvas. For crop height h and width w, the same scale is applied to both dimensions:

$$a = \min\left(\frac{64}{h}, \frac{640}{w}\right), \qquad (h', w') \simeq (ah, aw). \tag{22}$$

The archived first Owen01 line is 156×2170 pixels. Its scale is $\min(64/156, 640/2170) = 0.2949309$, giving a 46×640 resized image after integer rounding. It is placed nine rows below

the top of the canvas, left-aligned and padded with white. This preserves letter proportions, although a very long line becomes short in height when the width limit controls its scale.

3.4.6 Complete processing sequence and output

Figure 15 brings the stages together on a photographed page: lighting correction produces an ink mask, orientation aligns the lines, boundaries and ownership separate the ink, and the full-resolution page supplies the final crops. These intermediate images make it possible to identify the stage at which a stroke was removed or assigned incorrectly.

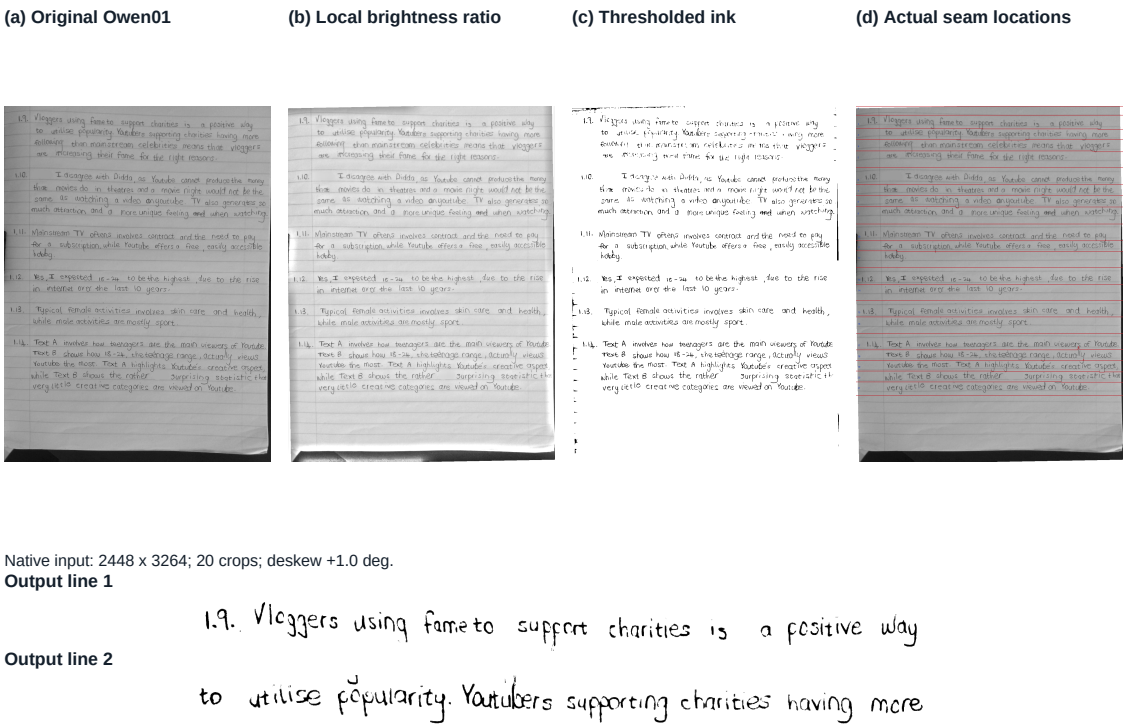


Figure 15. Processing stages used to obtain separate text-line images from a photographed page.

Algorithm 2 summarises the sequence now that its operations have been explained. Here G_f and G_h are the deskewed full- and half-resolution grey images, M is the ink mask, R is the brightness-ratio image and θ is the selected skew angle. The line pitch is P, the centres are c, the seams are b, and O is the refined ownership map. The output $\mathcal{L}$ contains ordered line crops with their coordinates and classifier views; diagnostics retain the angle, pitch and rejected-crop information.

The line-axis decision does not distinguish a page from its upside-down version. After crops are available, the server therefore asks the reader to compare up to three sampled lines in both directions. It selects the direction with the higher mean best-frame log score, reuses the selected readings and reverses the line order when necessary. This is a practical orientation heuristic; a confidently wrong reading can still select the wrong direction.

Algorithm 2 Photographed page to ordered line crops

Require: Grey page on its chosen line axis.
Ensure: Ordered full-resolution crops, coordinates and diagnostics.

$(G_f, G_h, M, R, \theta) \leftarrow \text{Deskew}(G)$
$M \leftarrow \text{RemovePrintedStructure}(M, G_h)$
$(M, \mathcal{C}) \leftarrow \text{DropArtifactPieces}(M, R)$
$(P, c) \leftarrow \text{LineCentres}(M)$
$b \leftarrow \text{SeamCuts}(M, c, P)$
$O \leftarrow \text{AssignOwnership}(M, \mathcal{C}, b, c, P)$
$\mathcal{L} \leftarrow \varnothing$; normalise full-resolution brightness
for each line index n **do**
    $C_n \leftarrow \text{BuildLineCrop}(n, O, G_f, P)$
    if C_n has sufficient ink and height **then**
        Append C_n, its fixed-canvas view and bounding box to $\mathcal{L}$
    end if
end for
$\mathcal{L} \leftarrow \text{DropSolidCrops}(\mathcal{L})$; renumber retained crops
return $\mathcal{L}$, seams, ratio image and diagnostics

3.4.7 Segmentation verification and limitations

The native-resolution Owen page in Figure 15 produced 20 crops for its 20 labelled lines. This checks the gross splitting of that page, rather than the correctness of every boundary pixel. Figure 16 examines a more difficult case using three actual camera-roll photographs of the same three-line page. Their shadows and presentation differ, but neither illuminance nor physical rotation angle was measured.

The striped-shadow capture produced three crops, while the diffuse-shadow capture produced four. The third capture remained on the wrong line axis: its gap scores of 0.379 and 0.400 give a ratio of 1.054, below the required 1.35, and it produced 28 slices. Fine deskewing reached its -4° limit but could not correct the missed quarter turn. These examples show why crop count and visible ink assignment must be inspected together; they do not establish a lighting operating range or a general success rate.

The scale-based thresholds and curved boundaries handle changes in handwriting size and overlapping vertical extents without fixing one pixel spacing for every page. Their limitations follow from the same assumptions: irregular spacing weakens a single pitch estimate, touching strokes can defeat component ownership, and sparse text provides weak orientation evidence. A global rotation also does not correct perspective distortion or page curvature. A more consistent camera and page mount reduces these disturbances at capture; more general rectification would require additional processing and verification.

Appendix B retains the capture filenames, extended digital perturbations and independent algorithm checks, including unsuccessful cases after further image reduction.

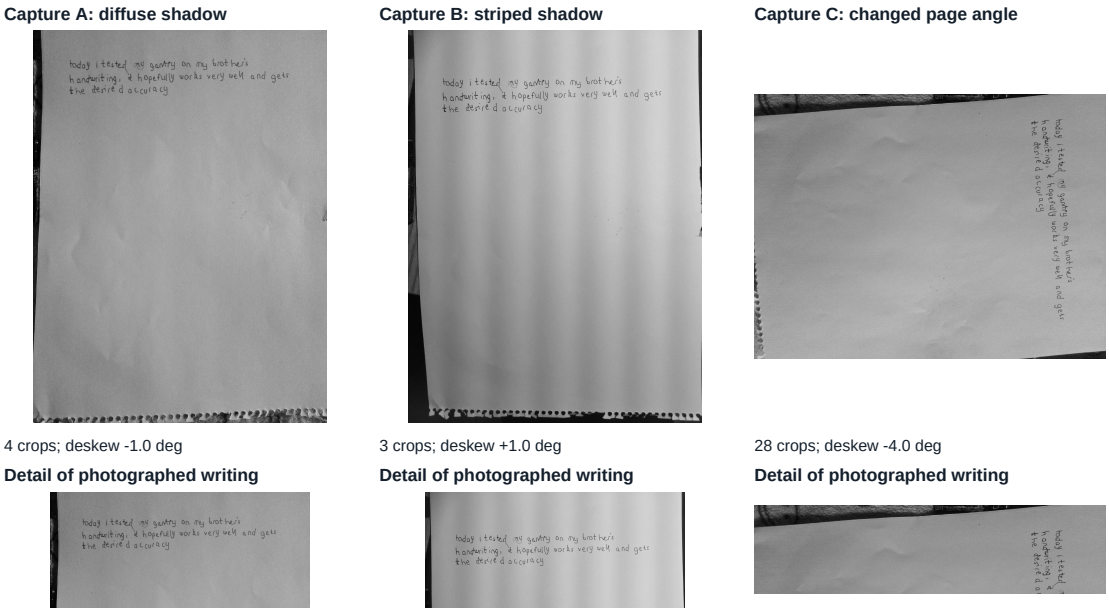


Figure 16. Actual repeated photographs and segmentation diagnostics for the same sparse page. Capture C retains the unsuccessful axis decision.

3.5 Neural network design, training and classification

The classification stage has two outputs: the text in a line and the identity of its writer. Both depend on stroke features, but text recognition must preserve their horizontal order while writer identification must combine evidence across the line. Two networks therefore use the same convolutional architecture with different heads. The recogniser follows the CNN–BiLSTM approach of Kizilirmak and Yanikoglu [12, pp. 4–5]; the project implements and integrates that architecture with its own preprocessing, training and handwriting-reproduction pipeline.

3.5.1 Training data, labels and evaluation separation

Public IAM handwriting [4] and the photographed personal contributions provide examples with known text and writer identity. Public line images supply text-recognition training data, while writer-labelled pages and the personal samples support writer identification and style extraction. The eleven writers selected for the reported evaluation and intended demonstration are six database writers (150, 152, 153, 384, 552 and 588) and five personal contributors. A stored profile, a classifier output class and an evaluated writer are different objects; the checkpoint's recorded class mapping defines how its output indices are interpreted.

The data pipeline pairs every crop with its transcript and writer label. The same preparation functions are used by the first-principles and framework training routes so that a comparison does not silently change segmentation or label handling. Blank or unusable labelled lines are excluded.

Characters outside the configured alphabet are filtered by the label encoder; they cannot be recognised as separate output symbols unless the alphabet and output layer are extended.

The page-based writer dataset holds out an IAM page per writer, while the personal data use a fixed-seed 15% line split. A page split and a line split provide different separation: a held-out personal line can share camera conditions, page background and writing session with training lines. It is therefore appropriate to describe those samples as held-out lines, rather than as wholly unseen capture sessions. Augmented versions of a sample must remain within its assigned training partition.

Preparing public forms also requires excluding printed prompts and signatures before matching detected lines to their transcripts. The eleven-writer classifier corpus contains 1186 labelled lines: 517 IAM and 669 photographed lines, split into 1031 training and 155 held-out lines. The final reader instead uses a joint corpus of Teklia pre-cut IAM lines and the five photographed writers; its exact training and validation counts are not preserved in the available record.

Evaluation separation concerns three inputs: the writer classifier's training lines, the reader's training lines and the pages used to build glyph profiles. Holding a line out of one classifier does not exclude it from the other corpus, and constructing a profile from an evaluation page already exposes its letterforms. Cross-corpus source identifiers and profile-page manifests are therefore needed before describing combined performance as independent of those inputs.

3.5.2 Shared convolutional feature extractor

The input tensor has dimensions $(B, 1, 64, 640)$, where B is batch size. The grey crop is inverted and scaled to $[-1, 1]$, making dark ink positive and white padding negative. Table 3 gives the implemented feature-map dimensions. Two pooling operations reduce the width to 160 positions; there is no third spatial pooling stage before height is collapsed.

Stage	Operation	Output excluding batch
Input	Aspect-preserving line fit and intensity scaling	$1 \times 64 \times 640$
1	Two 3×3 convolutions, each with batch normalisation and ReLU; 2×2 max pool	$32 \times 32 \times 320$
2	Four convolution/normalisation/ReLU blocks; 2×2 max pool	$64 \times 16 \times 160$
3	Six convolution/normalisation/ReLU blocks	$128 \times 16 \times 160$
Height reduction	Maximum over the remaining height	$128 \times 1 \times 160$

Table 3. Implemented convolutional backbone shared in architecture by the text and writer networks. The two models have separately trained parameters.

The convolutional feature extraction follows the reference architecture [12, pp. 4–5]. The local weighted-sum operation follows the CNN formulation [11, pp. 6–8]; Equation 23 writes it explicitly for the numerical implementation. For output channel o, the input-channel and kernel sums combine into one feature value.

$$z_{o,y,x} = b_o + \sum_{c} \sum_{i,j} W_{o,c,i,j} X_{c,y+i,x+j}. \tag{23}$$

For the first Owen01 line, channel 0 of the first convolution at canvas row 34, column 462 gives $\sum WX = -1.150897$ with learned bias $b = -0.000435125$. Equation 23 therefore produces $z = -1.151332$. The nine actual input and learned-weight values are retained in Appendix B.7.

Here W is the learned kernel, b its bias and X the padded input. Equation 23 is the cross-correlation convention used by the implementation. Image patches are rearranged into a matrix and multiplied by flattened kernels, rather than evaluating a Python loop for every multiply. This follows the matrix formulation of convolution [37]. The choice improves the practicality of the numerical route while leaving the layer's calculation visible.

Batch normalisation evaluates Equation 24, with channel statistics computed over the batch and spatial dimensions during training [15, pp. 4–5].

$$\widehat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma\widehat{z} + \beta. \tag{24}$$

At the same activation, the checkpoint stores $\mu = 0.2163021$, $\sigma^2 = 0.02585394$, $\gamma = 1.0193636$ and $\beta = 0.1324657$. Using $\varepsilon = 10^{-5}$ in Equation 24 gives $y = 1.0193636(-1.1513321 - 0.2163021)/\sqrt{0.02585394 + 10^{-5}} + 0.1324657 \simeq -8.53619$. The following ReLU sets this negative response to zero. These are the stored inference statistics, rather than statistics calculated from this single line.

Here γ, β are learned channel parameters and ε stabilises division. At inference, stored running statistics replace μ_B, σ_B^2, making Equation 24 a fixed affine operation. The following ReLU applies $\max(0, y)$ and max pooling retains the largest feature in each local window. These operations progressively convert pixel neighbourhoods into learned responses while reducing the spatial size passed to the task heads.

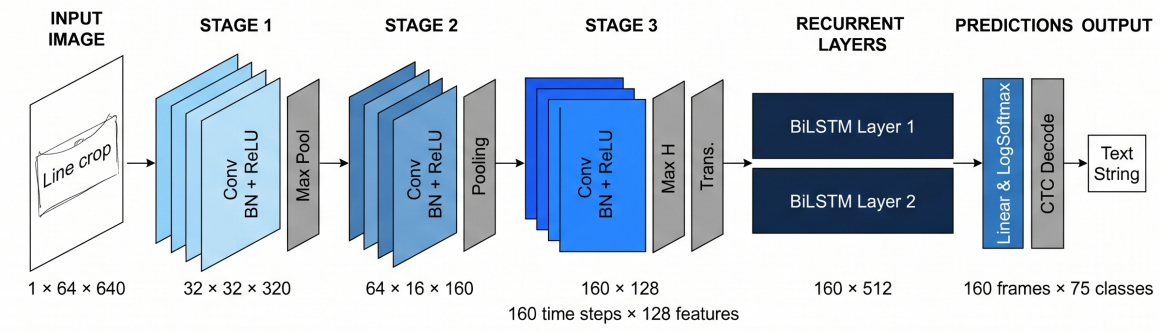


Figure 17. Architecture of the text reader and writer classifier.

Figure 17 makes the different uses of horizontal position explicit. The term "shared" refers to architecture and numerical building blocks; it does not mean that the trained writer classifier uses the reader's current weight tensors.

The arrays behind the architecture are shown in Figure 18, using the joint reader on the archived Owen line. Each row shows three channels selected for their activation strength at that stage; the maps are independently scaled for visibility, so their darkness is not a comparison of absolute response across layers. After the two spatial pooling stages, the final tensor is $128 \times 16 \times 160$. Taking the maximum over height gives $128 \times 1 \times 160$: 160 ordered feature columns, each containing 128 values. This links the convolutional representation to the sequence input without treating an output frame as a complete character.

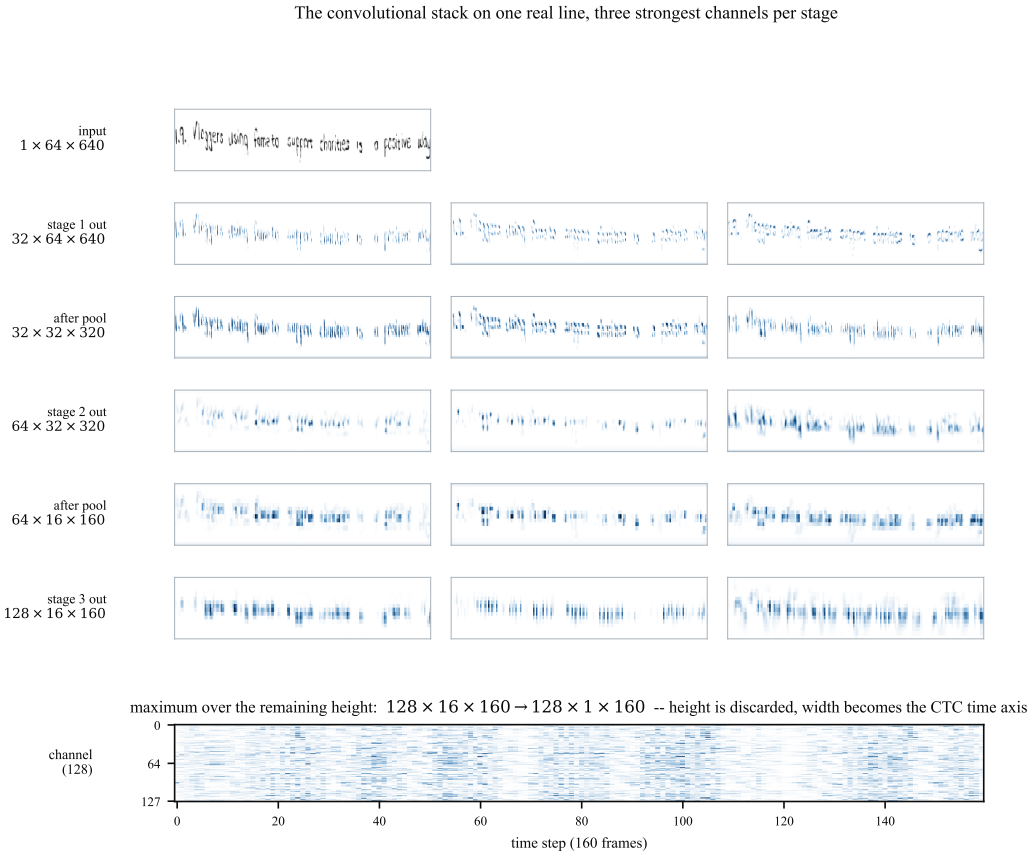


Figure 18. Feature maps and height pooling from a real line passed through the selected joint reader.

3.5.3 Text recognition using sequence modelling and CTC

The text head treats the 160 feature columns as a sequence. Each column contains 128 features after height pooling. Two bidirectional LSTM layers, each with 256 hidden units per direction, combine evidence from both sides of a character before a linear head produces scores over the alphabet and the blank class. The configured alphabet contains 74 characters, giving 75 output classes including blank. The chosen input geometry and decoding are part of this project's implementation rather than an assertion that every setting in the reference study was reproduced.

The LSTM state update in Equation 25 retains an additive memory path [3, p. 5].

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t). \tag{25}$$

In the first forward LSTM layer for that line, frame 10 and cell 0 have $c_{t-1} = -2.7277257$, $f_t = 0.7306111$, $i_t = 0.6666368$ and $g_t = -0.8560218$. Equation 25 gives $c_t \simeq -2.5635625$. With $o_t = 0.000145395$, the cell emits $h_t \simeq -0.000143680$. This records the memory calculation used while processing the real feature sequence.

Here c_t is cell state, h_t is hidden output, i_t, f_t, o_t are sigmoid input, forget and output gates, and g_t is the candidate update. Each is computed from the current feature vector and previous hidden state. The implementation omits the additional cell-to-gate peephole connections shown in the cited source. The backward direction applies the same type of recurrence to the reversed sequence, and the two outputs are concatenated. Equation 25 explains why the implemented reader has sequence state in addition to convolutional features.

The gates are obtained from one matrix calculation, split in input, forget, candidate and output order: $[a_i, a_f, a_g, a_o]^T = W_x x_t + W_h h_{t-1} + b_x + b_h$. Sigmoid is applied to a_i, a_f, a_o and tanh to a_g. Concatenating both directions gives 512 features per frame; the second recurrent layer therefore receives 512 inputs rather than the first layer's 128. At inference the entire line is available, so using right-hand context does not require predicting future input.

CTC trains from a line transcript without requiring character positions [14, pp. 2–3]. Equation 26 sums the probabilities of frame paths that collapse to the target string.

$$\mathcal{L}_{\mathrm{CTC}} = -\log \sum_{\pi:\mathcal{B}(\pi)=\ell} \prod_{t=1}^{T} p_t(\pi_t). \tag{26}$$

For the complete labelled line "1.9. Vloggers using fame to support charities is a positive way", the recorded forward–backward total is $\log Z = -0.47033843$. Equation 26 gives $\mathcal{L}_{\mathrm{CTC}} = 0.47033843$ and $Z \simeq 0.6247908$. This loss was evaluated offline from the trained reader's actual logits and known transcript, without updating its weights.

Here $p_t(k)$ is the frame probability for symbol k, ℓ is the transcript, and $\mathcal{B}$ merges consecutive repeats before deleting blanks. For example, the path $a, a, \varnothing, b, b$ collapses to "ab", while $a, \varnothing, a$ preserves "aa". The dynamic programme uses blank-interleaved states with transitions that stay, advance one state, or skip a blank when the adjacent labels differ. Arithmetic is performed in log space to avoid multiplying many small probabilities.

The CTC collapse and best-path decoding rules follow Graves et al. [14, pp. 2–3]; the implementation produces $T = 160$ frames. A target with L labels and r adjacent repeated-label pairs requires at least $L + r$ frames. The extended lattice has $2L + 1$ states, but not every state must be visited; that number is not a universal minimum sequence length. At inference, greedy frame

decisions are collapsed by the same rule. This avoids a dictionary or language model, but a locally plausible wrong frame path can remain wrong even where a beam search could recover an alternative.

Algorithm 5 describes the numerical loss and derivative. Let $\alpha_{t,s}$ include the current state's emission, while $\beta_{t,s}$ accounts only for emissions after frame t. The derivative used by the implementation is then Equation 27, with the occupancy convention obtained by adapting the CTC forward–backward derivation [14, pp. 5–6].

$$q_{t,k} = \sum_{s:\ell'_s=k} \exp(\alpha_{t,s} + \beta_{t,s} - \log Z), \qquad \frac{\partial \mathcal{L}}{\partial z_{t,k}} = p_t(k) - q_{t,k}. \tag{27}$$

At frame 11, the probability of V is $p_{11,V} = 0.98030665$ and the target-constrained occupancy is $q_{11,V} = 0.99999995$. The CTC logit derivative is therefore $p - q = -0.01969329$. It is negative because the admissible target paths require more V probability than the current output supplies at that frame; the real-input finite-difference check in Appendix A.6 checks this derivative.

Here Z is the total probability of admissible target paths, z denotes logits and q is posterior class occupancy. The convention for β avoids counting the current emission twice. Equation 27 is averaged over the batch in the trainer, without an additional target-length division. Algorithm 5 in Appendix A.5 gives the full forward–backward procedure with explicit blank/repeat transitions.

The word "Vloggers" in this actual transcript contains adjacent gs. They require separate emission runs with a blank between them: a direct skip between identical label states would collapse to one character. The same restriction is retained during forced alignment in Section 3.6.2; training sums admissible paths, whereas extraction selects one.

Figure 19 compares the two paths through the same reader output. Greedy decoding selects the largest probability at each frame, whereas forced alignment restricts the path to the supplied transcript, following the CTC conventions already defined above [14, pp. 2–4]. On this particular line, the decoded string matches the known transcript exactly; the blank class accounts for about 46% of the mean frame probability. The repeated letters in "Vloggers" illustrate why separated emission runs are needed. This is one worked recognition example, not a replacement for the pooled character-accuracy results.

3.5.4 Writer identification and stroke normalisation

Writer identification should combine the shape evidence distributed along a line. It therefore averages the 160 column features rather than passing them through a sequence decoder. A 128-element feature vector is mapped through a 96-unit hidden layer, ReLU and a final K-class output. Dropout with probability 0.2 is active in the hidden head during training and omitted during inference [16, pp. 1931, 1934]. Here K is the number of writers represented by that checkpoint.

The initial classifier operated on handwriting as photographed and performed well on those samples. Ink width can itself carry writer-identification information: Brink et al. explicitly use directional ink-trace width as a feature and relate it to the pen and writing style [20, p. 162]. The photographed input also retains darkness and capture variation, which the initial network was not constrained to ignore. The gantry uses its own pen, so reproducing the selected writer's geometry does not reproduce the same ink appearance. A classifier that relies on those cues

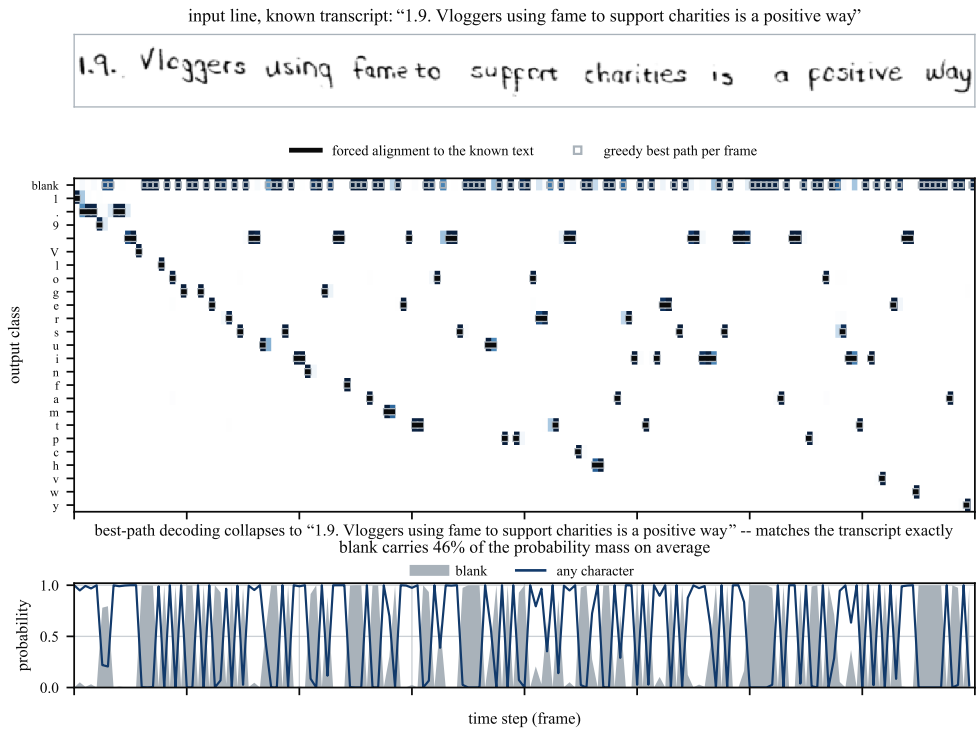


Figure 19. Forced alignment, greedy decoding and blank probability for one known Owen transcript.

would therefore provide an unsuitable basis for judging whether the robot preserved the writer’s style.

The later classifier uses stroke-normalised input to restrict this source of variation. Ink is binarised, reduced to a centreline with Zhang–Suen thinning [26, p. 237], then expanded using morphological dilation [35, p. 533] before the fixed-canvas fit. Equation 28 combines these established operations with the project-selected width rule.

$$I_s = 255\left[1 - \delta_k\left(\text{thin}(M)\right)\right], \quad w = \text{clip}\left(\text{round}(\rho h_x), 2, 9\right), \quad k = 2\lfloor w/2 \rfloor + 1. \tag{28}$$

The same Owen crop has an estimated body-band height of 38 pixels. At the selected $\rho = 0.17$, the width calculation gives $w = \text{clip}(\text{round}(0.17 \times 38), 2, 9) = 6$, then $k = 2\lfloor 6/2 \rfloor + 1 = 7$. The actual redraw therefore uses a seven-pixel square dilation before fitting the line to the network canvas.

Here h_x is estimated x-height, ρ is the selected width fraction and δ_k is square dilation with an odd k-pixel kernel. The actual discrete kernel is important: nearby fractions can produce exactly the same transformed image. The code contains a default fraction of 0.14 and supports explicit training widths; the final model and its preprocessing must use matching settings.

The stroke-normalised classifier is used for writer identification and synthesis evaluation in the final design described here. This reduces dependence on ink appearance while retaining the letter geometry that the gantry can reproduce. It does not make the classifier completely independent of capture conditions, since thresholding and skeletonisation can themselves change a faint or touching stroke. It also removes potentially useful evidence from original handwriting, making the choice a deliberate trade-off between photograph-specific discrimination and consistency across original and reproduced writing.

Figure 20 shows this transformation on existing real crops. The comparison makes the individual operations visible: binary extraction defines what ink is retained, thinning reduces it to a centreline, and dilation restores a controlled width. A matched ink-versus-shape comparison on original and reproduced samples can then test the reason for the choice without substituting classifier confidence for independent stylistic agreement.

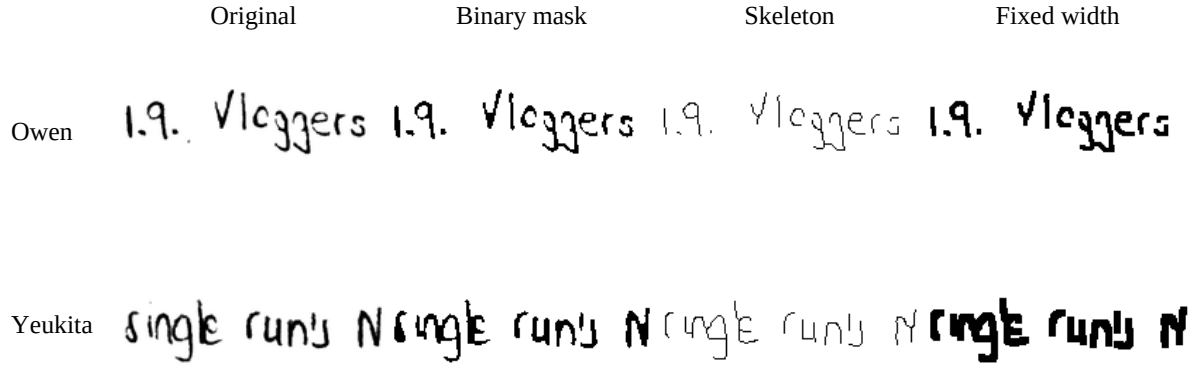


Figure 20. Stroke-normalisation stages on selected Owen and Yeukita crops at the selected width fraction of 0.17.

The width-selection tooling separates deterministic preparation from repeated learning. It skeletonises and redraws each crop once per candidate width, saving images together with writer names, labels and split membership. Training then augments those prepared images each epoch. A fixed split and controlled initialisation make the widths comparable without changing the writer population at the same time. The preparation code records cases where normalisation returns no usable ink and the raw crop is retained; these cases retain their original crop rather than a successfully normalised shape.

Table 4 records the supplied eleven-class validation sweep. All four runs had a 50-epoch budget and used the framework writer model on a Google Colab T4 GPU. The reported epoch is the best saved epoch, not the stopping epoch. The 0.17 setting gave the highest held-out accuracy in this sweep and defines the selected eleven-writer configuration; this is a preprocessing choice evaluated on validation data, not the integrated robot's writer-identification result.

Width fraction ρ	Best saved epoch	Held-out writer accuracy
0.06	38	98.06%
0.09	43	97.42%
0.11	39	98.06%
0.17	43	99.35%

Table 4. Stroke-width selection using the same eleven-writer population and held-out partition.

Appendix B.9 illustrates the actual integer dilation kernels selected by the sweep fractions; nearby fractions can coincide on a particular crop. The selected checkpoint stores its width as 0.17. Both preparation and inference for that configuration must use this value; the module default of

0.14 is not its training setting. Figure 20 illustrates the selected transformation on existing crops. Reusing the held-out partition to choose width makes it model-selection data; an independent test set remains necessary for an unbiased final performance claim.

3.5.5 Training procedure and model selection

The implemented training routes update the convolutional backbone and task-specific head together. Convolution weights use variance-scaled initialisation; normalisation scales begin at one and offsets at zero. Training-time augmentation changes line geometry while preserving its label. The implemented transforms include shear, small rotations, perspective changes and elastic distortion. These operations are applied only to the training view, while validation retains a consistent preprocessing path.

The text trainer supports public lines, page-derived lines and a joint source that mixes public and personal examples. In the joint configuration, a personal-sample fraction controls their representation rather than letting a much larger public dataset determine the entire batch distribution. The default fraction is 0.20. This is a sampling choice, not a claim that personal and public samples are interchangeable. Its value and the training source must accompany any comparison between checkpoints.

Table 5 summarises the numerical training recipe exposed by the current implementation. The number of epochs is a run setting and should be reported from the actual training record, rather than taking a command-line default as the completed run.

Setting	**Reader**	**Writer classifier**
Objective	CTC loss	Cross-entropy
Optimiser	RMSprop	AdamW
Initial learning rate	10^{-3} default	10^{-3} default
Weight decay	10^{-5}	10^{-4}
Learning-rate schedule	Plateau reduction, factor 0.5, patience 5, minimum 10^{-5}	Cosine annealing over the configured run
Gradient-norm clipping	Not applied by the current NumPy text loop	Maximum norm 5
Best-checkpoint criterion	Lowest validation loss	Highest validation accuracy

Table 5. Training settings implemented in the first-principles trainer. The table describes implemented defaults rather than a recovered record of every completed training run.

Cross-entropy penalises the writer model's predicted probability of the true class. For label y and probabilities p_k, its per-sample loss is $-\log p_y$, with logit derivative $p_k - \mathbf{1}[k = y]$ [3, p. 6]. The CTC backward pass similarly relates each predicted frame probability to its target-state occupancy, calculated by forward and backward recurrences through the alignment lattice [14, pp. 4–6]. These gradients are propagated through the task head and the convolutional stages; they are not supplied to the numerical trainer by automatic differentiation.

Validation runs with dropout disabled and stored normalisation statistics. A low training loss alone does not select the model. Separate checkpoint and best-model files preserve progress while retaining the selected validation model. Reproducible batching, declared splits and saved

writer mappings support later evaluation, but comparable recipes do not imply identical weights or learning curves across implementations.

Algorithm 6 identifies the operations actually required for a first-principles training batch. Forward caches store quantities needed by the derivatives, rather than rerunning stochastic augmentation or dropout during the backward pass. The complete numerical batch/update procedure appears in Algorithm 6 in Appendix A.5; the calculations remain explicit in the source.

3.5.6 First principles and framework implementations

The first-principles route in `train.py` evaluates the forward pass, backward pass, loss and optimiser updates using explicit NumPy operations. NumPy supplies array arithmetic and matrix operations; it does not supply a convolutional network, automatic differentiation or a CTC training function in this route. The learned parameters are stored as named arrays. The inference module reads these arrays and evaluates the forward operations without requiring a machine-learning framework.

The framework route in `trainlibrary.py` expresses the corresponding networks using PyTorch layers and automatic differentiation. Both routes use the project's data preparation, parameter naming and named-array export layout. This makes the framework implementation useful as a numerical reference while allowing either training route to supply weights to the NumPy inference code.

The selected reader and eleven-writer weights were trained using PyTorch and exported to named arrays. The first-principles contribution comprises the implemented numerical training operations and the operational NumPy forward path. Fixed-parameter comparisons verify the numerical calculations; the provenance of the selected weights remains the framework training route.

Matching the architecture is not sufficient to establish a matched training experiment. The current numerical CTC function averages unnormalised sequence losses across the batch, whereas the framework trainer constructs CTC with its default mean reduction, which normalises each sequence by target length before averaging [38]. The framework loop also zeroes impossible-alignment losses. These conventions must be matched for a controlled training comparison because target-length weighting changes the influence of short and long lines. Named-array export remains compatible with both routes.

The selected demonstration reader is `paper_cnn_bilstm_ctc_joint_best.npz`; the selected eleven-writer checkpoint is `author_shape_11_pw170_weights.npz`. Results in Section 4 compare these selected models with separately trained numerical-route reader and writer checkpoints. The joint reader also supplies the numerical Design traces, so those traces retain their original checkpoint provenance. The reader's training computer, completed epoch count and original learning-rate/batch settings are not preserved in the supplied record. The implementation defaults in Table 5 must therefore not be presented as recovered settings for that historical run.

The operation-by-operation comparison is retained in Table 16, Appendix A.7.

Equation 23 and the shape progression in Table 3 define the implemented forward calculation. Its numerical correspondence and the backward operations are checked separately in Section 3.5.7.

This separation distinguishes a correct description of an architecture from evidence that its calculations were implemented correctly.

The patch-matrix weight/input derivatives, their algebraic derivation and the patch-storage calculation are given in Appendix A.6.

Additional convolution and CTC derivative checks are illustrated in Appendix B.8; they test numerical operations separately from recognition accuracy.

3.5.7 Network verification and design limitations

Verification separates numerical correspondence from learning performance. For the first, the implementations receive the same parameters and inputs, with matched precision, normalisation state and stochastic masks. Differences in logits, loss and gradients then test the implementation. A second experiment trains each route on the same split with recorded initialisation and settings, and compares learning behaviour and held-out predictions. These experiments answer different questions: separately trained weights can differ even when the underlying operations agree.

The existing comparison archive contains a controlled text experiment using 500 training lines, 40 validation lines and 40 test lines over 15 epochs. It is useful supporting evidence for the implementation, but it is not the final eleven-writer system evaluation. A separate archived double-precision comparison uses two random 64 by 640 inputs and identical initial weights, checking the reader and a thirteen-output writer head. Table 6 summarises its recorded values. The writer dropout is disabled for this comparison so that unrelated random masks do not dominate the difference.

Recorded comparison metric	**Reader**	**Writer classifier**
Maximum logit difference / maximum magnitude	2.12×10^{-14}	3.04×10^{-15}
Absolute loss difference	1.14×10^{-13}	1.78×10^{-15}
Worst ranked gradient relative L_2 difference	2.80×10^{-13}	6.01×10^{-14}

Table 6. Archived float64 implementation comparison from the CleanPipeline2 verification script. This is a controlled numerical check, not a final-model recognition result.

The logit metric in Table 6 divides the largest absolute difference by the larger maximum magnitude, with a 10^{-8} denominator floor. Gradient differences are $\|g_N - g_T\|_2 / \|g_T\|_2$, where g_N and g_T are numerical-route and framework gradients. Tensors whose maximum gradient magnitude is below 10^{-6} are retained in the archive but excluded from the worst-ranked summary: dividing roundoff by a nearly zero gradient gives an unhelpful relative error. These results support close agreement for that tested configuration; they do not establish equivalence for every input, code revision or training trajectory.

A separate, newly run small-lattice check enumerated all $4^5 = 1024$ frame paths for a repeated-label target and compared their summed probability with the numerical CTC loss. The absolute loss difference was 4.44×10^{-16}; central differences over all 20 logits gave a maximum absolute derivative difference of 3.63×10^{-11}. This verifies that finite example, including the required blank between repeated labels. Its construction and limits appear in Appendix B; it is not a learned-model accuracy measurement.

The finite-difference reference, numerical substitution and its treatment of non-smooth operations are recorded in Equation 48 in Appendix A.6.

The main design limitations concern what the input and training population can support. Fixed-width fitting limits detail in long lines; a closed writer head always selects among its trained classes; and line-level personal holdout does not establish independence from page or capture conditions. Stroke normalisation improves consistency with the pen plotter but can destroy detail when thresholding or thinning fails. These limitations should be assessed through appropriate examples and held-out measurements, rather than by treating a high softmax probability as proof of generalisation. Formal recognition and writer-identification qualification remains in Results.

3.6 Style extraction and handwriting reproduction

Recognising a line does not provide the motion needed to write it. The reproduction stage must recover usable pen paths from a writer's examples, store the properties that distinguish that writer, and assemble the paths into text that was not necessarily present in the examples. The chosen representation is a library of normalised glyph trajectories accompanied by line-level style measurements. Profile extraction and trajectory assembly are implemented in `style.py` and `synthesise.py`, respectively. This representation provides explicit control over scale and movement, while making the limits of character-based reproduction visible.

3.6.1 Choice of reproduction representation

The available writer samples are a few labelled pages photographed after writing. A glyph library can reuse these observations without fitting a separate generative network for each writer. Haines et al. provide a relevant precedent: their standard experiments used approximately four A4 pages per writer [23, p. 4]. This is evidence that a small-page collection can support a glyph-based approach, not a guarantee that four pages cover every character or writing context. The present library likewise depends on the characters and variants actually observed.

The offline data also influenced the choice. Graves's recurrent synthesis model learns from ordered pen positions and end-of-stroke markers [3, p. 18]; these are not recorded in this project's photographs. A skeleton supplies geometric paths, but cannot recover their true order, speed or pressure. Using it as measured online training data would introduce an unsupported assumption.

Learned image generators remain a valid alternative, including methods that need very few references from a new writer. For example, One-DM uses one style reference at generation, but its IAM experiment trains with 339 writers and its implementation uses four RTX 3090 GPUs [22, p. 9]. A small reference set must therefore be distinguished from the corpus and computation needed to train the generator. Developing and training an additional generator from first principles on the ODROID was outside the chosen computational scope; no training-time comparison was measured. The glyph approach instead retains explicit trajectories that can be scaled, joined and converted directly to machine coordinates. Its trade-off is weaker modelling of connected, context-dependent letterforms, examined at the end of this subsection.

3.6.2 Character alignment and glyph extraction

The extraction input is a line image and its known transcript. Character boundaries are not supplied as labels. The alignment builds on the blank-interleaved CTC lattice [14, pp. 4–5]. Instead, the trained reader gives a probability distribution for every output frame, and forced alignment finds the highest-scoring CTC path consistent with the supplied text. This is a localisation problem: the transcript constrains which labels must occur, while the frame scores indicate where they most plausibly occur.

The alignment uses the same permitted transitions as the CTC training lattice, but maximises a path score instead of summing all path probabilities. Equation 29 gives the recurrence by replacing path summation in the CTC lattice with maximisation [14, pp. 4–5].

$$V(t,s) = \log p_t(\ell'_s) + \max_{r \in \mathcal{P}(s)} V(t-1,r). \tag{29}$$

For the V in that Owen line, frame 11 corresponds to state 11 of the blank-interleaved transcript. Its log emission is −0.01988991 and the three admissible predecessor scores are −13.91888, −10.01139 and −1.3624485. Equation 29 selects their maximum and gives $V = -0.01988991 - 1.3624485 = -1.3823384$. Backtracking assigns V the frame interval $[11, 12)$.

Here ℓ' is the blank-interleaved transcript and $\mathcal{P}(s)$ is the permitted predecessor set: stay, advance one state, or skip an admissible blank. Backtracking yields the frames assigned to each character. Equation 29 makes clear why correct text does not guarantee correct boundaries: several spatial allocations can remain plausible under the same transcript.

An output frame is converted through the network canvas to the original crop using the inverse of its aspect-preserving scale. If a frame edge is t out of T and the crop was scaled by a, its initial horizontal location is approximately $(640t/T)/a$, clamped to the source width. This undoes the resize rather than assuming that every crop occupied the complete 640-pixel width.

The baseline and x-height are operational estimates from the line's horizontal ink profile, not annotated typographic measurements. With $p(y) = \sum_x M(y,x)$, the implementation takes the first and last rows satisfying $p(y) \geq 0.45 \max p$ as the body-band top and baseline; their difference gives h_x. Bands below three pixels are rejected. This common scale makes glyph dimensions comparable across source resolutions, although unusual capitals, short lines or a large descender can bias the estimated body band.

The worked example in Figure 21 uses an archived original Owen line crop, visually matched to the first line of its page transcript, with the joint reader checkpoint. The reader recovered the full line, and the extraction routine then generated the displayed boundaries and normalised paths. Correct text does not certify a correct individual cut: small punctuation and neighbouring strokes still require inspection. The archived crop is identified separately from the new whole-page segmentation run in the evidence manifest.

The initial boundaries are refined by looking for thin, low ink near the expected cut. Equation 30 defines the project-specific column cost used for this refinement. It combines low ink density with a preference for cutting a shallow connector near the baseline.

$$C_x = \frac{\overline{p}_x}{q_{0.60}(\{\overline{p}_j : \overline{p}_j > 0\})} + 1.15 \operatorname{clip}\left(\frac{b - y_{\text{top}}(x)}{h_x}, 0, 3\right). \tag{30}$$

The refined right boundary of this V is source column 193. Although smoothing gives $\overline{p}_{193} = 2.8$ and the positive-column reference is 14.36, the raw ink count at column 193 is zero. The explicit blank-column rule therefore overrides 2.8/14.36 and assigns zero cut cost. This is a real case in which the gap rule prevents smoothing alone from penalising a clean separation.

Here $\overline{p}_x$ is smoothed column ink count, the denominator is its 60th percentile over positive counts, b is baseline position, and $y_{\text{top}}(x)$ is the highest ink row in that column. A denominator floor prevents division by zero. A true blank column is assigned zero cost. The search radius is approximately half an x-height, with a smaller penalty for distance from the initial boundary. The two terms in Equation 30 favour a narrow connector near the baseline over cutting through the body of a letter.

One shared boundary array is enforced to remain increasing, with a minimum cell width of $\max(1, 0.08h_x)$ pixels. Independently snapping neighbouring boundaries could place both at the same gap and create an empty glyph. Figure 21 shows the refined cuts and resulting character shapes.

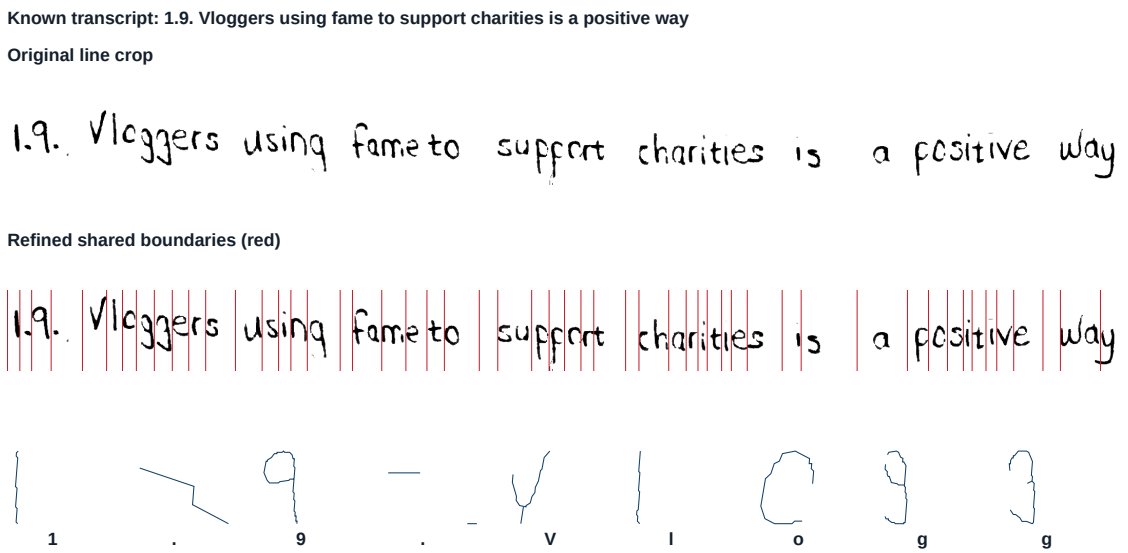


Figure 21. Transcript-constrained character localisation and refinement of glyph boundaries.

3.6.3 Skeleton tracing and pen path construction

Ink is an area, whereas the gantry needs a path. Zhang–Suen thinning reduces the line mask to a one-pixel centreline [26, p. 237]. The complete line is thinned before character cells are separated. Thinning each cut cell independently would introduce an artificial image edge at every character boundary and could change the recovered centreline there.

The skeleton is treated as a graph: adjacent ink pixels form edges, and endpoints and junctions determine where paths begin or split. Redundant diagonal edges are suppressed when an orthogonal route already connects the same local pixels. The default tracer first splits paths at junctions, prunes short dangling spurs, then greedily rejoins nearby ends with a continuation angle below 75°. Thus the default is not simply one pen lift per junction. A separate through-

junction tracer exists but is not enabled by default. Neither procedure establishes natural pen order: the image records final ink, and several stroke sequences can create the same graph.

Traced paths are divided at the character boundaries and simplified while limiting geometric deviation, following the polyline-simplification principle of Douglas and Peucker [25]. Short spurs and cut ligature tails require care because a genuine small stroke can resemble a skeleton artefact. Isolated dots may thin to a single point with no traceable edge; these are recovered as short paths rather than silently discarded.

A detached `t` or `f` crossbar can extend over the next character cell. The extractor checks its width, flatness and height and, when the configured crossbar rule applies, assigns the complete bar to the corresponding letter instead of splitting it at a vertical cut. Such an overlap is not recorded as an in-word pen connection. Without this distinction, a crossbar would both contaminate the next glyph and inflate the profile's connectedness.

The project uses the following coordinate transform to separate source-image size, baseline and slant from the geometry to be stored. Slant normalisation uses the shear principle described for cursive writing [9]; the following coordinate convention is defined for this project. Equation 31 first expresses each path point relative to the cut boundary x_L and baseline b, using the estimated x-height h_x and slant ϕ.

$$v = \frac{b-y}{h_x}, \qquad u_0 = \frac{x-x_L}{h_x} - v\tan\phi, \qquad u = u_0 - \delta. \tag{31}$$

A traced point on this `V` is $(x,y) = (173,59)$, with left cut $x_L = 146$, baseline $b = 107$, $h_x = 38$ and $\tan\phi = -0.0787017$. Equation 31 gives $v = 48/38 = 1.263158$ and $u_0 = 27/38 - 1.263158(-0.0787017) = 0.809939$. After the implementation's three-decimal intermediate rounding and measured post-trim shift $\delta = 0.4472489$, the stored point is $(u,v) = (0.363, 1.263)$ and the recorded lead is 0.447 x-heights.

Image rows increasing downwards become glyph height increasing upwards. After trimming eligible ligature tails, the code sets $\delta = \min(u_0 + v\tan\phi)$ over the remaining points, subtracts it from the stored horizontal coordinates and records lead $l = \mathrm{clip}(\delta, 0, 1.5)$. Thus the stored path and its lead are separate quantities: they preserve the relation between the character cell and the ink rather than forcing every cell to start at its first stroke. Slant is removed for storage and reapplied during placement. The glyph can then be drawn at a chosen physical x-height instead of a fixed number of source pixels.

Entry and exit positions are recorded along with whether the skeleton crosses a character boundary. This distinguishes a measured original connection from a merely small visual gap. The extracted paths still represent a geometric reconstruction, rather than the writer's true pen trajectory: retracing, pressure and movement direction cannot generally be recovered uniquely from a static image.

Figure 22 follows a real Owen line fragment from ink area to centreline and then to simplified paths. Across the complete traced line, 85 paths contain 3 738 vertices before simplification and 974 afterwards, using the extraction tolerance of 0.6 source pixels. This removes 2 764 vertices, approximately 74%, before later glyph placement. Path colours and start markers also expose the separate pen strokes. The order is inferred from static ink and does not recover the writer's original stroke timing. Appendix B.10 connects the resulting points to the stored glyph frame and connectedness count.

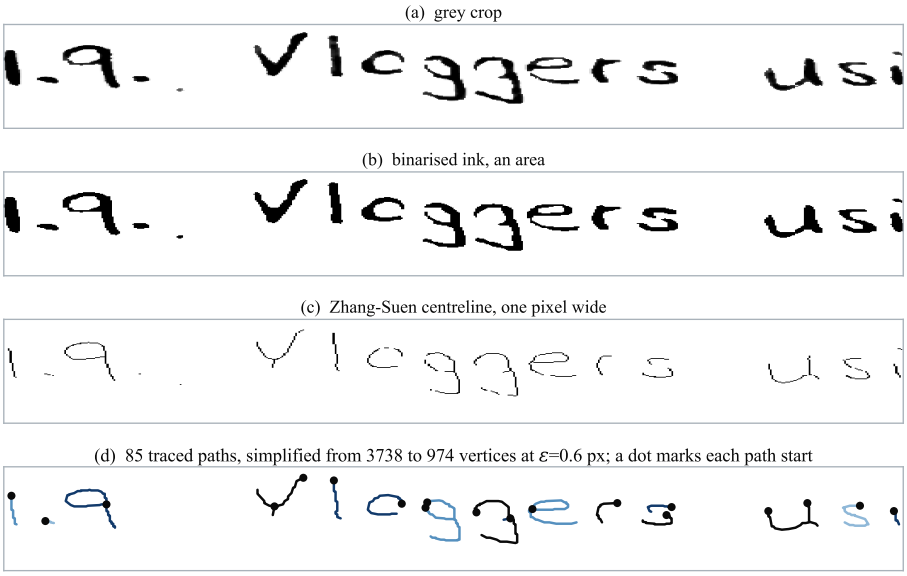


Figure 22. Real ink, its thinned centreline and the traced paths after bounded simplification.

3.6.4 Glyph selection and library quality

The library must retain variation without giving every cut equal credibility. A recogniser can assign the correct transcript while placing one boundary inside a letter, so alignment is followed by geometric and consistency checks. Low-confidence lines are filtered relative to the writer's own alignment distribution. Per-glyph checks reject implausible width, height, stroke count, pen length and disagreement between ink width and the allocated character advance.

The width gate allows up to 2.1 x-heights for ordinary characters and 2.9 for wide characters such as `m` and `w`. Further width and height checks use the character's own distribution, rather than applying one absolute size to every letter. The 40th-percentile width provides a reference less easily enlarged by a few cuts that contain part of a neighbour. Tightening is retained only when enough variants survive to form a usable pool. These gates encode plausible glyph geometry; their exact values are engineering heuristics, not universal typographic laws.

A cross-writer shape prior supplies a second check. Each variant is represented by a smoothed, normalised 12×12 occupancy descriptor. Pooling descriptors of the same character across writers gives a robust reference shape. Its role is to reject malformed cuts from a writer's own library, not to copy another writer's letter into it. Internal consensus then favours variants close to the writer's typical form. The ordinary variant pool is limited to twelve per character; an accepted denoised prototype can be prepended as an additional candidate.

For unit-length descriptor v_g and the normalised median reference r_c for character c, the prior distance is $d(g,c) = 1 - v_g^T r_c$. Internal consensus instead uses pairwise L_1 descriptor distances and chooses the medoid with the lowest total distance. These compare a pooled character reference with this writer's own forms. Dylan's first stored `a` variant has recorded prior distance 0.114. Recomputing the pairwise rule on all nine final stored variants gives the smallest total distance,

33.3413, for variant index 4; the other totals range from 34.4306 to 42.9091. This demonstrates the selection rule on the final pool, without treating a new descriptor calculation as the historical filtering log. Algorithm 3 gives its place in profile construction.

Algorithm 3 Labelled handwriting to a persistent writer profile

Require: Labelled original lines $\mathcal{L}$, trained reader and writer identifier.

$\mathcal{G} \leftarrow \varnothing$; $\mathcal{S} \leftarrow \varnothing$
for each $(G, \ell) \in \mathcal{L}$ **do**
  $A \leftarrow \text{ForcedAlign}(G, \ell)$; estimate baseline, x-height and slant
  if alignment or body band is unusable **then continue**
  Refine shared increasing cuts; thin full line; trace skeleton paths
  Split paths at cuts; assign crossbars; recover untraced dots
  Normalise glyph coordinates; record advance, lead and boundary crossings
  Append glyphs to $\mathcal{G}$ and line measurements to $\mathcal{S}$
end for
Cache raw extraction; reject low-confidence lines relative to this writer
for each character's glyph pool $\mathcal{G}_c$ **do**
  Apply size, height-class, path-length and cell-consistency gates
  Refine size range; apply cross-writer prior and within-writer consensus
  Rank and smooth retained variants; construct medoid and eligible prototype
end for
return persistent profile containing $\mathcal{G}$ and robust summaries of $\mathcal{S}$

Two additional representatives support stable generation: a medoid that is an actual extracted example, and a denoised prototype formed from compatible variants. Prototype strokes are aligned and resampled before taking robust summaries; different stroke counts are not averaged indiscriminately. A prototype is admitted as an additional candidate only when the implemented shape-prior test improves sufficiently. The filtering distinguishes handwriting variation from implausible extraction geometry.

Synthesis preferentially samples well-formed variants and can favour a compatible entry height when joining. Where the requested character is unavailable, the code can use a case-related shape, a similar form, an anchor glyph or a simple fallback. These options preserve the ability to generate text, but they do not establish that the missing character was reproduced in the writer's own style. The trajectory records glyph-source information so that fallback use can be distinguished from genuine extracted forms.

The final stored profiles supply the glyph geometry used during the demonstration. Figure 23 compares the final nine-variant `a` pools for Owen, Dylan and Yeukita alongside their representative forms. These paths can be inspected independently of a photograph or a generated sentence, making it possible to distinguish the stored shape from later slant, jitter and connector operations.

Each displayed `a` pool contains nine variants. Their overlaid paths retain recognisable loop geometry while differing in closure and curvature; the distance bars show why a particular real example is selected. The medoid and median prototype remain distinct: one selects a real variant, while the other combines compatible paths. A low prior distance expresses resemblance to the character reference, not proof of a better match to the writer.

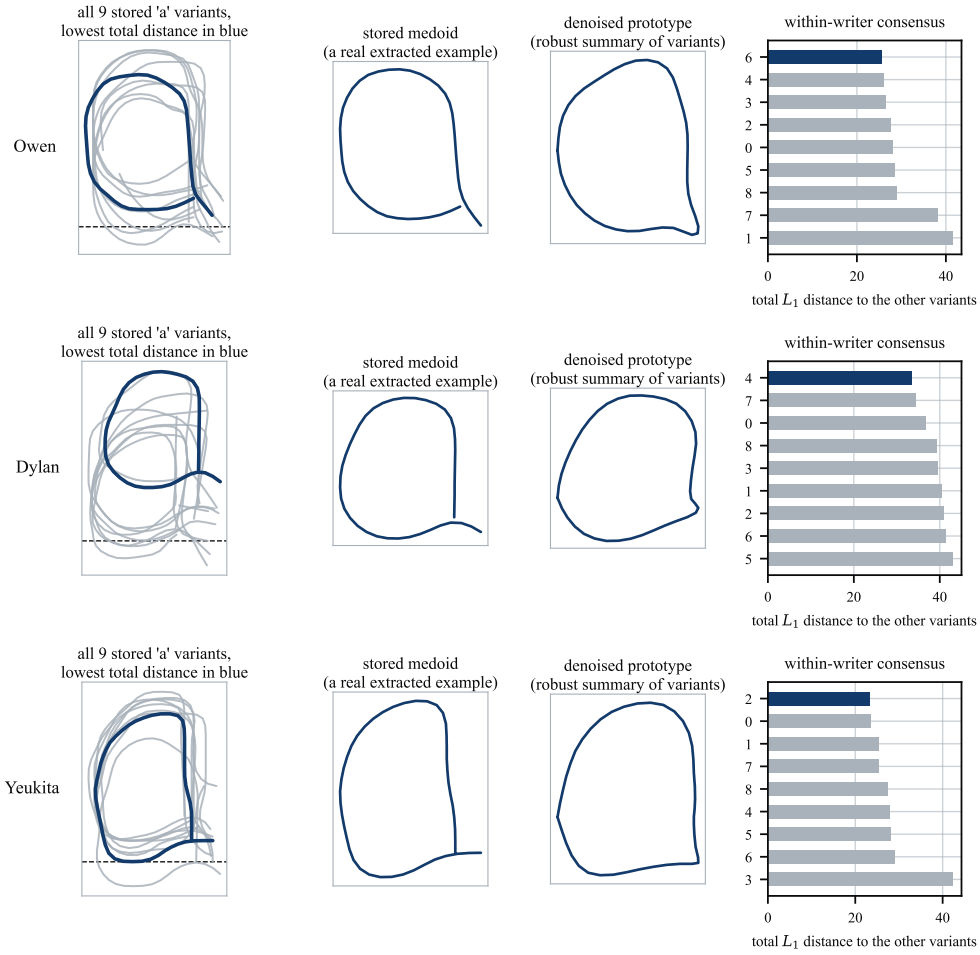


Figure 23. Variant clouds, actual medoids, denoised prototypes and within-writer distance totals for three final profiles.

3.6.5 Writer style measurement and storage

Letter shapes alone do not determine the appearance of a line. The profile also stores slant, x-height, ascender and descender extent, word spacing, per-character advance and connectedness. Measurements use medians where a few unusual cuts could otherwise dominate the result. Table 17 in Appendix A.7 gives the stored quantities, their units and their use in synthesis.

Slant is estimated by shearing the ink through candidate angles and measuring concentration of the resulting column profile. The stored angle is the one that most nearly aligns the dominant strokes. This treats slant as a property measured over a line, so it is less dependent on one unusual letter. It remains an estimate affected by letter content; it is not a direct observation of the writer's hand motion.

Building on the use of textural and shape features for writer characterisation [18, p. 701], this project defines connectedness as the fraction of eligible adjacent non-space glyph boundaries crossed by a source skeleton, as shown in Equation 32.

$$c = \frac{N_{\text{cross}}}{\max(1, N_{\text{adj}})}. \tag{32}$$

For the single extracted Owen line used above, 39 eligible adjacent non-space glyph pairs were counted and three boundaries were crossed by a recovered skeleton. Equation 32 gives $c = 3/39 = 0.076923$. This line-level extraction trace illustrates what is counted; the final stored writer profile summarises its own complete source collection.

Here N_{adj} counts eligible adjacent non-space glyph pairs and N_{cross} counts boundaries actually crossed by a traced centreline. Equation 32 measures connections in the source writing; counting pen lifts in a generated file would instead measure the extractor and synthesiser's decisions. The difference is important when assessing how faithfully the library represents cursive writing.

Profiles are stored as structured files containing the writer identifier, measurements and glyph paths. The permanent representation supports the proposal's multiple-writer requirement without re-extracting the source pages for each request. Cached raw extraction also permits a change to filtering without repeating all tracing. The extraction cache and final profile have different purposes: the former preserves intermediate work, while the latter is the version selected for reproduction.

3.6.6 Text layout and connected handwriting synthesis

The synthesiser receives digital text, a profile, a physical x-height and a line-width limit. It selects a variant for each character, applies controlled shape variation and places the resulting paths into words and lines. Character advance and word spacing are taken from the profile rather than placing every letter at a fixed cell width. A word is moved to the next line when its predicted width exceeds the remaining space.

Equation 33 is the project's inverse placement transform: it reapplies slant and physical scale to the stored glyph coordinates before machine-coordinate conversion.

$$x = p_x + h_x^{\text{mm}}(l + u + sv), \qquad y = p_y + h_x^{\text{mm}}(v + d). \tag{33}$$

In the generated Owen sentence “Writing with style.”, the first placement call, after calibration and jitter, uses $(u,v) = (-0.01134565, 1.47270173)$, lead $l = 0.25117988$, zero shear and drift, and $h_x^{\mathrm{mm}} = 4$. With the initial placement origin $(0,0)$, Equation 33 gives $x = 4(0.25117988 - 0.01134565) = 0.9593369$ mm and $y = 4(1.47270173) = 5.8908069$ mm. This is the actual assembly point before later path processing and machine placement, not a replay of the earlier V crop.

Here (u,v) is a stored glyph point, l is its lead-in offset, s is the applied slant shear, d is local baseline drift, and (p_x, p_y) is the placement origin. The physical x-height h_x^{mm} converts the normalised path to millimetres. Equation 33 keeps the source shape, line layout and physical scale as separate quantities.

Small random translations, rotations and scale changes avoid using identical paths for every occurrence of a letter. A correlated drift term changes the baseline gradually instead of introducing unrelated vertical jumps. The implemented drift update uses a persistence factor of 0.85 and bounds displacement to ±0.25 x-heights. Such variation is controlled rather than unrestricted: too much jitter degrades letter identity and can cause neighbouring glyphs to collide. A collision guard maintains a nominal 0.05 x-height clearance where appropriate.

Variant choice is weighted rather than uniform: the best-ranked forms receive greater probability, and a compatible entry height is preferred when joining. The legibility anchoring strength is also profile-dependent. In the automatic setting, Equation 34 maps the median stored prior distance $\widetilde{d}$ into a value λ.

$$\lambda = \mathrm{clip}\left(\frac{\widetilde{d} - 0.24}{0.32 - 0.24}, 0, 1\right). \tag{34}$$

The final Owen profile’s stored glyph distances have median $\widetilde{d} = 0.18355$. Equation 34 gives $\lambda = \mathrm{clip}[(0.18355 - 0.24)/0.08, 0, 1] = 0$. With no stored override, this is the anchoring value selected by the automatic rule for the generated job.

A clean library therefore retains more sampled personal variants; a library with poor descriptor agreement uses stronger anchors. The anchor first prefers an acceptable representative from the writer’s own profile, then uses a generic form when necessary. Equation 34 is a heuristic for allocation of trust, not a learned probability that a glyph is correct. Source tags identify own, case-related, similar, core, fallback or blank output. In particular, “core” can represent either a personal representative or a generic anchor; that tag alone cannot quantify how much personal geometry was preserved.

For connected writing, the profile’s connection fraction is used as a probability for eligible neighbouring letters. The code suppresses joining for profiles below a 0.45 connection threshold. A proposed join also requires a compatible glyph source and a gap no greater than 0.80 x-heights. This prevents a generic print fallback from being treated as a natural cursive attachment and prevents a long flat connector across an unusually large space. The body stroke is distinguished from a dot or crossbar so that the next letter does not connect to a detached mark.

The default connector uses a shallow curved path between the exit and entry points, with sag limited relative to x-height and separation. A tangent-based alternative exists, but its flag is disabled in the current configuration. When a join is accepted, the connector and both bodies form one continuous pen-down path.

Line-level style calibration compensates for differences between measurements of individual stored glyphs and measurements of the assembled line. A probe is rendered without jitter,

measured using the same style functions, and compared with the reference slant and vertical proportions. Limited shear and extent corrections are applied over three iterations. This improves internal consistency of the representation, but it remains calibration against the project's own measurements; independent evaluation is needed to establish physical stylistic fidelity.

3.6.7 Candidate selection for legibility and writer identity

One set of random variant choices can be more readable than another without being more characteristic of the writer. Candidate selection therefore evaluates both quantities. Each candidate trajectory is rendered at constant ink width, read by the text recogniser, and judged by the stroke-normalised writer classifier introduced in Section 3.5.4. Uniform rendering keeps candidate selection focused on reproducible geometry, while the shape classifier applies the selected stroke-normalisation convention.

The project-defined score in Equation 35 combines text recovery and the requested writer's probability. It uses the insertion, deletion and substitution error convention of Graves et al. [14, p. 2], then combines the resulting character score with the writer posterior using a harmonic mean.

$$A = \max\left(0, 1 - \frac{d_{\text{edit}}(\widehat{t}, t)}{|t|}\right), \qquad J = \begin{cases} 2AP/(A+P), & A+P>0, \\ 0, & \text{otherwise.} \end{cases} \tag{35}$$

For the recorded Owen candidate 1, the 19-character target "Writing with style." differs from its reader output by one substitution, giving $A = 18/19 = 0.9473684$. The selected-writer posterior is $P = 0.5786133$, so Equation 35 gives $J = 0.7184359$. This is the actual score used to select the candidate in Figure 24; it evaluates the software rendering rather than physical stylistic agreement.

Here t is intended text, $\widehat{t}$ is recognised text, d_{edit} is the minimum number of insertions, deletions and substitutions, using the reader error convention [14, p. 2], and P is the model probability of the selected writer. The harmonic mean J in Equation 35 makes a weak component limit the combined score. The actual candidate comparison below holds text and writer fixed, showing how the writer posterior affects selection.

Algorithm 4 gives the bounded search and repair procedure. The server default is fourteen candidates, with the number configurable. Because an entire candidate is rerendered and rescored, a repair is judged in its assembled context rather than by an isolated character score.

A repair is retained only when the complete trial improves the joint score. This bounds the search and protects the best candidate already found. A separate reader-only helper uses a different escalation policy; it is not part of this joint selection procedure.

More trials increase the opportunities to find a useful combination, but also add recogniser and writer-model calls. Search quality therefore has a computational cost for fixed text and profiles. The repaired or fallback glyph proportion also matters: legibility gained by replacing distinctive forms has a stylistic cost. Figure 24 relates the individual candidate scores to the selected trajectory.

In the worked Owen example, three pre-repair trials use the fixed text "Writing with style.", seeds 2026, 3003 and 3980, and the selected eleven-writer shape model with an explicit width fraction of 0.17. Candidate 1 has $(A, P, J) = (0.947, 0.579, 0.718)$, while candidate 3 has the same character score but $(P, J) = (0.464, 0.623)$. The combined criterion therefore selects candidate 1.

Algorithm 4 Joint candidate search and bounded legibility repair

Require: Text, writer profile, reader, shape classifier, seed s and trial budget K.

$J^* \leftarrow -1$; $R \leftarrow \varnothing$; determine anchoring strength
for $k \leftarrow 0$ **to** $\max(1, K) - 1$ **do**
  $T \leftarrow \text{SynthesizeText}(\text{text}, \text{profile}, s + 977k)$
  $I \leftarrow \text{RenderUniformInk}(T)$; $(A, P, J) \leftarrow \text{Score}(I)$
  if $J > J^*$ **then** $(T^*, I^*, J^*) \leftarrow (T, I, J)$
  if $J^* \geq 0.99$ **then break**
end for
for at most three configured repair rounds **do**
  if $J^* \geq 0.995$ **then break**
  $W \leftarrow$ up to two weak aligned positions not already in R
  if $W = \varnothing$ **then break**
  $R' \leftarrow R \cup W$; regenerate with stronger anchors at R' and seed s
  Render and rescore the complete trial trajectory
  if $J > J^*$ **then** commit R' and the new best trajectory
end for
return T^* with selection and accepted-repair metadata

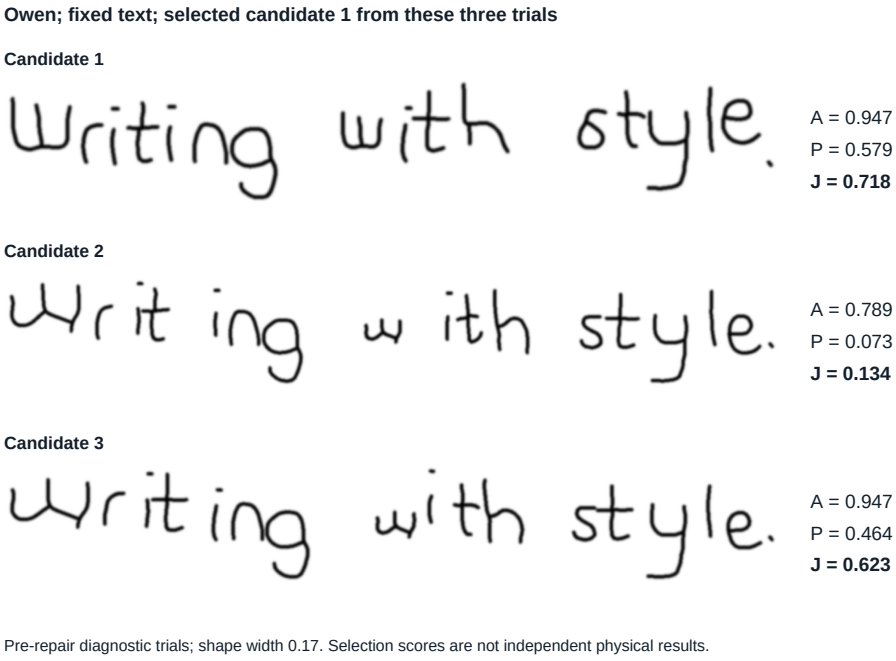


Figure 24. Example of joint candidate selection for a fixed writer, input text and search configuration.

Its reader output begins with a lower-case “w”, which accounts for one edit rather than perfect text recovery. This is an inspectable scoring example before repair, not a claim that the full default search or the physical system attained those scores.

The same models guide selection, so their scores cannot independently prove the output’s quality. A classifier can prefer an image for model-specific reasons, and its posterior is not an 85% stylistic correlation measurement. The search is a design mechanism; final physical handwriting must be qualified separately. Increasing the search budget also does not recover a letterform that is absent or incorrectly extracted from the source library.

3.6.8 Machine coordinate mapping and G-code generation

The selected trajectory is mapped into the calibrated workspace. Normalised glyph size, line spacing and machine margins are handled separately: fitting the height of a tall descender should not silently change the meaning of the ruling or move a line outside the available area. Robust extent measurements limit the effect of a single unusually tall glyph on the scale of the complete passage.

The physical x-height is first chosen from the page ruling and a stored x-height-to-pitch ratio where available. If that ratio is absent, the configured fallback gives $8.4(0.46) = 3.864$ mm at the default 8.4 mm ruling. This is the fallback sizing rule, rather than the value passed to every job: the traced Owen job explicitly requests 4 mm. A probe of the requested text can reduce the selected value if robust line extent exceeds 1.25 rulings. The extent uses the 98th-percentile stroke tops and second-percentile bottoms, with exact extremes for fewer than eight strokes. It limits outliers but does not guarantee every extreme lies inside the nominal band.

Two placement modes are supported. Ruled placement adjusts each line’s measured centre to the configured ruling independently of the character scale. Free placement preserves the synthesised pitch after scaling, which can avoid forcing large ascenders and descenders from neighbouring lines into contact. A line offset selects a later starting position and reduces the remaining page height. The full conversion procedure is retained as Algorithm 7 in Appendix A.8.

For the basic mapping, Equation 36 places a trajectory with bounds (x_0, y_0, x_1, y_1) within the page margins.

$$x_m = o_x + m_x + a(x - x_0), \qquad y_m = o_y - m_y - d_y - a(y_1 - y) + \Delta_y. \tag{36}$$

The generated Owen trajectory has $x_0 = 0.9593369$ mm and $y_1 = 6.6340616$ mm. A point $(2.9829210, 5.3303251)$ mm is placed using the configured origin $(0, 251.3)$ mm, margins $(15, 10)$ mm, unit fit scale and zero line shift. Equation 36 gives $(x_m, y_m) = (17.0235840, 239.9962635)$ mm. Three-decimal file emission writes this point as `X17.024 Y239.996`, connecting the trajectory calculation to the motion segment examined in Section 3.3.6.

Here (o_x, o_y) is the configured machine origin, m_x, m_y are margins, a is a fit scale, d_y selects the line offset and Δ_y is any line-spacing adjustment. Both stored trajectories and machine coordinates use upward-increasing y; Equation 36 positions later lines lower on the page rather than introducing another arbitrary image flip. Requested points are restricted to the usable travel, but clamping is a last boundary protection: it can distort a letter and must not be treated as successful layout.

Strokes are ordered within text rows so that path shortening does not scramble reading order. Short path segments can be decimated, and nearby consecutive strokes may be joined without a lift. The current default distance is 0.6 mm for such joins and for closing a near-complete ring. These operations save travel and pen changes, but can also introduce unintended ink if set too aggressively; their scale relative to the chosen writing size matters. The code retains clipped-point and stroke information to make these effects inspectable.

The generated file selects millimetres and absolute positioning. A travel move reaches the start of a stroke with the pen raised, the pen is lowered, drawing moves follow, and the pen is raised before the next travel. Figure 25 reproduces the beginning of the actual generated Owen file used for these calculations. The interpreter assigns `M3` and `M5` to pen down and pen up respectively.

```
G21
G90
G0 F2400
G0 F2400 X15.000 Y240.557
M3
G4 P0.350
G1 F900
G1 X15.001 Y240.207
G1 X15.002 Y239.857
G1 X15.003 Y239.506
G1 X15.005 Y239.156
G1 X15.006 Y238.806
G1 X15.007 Y238.456
```

Figure 25. Opening commands of the generated Owen sentence, including its actual travel/drawing feeds and first drawing coordinates. The file was inspected offline and not executed on the gantry.

Feed is modal, so the travel feed is restated after drawing rather than assuming that the first `G0 F` remains active. Pen dwell defaults to 350 ms in each direction and is configurable separately for raising and lowering. This dwell is additional to the switch-confirmed actuation performed by the interpreter. A timing estimate that counted only geometric travel would omit those repeated state-change costs. Conversely, a trapezoidal acceleration parameter in a configuration object does not establish that the actual GPIO loop executes acceleration ramps.

Equation 37 is a dimensional calculation of the nominal job budget: distance divided by feed gives travel time, and pen-state changes add their configured dwells.

$$t_{\text{nom}} = \frac{60D_d}{F_d} + \frac{60D_u}{F_u} + \frac{N_p\left(t_{\text{up}} + t_{\text{down}}\right)}{2}. \tag{37}$$

For this same generated file, the unrounded distance accumulators are $D_d = 139.0263729$ mm and $D_u = 158.1555743$ mm. At $F_d = 900$ and $F_u = 2400$ mm/min, with 56 pen transitions and two 0.35 s dwell settings, Equation 37 gives $t_{\text{nom}} = 9.2684249 + 3.9538894 + 19.6 = 32.8223142$ s. The job metadata reports 32.8 s after rounding. This is the converter's actual geometric budget, not a physical completion-time measurement.

Here D_d, D_u are drawing and pen-up distances in millimetres, F_d, F_u their feeds in mm/min, N_p is the number of pen transitions and the dwell times are in seconds. The last term assumes

alternating up/down events. Equation 37 separates settings that affect ink quality from settings that mainly affect non-writing time. It excludes the unknown initial approach to the first stroke, synthesis, switch-confirmed motor actuation, GPIO overhead and the directional pulse-delay effect in Equation 14; the saved estimate is therefore not the measured R3 completion time.

Figure 26 applies the same budget calculation to a separate offline generation of "Writing with style." in four final profiles. The Owen job contains 58 pen transitions: its recorded components are 9.249 s drawing, 3.198 s travel and $58(0.35) = 20.3$ s dwell, giving 32.747 s in total. Thus dwell contributes about 62% of this nominal budget; across the four jobs it contributes approximately 61–65%. The different path and transition counts distinguish these jobs from the earlier worked file. The breakdown identifies a drawing-side optimisation opportunity, but shortening dwell would require a physical test that the pen still clears and settles reliably. These estimates exclude the execution costs stated above and are not additional R3 timing measurements.

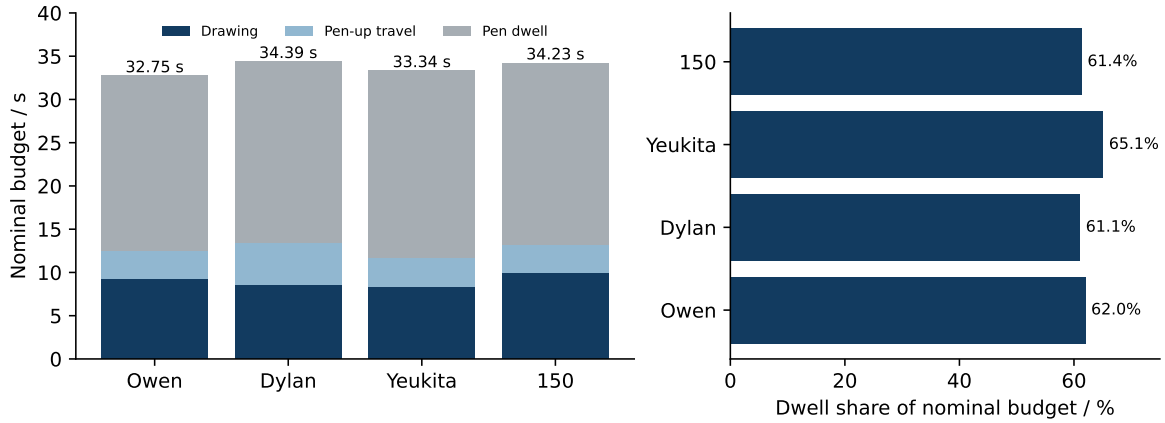


Figure 26. Nominal drawing, travel and dwell budgets for the same sentence generated from four final profiles.

3.6.9 Reproduction verification and design limitations

The reproduction checks follow the representations established above: aligned source ink, stored glyph paths, assembled trajectories and emitted machine commands. Re-rendering the emitted G-code can reveal a coordinate inversion, scale change, missing stroke or additional joining segment before drawing. Comparing that render with the selected trajectory checks conversion; comparing the physical drawing with the render checks execution. These comparisons localise errors without substituting for end-to-end qualification.

Appendix B shows the same sentence generated from four stored profiles alongside paths independently parsed from their emitted G-code. Nine controlled conversion fixtures additionally preserved path vertex counts with no clipping and a maximum coordinate discrepancy of 0.0004 mm, consistent with writing coordinates to three decimal places. Joins and ring closure were disabled only for those correspondence fixtures so that the vertices could be paired; the four profile illustrations used the normal configuration. Neither check measures the mechanism's positional accuracy.

The representation's main advantage is that its geometry is explicit and inspectable. A defective glyph can be traced to a source crop, stored paths can be viewed independently of the machine, and physical size follows from a defined transformation. The main limitation is the loss of context when a connected word is divided into reusable characters. Real letterforms change with their neighbours, whereas a finite glyph pool and join rules approximate that dependence. Skeletons also omit pressure and true temporal order. These losses explain why a plausible letter library need not reproduce every connected style equally well.

Possible extensions include context-dependent multi-character units, better alignment, trajectory acquisition and a verified alternative junction tracer. Each addresses a different loss: multi-character units preserve local letter interactions, trajectory acquisition restores temporal information, and improved tracing changes how an existing skeleton becomes strokes. Their implementation would require corresponding changes to extraction, storage or the source data, rather than a larger candidate search alone. The retained design therefore prioritises extraction from the available offline pages and explicit control of the plotted geometry. Recovering natural pen dynamics would require new trajectory data, while replacing the library with a learned generator would add a separate training and validation problem. The alternative tracer remains an implementation option; its presence alone does not establish an improvement over the default.

3.7 System integration and embedded operation

The server connects the numerical processing, models, profiles and motion interpreter. The browser supplies images or writing requests and displays their outputs; the engineering operations remain in the processing modules.

3.7.1 Classification and reproduction workflows

Captured and uploaded pages follow the same conditioning and recognition path. Writer predictions are combined across the retained line crops using Equation 38, rather than choosing one line to represent the page.

$$\overline{p}_k = \frac{\sum_i w_i p_{i,k}}{\sum_i w_i}, \qquad \widehat{k} = \arg\max_k \overline{p}_k. \tag{38}$$

Applying this vote to the first two archived Owen01 crops gives ink weights 20204 and 21750, with Owen probabilities 0.9996674 and 0.9999279 from the selected 0.17-width model. Equation 38 gives $\overline{p}_{\text{Owen}} = (20204 \times 0.9996674 + 21750 \times 0.9999279)/41954 \simeq 0.9998024$. This is a two-crop calculation trace, not the full-page evaluation reported in Results.

Here w_i counts crop pixels below grey value 128; all-zero weights fall back to equal weighting. More ink therefore gives a line greater influence, including a possible dark artefact. Crop rejection precedes aggregation for this reason. The pooled probability and the fraction of individual lines agreeing with the selected writer are reported separately; neither establishes accuracy without a reference.

A reproduction request selects text, profile and drawing settings, then stores the trajectory, preview, G-code and metadata as one generated job. Characters absent from the profile use the fallbacks in Section 3.6.6. Drawing is a separate action requiring connection and calibration,

allowing inspection before execution. Table 18 in Appendix A.7 records each representation's inputs, outputs and coordinate conventions.

3.7.2 Embedded execution, interfaces and job control

The ODROID N2+ runs NumPy inference and motion control from stored arrays, without the training framework. Canvas size, line count and candidate count still determine its processing load. NumPy supplies array arithmetic; Flask handles requests; image libraries handle file I/O; OpenCV serves the camera interface; and `gpiod` exposes GPIO. Framework network operations belong to the separate training route. The complete source code is supplied separately from this report for the AMS submission.

During motion, the pulse loop checks an abort event and confirmed end stops. A separate request thread permits a stop while drawing is active. End-stop trips latch a fault; the recovery routines attempt switch retraction, pen raising and parking under their required preconditions. The stop request is a software path, not an independent motor-power disconnect. A pen-state timeout remains consequential because subsequent travel may mark the sheet.

The resulting job states distinguish generation, refused execution, completed execution and interruption/failure. Calibration establishes the coordinate reference in software, but missed steps can invalidate its correspondence with physical position. Neither a saved preview nor emitted pulse counts prove that the nib followed the intended path.

3.7.3 Integration verification and remaining constraints

Interface checks should confirm consistent upload/camera processing, checkpoint-to-profile writer mapping, bounded G-code and truthful interrupted-job status. Formal classification accuracy, drawing accuracy and timing against the proposal remain in Results.

The diagnostic retracing route provides a useful bypass: page conditioning and line separation are followed by thinning, tracing and ordinary machine conversion, omitting the reader, glyph library and synthesis. Comparing its G-code render with a drawing helps distinguish execution from synthesis errors. Comparing the drawing directly with the photograph still includes conditioning, skeletonisation, filtering and coordinate errors; it does not isolate mechanics alone.

Per-stage records separate acquisition, segmentation, inference, generation and execution time. Generation metadata is carried into the writing record so that a complete reproduction interval can be reconstructed. R3/R4 qualification must specify its start/end events and use the intended platform; Windows development timings cannot establish an ODROID requirement.

4. Results

The qualification tests follow the six requirements of the Project Proposal in order of importance. The physical reproduction, photographed-page analysis and embedded timing records are kept identifiable so that each result can be traced to the part of the system that produced it. Section 5 interprets the engineering consequences; this section states the procedures and measurements.

4.1 Summary of results achieved

Table 7 compares the proposal's intended functions and numerical specifications with the available measurements. A result above a numerical target is distinguished from a complete qualification of the corresponding requirement.

Intended outcome	Actual outcome	Location
R1. Reproduce any desired digital text in a trained writer's style, with at least 85% character accuracy and 85% stylistic correlation.	Fourteen physical pages, 184 labelled lines and 5 471 reference characters give 84.41% read-back character accuracy. Seven of eleven writers individually exceed 85%. Writer identification is 86.41% by line and all fourteen page pools are correctly attributed. A new geometric comparison of the physical output with 156 original lines gives shape correlation 0.697, below 0.85. The two pooled targets are not met; the writer breakdown identifies demonstrated stronger cases.	QT 1; Appendix C
R2. Decode handwritten content and identify its writer from the training set with 95% overall accuracy.	Writer identification reaches 153/156 lines, 98.08%, and 11/11 writer pools. The selected joint reader reaches 94.06% character accuracy on 1 076 validation lines; the numerical comparison reader reaches 94.39%. The writer point estimate exceeds 95%; the selected reader is 0.94 percentage points below it. A joint text-and-writer success rate is not inferred from these different populations.	QT 2
R3. Reproduce desired text in a trained style at no slower than 4 s per character.	**Met on the completion-confirmed timing records.** All 25 positive-character ODROID records with an explicit program-end marker satisfy their allowances. The worst is 166.61 s for 43 characters, or 3.875 s per character.	QT 3

Table 7. Summary of results achieved against the Project Proposal.

Intended outcome	Actual outcome	Location
R4. Decode the text and identify its writer within $2 + 0.025x$ s for x handwritten characters.	Two recorded ODROID requests take 46.26 s and 55.49 s, against logged-count allowances of 14.50 s and 26.55 s. **Not met on these requests.** A separate development-host experiment measures a 2.76-fold network speed improvement using batched library inference.	QT 4
R5. Draw character trajectories while maintaining absolute positional error no greater than 0.5 mm.	All seven measured square/triangle feature lengths differ from nominal by less than 0.5 mm; the largest difference is 0.37 mm. This supports the calibrated dimensional accuracy. Absolute point-position error and repeated-word overlay performance are not fully qualified by feature lengths alone.	QT 5
R6. Train at least ten writing styles and permanently store their character trajectory mappings.	**Met in the evaluated stored set.** Eleven writer identities have stored profiles and glyph mappings and are represented in the evaluated classifier. All eleven are exercised by the physical-output and writer-identification measurements.	QT 6
FC1. Operate in a well-lit laboratory or office at 400–1000 lux.	Indoor camera captures and robot-page photographs demonstrate operation under the recorded capture conditions. No calibrated illuminance series establishes the complete 400–1000 lux range.	Sec. 4.2.2
FC2. Operate on a stable, level, vibration-free surface.	The physical outputs demonstrate operation in the fixed test arrangement. Surface level and vibration are not quantified, so the evidence applies to that arrangement.	Sec. 4.2.2

Table 7. Summary of results achieved against the Project Proposal (continued).

4.2 Qualification tests

4.2.1 Test material, configuration and measurement definitions

The final evaluation uses six IAM writers (150, 152, 153, 384, 552 and 588) and five photographed contributors, denoted A, B, O, T and Y. These are the eleven writers introduced in Section 3.5. The robot drew fourteen pages because some writers' text occupies two pages. The pages contain a common paragraph, a pangram and, where available, text from the writer's source material. The analysis contains 184 lines with corresponding transcripts; the writer-model comparison additionally evaluates one unpaired crop and therefore contains 185 crops.

The physical pages were produced by the gantry under ODROID control. Accuracy was subsequently scored from saved images on the development host. The embedded timing measurements were extracted from the preserved ODROID server log. This arrangement allows large image sets to be analysed while retaining actual on-device evidence for the time specifications. Numerical equivalence across processors is not assumed to be a new hardware accuracy test.

The reproduction read-back uses the joint reader used in the Design examples. The separate reader evaluation compares that reader with the evaluated scratch reader; the selected joint reader supplies the 94.06% system result and the scratch reader supplies a 94.39% comparison result. Both share the architecture in Section 3.5. Writer identification uses the eleven-class stroke-normalised classifier at the selected width of 0.17. Appendix C identifies the files by their content hashes, so checkpoints with the same filename cannot be confused.

Character accuracy is computed from the summed Levenshtein edit distance rather than by averaging percentages of unequal-length lines. With reference transcript t_i, recognised transcript $\hat{t}_i$ and distance d_{edit}, the scoring definition follows the character-error convention used in handwriting recognition [14, p. 2]:

$$A_c = 100\left(1 - \frac{\sum_i d_{\text{edit}}(\hat{t}_i, t_i)}{\sum_i |t_i|}\right). \tag{39}$$

For the physical pages, Equation 39 gives $100(1 - 853/5471) = 84.4087\%$. The selected reader gives $100(1 - 2719/45747) = 94.0564\%$, while the numerical reader gives $100(1 - 2567/45747) = 94.3887\%$. Insertions, deletions and substitutions all contribute to the numerator; the score is therefore not a binomial count of independently correct characters.

Writer identification is the number of correctly classified lines divided by the number evaluated. Pooled writer decisions aggregate the evidence from a group of lines and are reported separately. The archive's word score counts matching words at equal positions, divided by reference words; it is called *word match* here. A word insertion can shift later positions, so this score must not be interpreted as one minus word edit error rate.

The measurement tools save transcripts, reference lengths, edit counts and predictions before aggregation. The reviewed tables and figures are regenerated from these saved records. Statistical intervals on line classifications use Wilson score intervals [39, pp. 209–212]; because lines from one writer share writing and capture conditions, these are descriptive line-level intervals. Resampling whole writer groups gives an additional sensitivity check for the physical character score. Neither procedure creates additional independent writers.

Qualification test 1: Reproduction of digital text in a trained writer's style (R1)

Objectives of the test or experiment

Determine whether the complete image-to-profile-to-gantry system produces legible new text in the selected stored style. Evaluate the proposal's character target and distinguish measured geometric style similarity from the supplementary ability to identify the writer of the output.

Equipment used

The Cartesian gantry, two A4988 axis drivers, pen-lift mechanism and ODROID N2+ described in Section 3.3; the IMX179 USB autofocus camera; the stored writer profiles; the joint reader and eleven-class writer model; and the development host for image scoring. The retained page images are 3264×2448 pixels. The camera model and signal wiring follow the Hardware section.

Test setup and experimental parameters

The paragraph, pangram and source-text blocks are designated A, B and C. Their different lengths and content are retained in the analysis. A separate transcript follows the actual line breaks of each reproduced page. The final stored glyph libraries supply the paths; the scoring experiment does not introduce a manual glyph-editing step. Greedy CTC decoding is used without a language model. Writer images are normalised to 0.17 before classification. The physical pages represent the recorded jobs; a common search budget or feed is not inferred for jobs whose settings were not retained.

Steps followed in the test or experiment

1. Generate a writing job for the required text and profile, execute it on the gantry and photograph the output.
2. Segment and orient each photograph, then pair every scored crop with its intended transcript line and block.
3. Record the reader output, character edits, reference lengths and writer prediction for each pair.
4. Aggregate counts by block and writer, then pool each physical page's writer evidence.
5. Compare the character score with 85% and retain the separate style measurements with their own populations and definitions.

Results or measurements

Table 8 gives the 184 paired lines by text block. Figure 27 exposes the variation across writers; the full values are in Table 23. Figure 28 provides examples of actual robot output alongside both strong and more difficult cases.

The automated reader recovers 84.41% character accuracy overall, with 45.97% positional word match. The writer is correctly identified on 159/184 lines (86.41%), and all fourteen archived page pools are attributed correctly. Seven writers exceed 85% character accuracy. Y, 150, O and B exceed 93%, and Y, O, B and 384 exceed both 85% character accuracy and 85% line-level

Text block	Lines	Characters	Character / %	Writer / %
A: paragraph	119	4041	84.88	91.60
B: pangram	21	463	78.40	76.19
C: source text	44	967	85.32	77.27
All	184	5471	84.41	86.41

Table 8. Read-back character accuracy and writer identification on the photographed robot output. These are different measures; writer identification is supplementary to the correlation target.

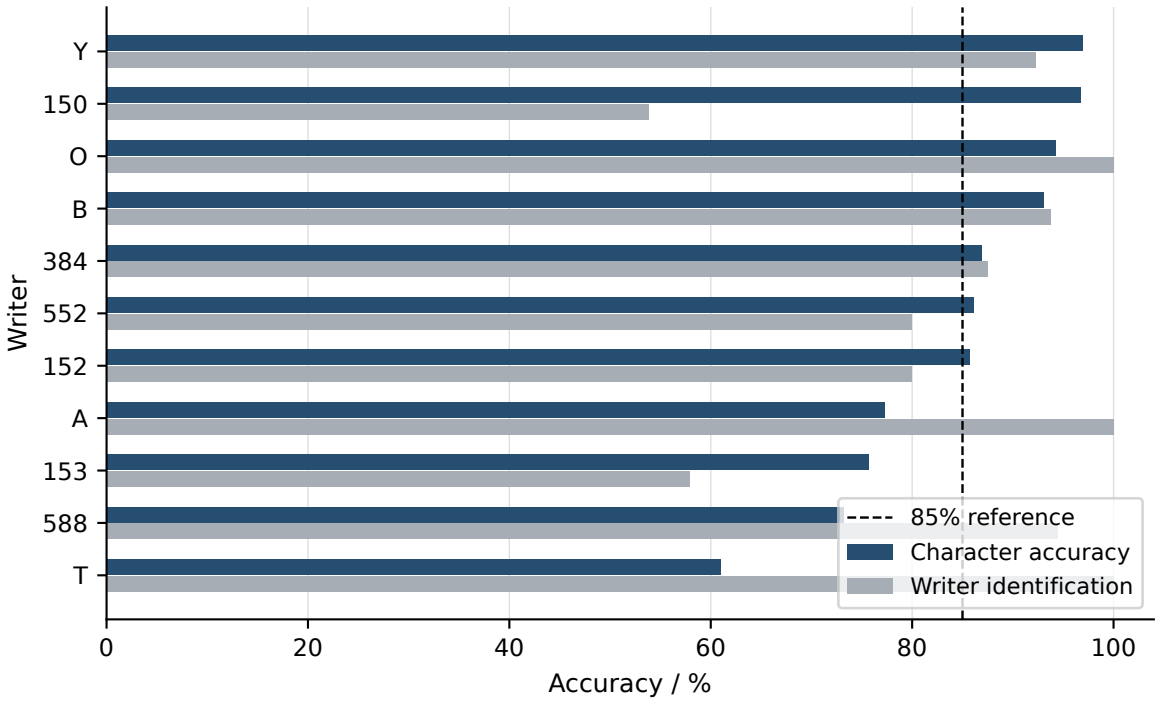


Figure 27. Physical reproduction by writer. Character accuracy and writer identification are measured separately. The dashed 85% line is the character target and a visual reference for identification; identification does not replace the proposal's correlation criterion.

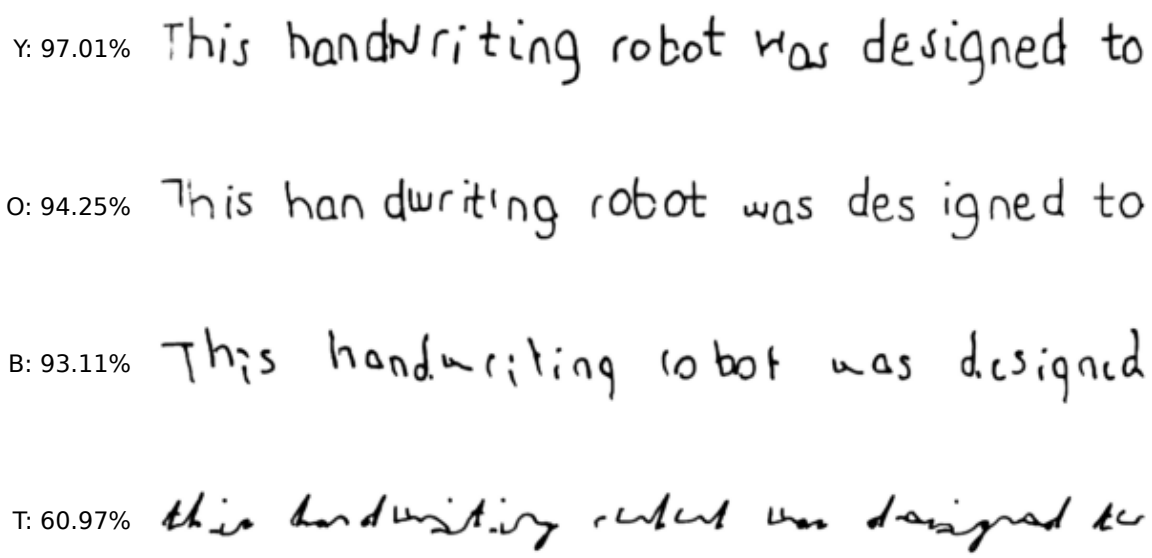


Figure 28. Stored crops from photographed robot pages for Y, O, B and T. The percentages label each writer's aggregate character score, not the accuracy of the single strip shown. These physical-output examples illustrate the range rather than constituting a new controlled same-sentence experiment.

writer identification. These are demonstrated operating cases, rather than replacements for the pooled result.

The new physical style comparison applies the same fixed descriptor definitions to the 184 reproduced lines and 156 original validation lines, grouped by the eleven writers. All lines return usable descriptors and no writer is excluded. Original and reproduced text content is not matched sentence by sentence, so the comparison concerns writer-level geometry across these samples. Nine descriptors include pen-dependent ink density and stroke width; the seven shape descriptors omit those two quantities, consistently with the constant-pen design rationale.

Table 9 gives both variants. Standardising each feature across writers separately in the original and physical domains, then correlating the flattened arrays, gives 0.713 over all nine features and 0.697 over seven shape features. With original-domain statistics used for both arrays, the corresponding values are 0.633 and 0.611. Appendix C.3 gives the extraction and individual descriptors. This is a new retrospective measurement of saved physical output, not an extension of the earlier ten-writer synthetic experiment.

Feature set	Features	Separate domains	Absolute variant
All geometric descriptors	9	0.713	0.633
Shape descriptors (ink omitted)	7	0.697	0.611

Table 9. New physical-to-original comparison across eleven writers. The first correlation standardises features separately in each domain; the absolute variant uses original-domain statistics for both. The target reference is 0.85.

The physical slant coefficient is 0.991, ascender extent 0.817, character width 0.798 and word gap 0.486. For example, writer 150's mean estimated slant changes from −5.17° in the original reference lines to −5.42° in the physical output, approximately −0.26°. Across equally weighted, separately standardised features, the pooled shape correlation is the mean of the seven per-feature correlations; their sum 4.8767 divided by seven gives 0.6967. It lies 0.1533 below the 0.85 reference.

Observations

The pooled character score is 0.59 percentage points below the 85% target. Writer scores vary from 60.97% to 97.01%. The character score and writer-identification score do not rank writers identically: T is identified on every scored line while its character score is the lowest, and writer 150 has 96.75% character accuracy with 53.85% identification. Thus the two outputs measure different properties of reproduction.

The measured components-per-character descriptor of genuine handwriting is positively associated with reproduced character accuracy ($r = 0.855$, Spearman $\rho = 0.845$, eleven writers); a lower descriptor indicates more joined handwriting. The corresponding writer-identification rank association is $\rho = -0.027$. These are cross-writer associations, not measurements of the loss caused by an individual joining operation.

Statistical analysis

The descriptive Wilson interval on 159/184 identifications is 80.7–90.6%. A 10 000-repeat resampling of the eleven whole writer groups gives a central 95% range of 77.82–90.27% for pooled character accuracy. The new physical shape coefficient has a central writer-resampling range of 0.555–0.827. These ranges reflect sensitivity to which observed writers are represented. Shared texts, profiles and capture conditions limit an interpretation as independent generalisation trials. The final targets are assessed using the observed pooled scores and their breakdowns, without claiming a universal pass from the better individual cases.

Qualification test 2: Reading handwritten content and identifying its writer (R2)

Objectives of the test or experiment

Measure the two classification outputs against the 95% target using data excluded from weight updates for the relevant network. The reader and writer model have different training populations, so their scores are reported separately rather than multiplied into an invented combined accuracy.

Equipment used

The saved IAM line corpus and photographed line crops; the evaluated scratch reader and joint-reader comparison; the 0.17-normalised eleven-writer model; and the transcript-scoring and per-line writer-classification tools on the development host. No gantry movement is required during this scoring stage.

Test setup and experimental parameters

The reader evaluation contains the distributor's 976 IAM validation lines (42 072 characters) and 100 photographed validation lines (3 675 characters). The latter use the recorded seeded 15%

line split. Transcript lookup found that many lines from IAM writers 150, 152 and 153 occur in the public training partition; their project-page scores are therefore not used as additional reader validation. The selected photographed and public validation sets are kept distinct. Validation data may have informed checkpoint selection, so these scores describe validation performance rather than a newly collected blind final test.

The writer evaluation contains 156 lines: a held-out labelled page for each of six database writers and the seeded photographed-line split. Photographed validation lines may share a writing session and page background with training lines. This is the split described in Section 3.5; its population is stated explicitly when interpreting the result.

Steps followed in the test or experiment

1. Check the reader evaluation transcripts against the public corpus partitions and retain the specified validation populations.
2. Read each validation line with greedy CTC decoding and retain the raw edit and reference counts.
3. Classify each writer-validation crop after the selected stroke normalisation; retain its reference, predicted class and confidence.
4. Aggregate by population and writer, and pool each writer's validation lines for the separate group-level result.

Results or measurements

Table 10 gives the reader scores. The scratch checkpoint yields 94.39% pooled character accuracy, while the joint reader yields 94.06% on the same population. The photographed subset yields 94.42% with the scratch reader and 93.33% with the joint reader. The author selects the library-trained joint reader for the demonstration. The comparison therefore supports the numerical implementation without substituting its result for the selected system.

Evaluation set	Lines	Characters	Joint / %	NumPy / %
Public IAM validation	976	42072	94.12	94.39
Photographed A	31	934	94.00	94.65
Photographed B	19	739	95.81	97.70
Photographed O	22	1017	94.40	94.79
Photographed T	11	345	85.51	89.28
Photographed Y	17	640	92.03	92.50
All	1076	45747	94.06	94.39

Table 10. Character accuracy of the selected library-trained joint reader and the evaluated NumPy-trained reader, using greedy CTC decoding. The public set is the distributor's validation partition, not an independent final test set.

The library-trained writer model correctly classifies 153/156 lines, or 98.08%. Pooling the available validation lines gives the correct identity for all eleven writers. The individual correct

counts are 8/9, 7/8, 9/9, 12/12, 10/10 and 8/8 for the six database writers, and 31/31, 18/19, 22/22, 11/11 and 17/17 for A, B, O, T and Y. Figure 29 gives the complete confusion matrix rather than only the overall percentage.

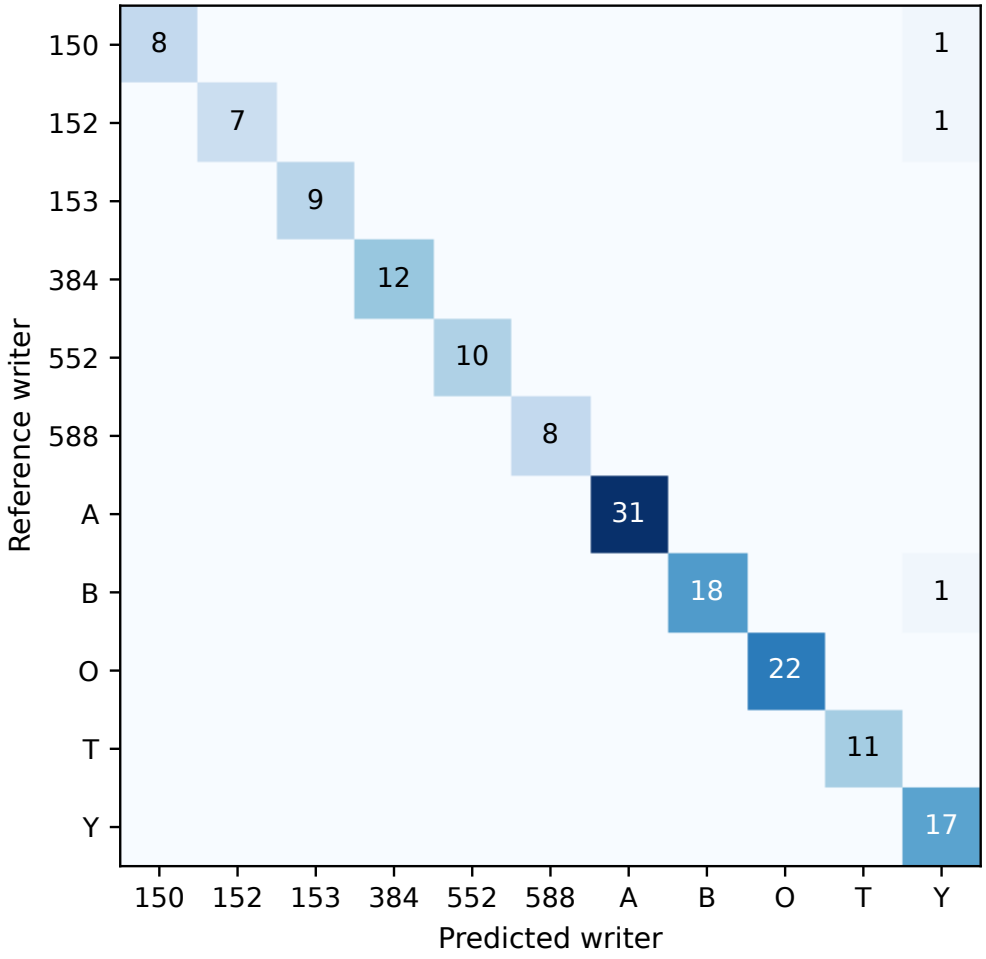


Figure 29. Writer classification on 156 validation lines. Rows are reference writers and columns are predicted writers; blank cells are zero. Three lines are misclassified, all as Y. Photographed writers use the seeded line split, while database writers use held-out pages.

Observations

The writer point estimate exceeds 95% by 3.08 percentage points. The selected reader's pooled text score is 0.94 percentage points below 95%; the strongest photographed writer's character score under the selected reader is 95.81%. The selected reader gives 94.12% on the public subset and 93.33% on photographed lines, and recovers 221/1 076 complete lines without an edit. The numerical reader's corresponding character scores are 94.39% and 94.42%, with 242 exact lines. The selected and numerical readers have pooled positional word-match scores of 72.16% and 73.47%, respectively.

The three writer errors have confidences of 44.2%, 83.8% and 39.5%. Correct lines have mean confidence 97.7%. The error with 83.8% confidence shows that a high posterior is not a guarantee of correctness. The error indices are the last entries of their writer's evaluation group;

this ordering alone does not establish a physical last-line or short-text cause for the randomly selected photographed subset.

Statistical analysis

The descriptive Wilson interval on writer accuracy is 94.5–99.3%, and the interval on 11/11 correct writer pools is 74.1–100%. The observed point estimates are strong, while the lower limits do not establish a population-wide 95% guarantee. Treating all six predictions below 80% confidence as wrong, in addition to the one higher-confidence error, leaves 149/156 correct (95.51%); this is a sensitivity calculation on the same sample, not a measured rejection policy. For character accuracy, Equation 39 is an edit ratio, so a binomial confidence interval on 45 747 independent characters is not used.

Qualification test 3: Complete reproduction time on the embedded platform (R3)

Objectives of the test or experiment

Compare complete recorded reproduction times with the proposal allowance $4x$ s. The relevant operation includes generation and physical drawing, rather than physical motion alone.

Equipment used

The ODROID N2+, gantry and server stage timer. The primary measurement is the retained server log, including program-end and stop messages. No development-host inference benchmark supplies the physical timing result.

Test setup and experimental parameters

Generation and drawing use separate requests. The drawing request carries generation stages into its recorded reproduction total, as described by the job timing boundary in Section 3.7. The primary analysis selects console records containing a drawing stage, a positive character count and an explicit successful program-end marker. Stopped jobs, zero-character commissioning shapes and unconfirmed summary-only entries are excluded from the pass count. Repeated records refer to recorded requests, not an assumed random sample of all possible text and styles.

Steps followed in the test or experiment

1. Associate a reproduction timer block with the preceding physical job's terminal status.
2. Retain explicitly completed positive-character jobs and read the recorded total and character count.
3. Calculate t/x and compare t with $4x$ for each retained record.
4. Group by text length and report the worst recorded total in each group.

Results or measurements

Table 11 and Figure 37 give 25 completion-confirmed records. Every retained total meets its numerical allowance. The largest normalised time is $166.61/43 = 3.8747$ s per character, leaving

Characters	Records	Allowance / s	Worst total / s	Worst / s per char
43	9	172	166.61	3.875
75	1	300	188.60	2.515
82	3	328	260.33	3.175
91	2	364	222.63	2.446
132	1	528	277.90	2.105
378	9	1512	1085.01	2.870

Table 11. ODROID reproduction records with explicit program-end confirmation. Generation and drawing are included in the recorded total. All 25 retained records meet their allowances.

172 – 166.61 = 5.39 s on that request. At 378 characters the nine confirmed totals range from 507.57 s to 1 085.01 s, against 1 512 s allowed.

Observations

The shortest tested length has the narrowest worst-case margin: 3.1% of its allowance. The longest tested paragraph has a worst-case margin of 28.2%. The times for the same paragraph vary by a factor of 2.14. The log includes coordinate-clamping messages in some jobs, so completion of the interpreter and satisfaction of the timer do not by themselves prove that every intended trajectory remained unchanged. Character fidelity is assessed separately in QT 1.

Statistical analysis

All 25 confirmed records pass; their descriptive Wilson interval is 86.7–100%. This is not a claim that future failure has been excluded. The experiment covers six observed lengths from 43 to 378 characters, with one to nine records per length. It provides no direct timing result for shorter requests or a probability of compliance across unseen styles.

Qualification test 4: Time to decode a photographed page and identify its writer (R4)

Objectives of the test or experiment

Compare the embedded classification request with $2 + 0.025x$ s and identify the time available for a practical improvement. Supporting acceleration experiments retain output checks alongside time measurements.

Equipment used

The ODROID server and stage timer for the two actual requests; the development host for the preserved comparison of sequential NumPy inference with batched PyTorch inference and for the image-resolution experiment. These are different timing platforms.

Test setup and experimental parameters

The ODROID log reports 500 characters over 16 lines and 982 characters over 22 lines. Their recorded totals include loading, conditioning, segmentation, cropping, reading and writer classification. Allowances below use the logged character counts; independent ground-truth counts

and any acquisition latency outside the timer are not established by those records. The combined time is the pass criterion. Grouping image and network stages helps diagnose the overrun; it does not impose an additional independent specification on each group.

Steps followed in the test or experiment

1. Extract each recorded ODROID classification total and its component stages.
2. Compute the allowance from the recorded count and compare the total with it.
3. For the separate acceleration study, process the same stored pages with the same network weights and compare time and decoded text.
4. Repeat image preparation at half linear resolution, retaining line counts, failed pages and agreement with full-resolution output.

Results or measurements

Table 12 and Figure 38 show the two ODROID requests. Neither meets R4: $46.26/14.50 = 3.19$ times the allowance and $55.49/26.55 = 2.09$ times the allowance. The network stages account for approximately 67.6% of their combined time, and image stages 32.4%.

Recorded quantity	500 characters	982 characters
Loading / s	0.63	0.53
Conditioning / s	6.61	6.04
Segmentation / s	4.29	5.59
Cropping / s	4.61	4.65
Image stages / s	16.14	16.81
Reading / s	19.02	24.66
Writer classification / s	11.09	14.01
Network stages / s	30.12	38.67
Total / s	46.26	55.49
R4 allowance / s	14.50	26.55
Overrun / s	31.76	28.94

Table 12. Two recorded ODROID classification requests. Totals are taken from the timer; rounded stage values may differ by 0.01 s when summed.

On fourteen stored pages, the development-host network experiment records 359.8 s for sequential NumPy inference and 130.3 s for equivalent library modules batched by page, a measured factor of 2.76. Decoded text agrees on every evaluated line. Both the execution backend and the batching strategy change together, so the factor cannot be assigned to batching alone. Writer-label equivalence is not established merely by identical reader text.

The image-resolution experiment finds 247 lines at full resolution and 182 at half resolution across the twelve pages that return results at both scales. None of those twelve retains its line count, and two additional reduced-resolution pages fail to return a segmentation. The reported 4.37-fold image-stage speed factor accompanies a mean text-agreement score of 33.8%, so it is not an equivalent-output replacement. These comparisons are recorded in Appendix C.5.

Observations

The observed network cost is 60.2 and 39.4 ms per logged character. Applying the host's 2.76-fold factor to those stage times would give 21.8 and 14.3 ms per character, respectively. These are useful design projections, not ODROID measurements. With unchanged image stages, even that projection gives totals of approximately 27.05 s and 30.82 s, still above the two allowances. A faster network path alone therefore does not close the demonstrated total-time gap under this projection.

Statistical analysis

Two ODROID requests establish two observed overruns; they do not establish a timing distribution or failure at every possible page length. The development-host comparison contains fourteen page conditions rather than repeated controlled ODROID runs. Its factor supports an optimisation candidate, while the embedded speed gain and complete R4 pass require direct measurement. A C or C++ implementation is not credited with an unmeasured speed-up.

Qualification test 5: Gantry dimensional accuracy and geometric consistency (R5)

Objectives of the test or experiment

Measure the drawn geometry relevant to the 0.5 mm positional target and distinguish dimensional checks from complete absolute-position and repeatability qualification.

Equipment used

The calibrated gantry, pen and paper; a metal non-digital vernier caliper, whose make, model and least count are not present in the retained record; and the generated 100 mm square and isosceles triangle of 80 mm base and nominal 40 mm height. The physical readings are author-recorded inputs, not outputs that can be regenerated by a script.

Test setup and experimental parameters

The retained measurements are from one square and one triangle after calibration. Four square sides and three triangle sides are compared with their commanded lengths. Caliper least count is not assumed from the number of digits typed in a result; reading uncertainty also depends on finite-width pen marks. The geometry provides a useful commissioning check for the coordinate and stepping model in Section 3.3.6.

Steps followed in the test or experiment

1. Home and calibrate the gantry, then execute the square and triangle jobs.
2. Measure each reported side and calculate its difference from the nominal length.
3. Compare the dimensional differences with 0.5 mm and calculate the mean square-side error.
4. Use the unequal triangle sides to estimate an apparent apex offset under the stated geometric assumptions.

Results or measurements

Table 13 retains all seven original readings. Figure 39 shows their signed length errors. The four square sides have a mean absolute error of $(0.30+0.20+0.10+0.10)/4 = 0.175$ mm and maximum 0.30 mm. The largest absolute feature-length difference across both shapes is $|56.20 - \sqrt{40^2+40^2}| = 0.369$ mm.

Shape	Feature	Nominal / mm	Measured / mm	Error / mm
Square	Bottom X	100.00	100.30	+0.30
Square	Top X	100.00	100.20	+0.20
Square	Left Y	100.00	100.10	+0.10
Square	Right Y	100.00	99.90	-0.10
Triangle	Base	80.00	80.30	+0.30
Triangle	Left slope	56.57	56.20	-0.37
Triangle	Right slope	56.57	56.90	+0.33

Table 13. Author-recorded dimensions from one calibrated square and one triangle. These are feature-length errors, rather than an absolute coordinate-error map.

Place the measured base endpoints at $(-b/2, 0)$ and $(b/2, 0)$ and the apex at (d, h). Subtracting the two squared-distance equations gives the project's geometric diagnostic:

$$d = \frac{\ell^2 - r^2}{2b}, \qquad \phi_{\text{app}} = \arctan(d/h). \tag{40}$$

Using $b = 80.30$ mm, $\ell = 56.20$ mm, $r = 56.90$ mm and the nominal $h = 40$ mm gives $d = -0.493$ mm and $\phi_{\text{app}} = -0.706°$. The angle is conditional on the assumed height and a shear interpretation; it is not an independent measurement of rail squareness. Both an ideal square and a rhombus can have equal side lengths, which is why diagonal or coordinate measurements would add information absent from the square-side test.

Observations

Every measured feature length is within 0.5 mm of nominal. The two horizontal square sides are long by 0.20–0.30 mm, while the vertical sides straddle nominal by 0.10 mm. The unequal sloping triangle sides are consistent with lateral geometric asymmetry. The measurements do not isolate scale, shear, pen width and reading uncertainty into separate physical causes.

Statistical analysis

There is one specimen of each shape. The spread among sides is not repeated-draw variability, and no standard deviation of repeatability is assigned. Side lengths alone also leave a rigid translation undetected, so the 0.5 mm absolute point-position requirement is not fully proven. The positive result is a dimensional error below the numerical limit on every measured feature; repeated word overlays and bed-coordinate measurements would complete the distinction requested in the proposal.

Qualification test 6: Storage and use of multiple trained styles (R6)

Objectives of the test or experiment

Confirm at least ten represented writers, stored glyph mappings and the ability to use those mappings for reproduction.

Equipment used

The evaluated eleven-class writer checkpoint, stored style profiles and glyph libraries, saved generated jobs and fourteen physical output pages.

Test setup and experimental parameters

The required count is ten. The evaluated set is 150, 152, 153, 384, 552, 588, A, B, O, T and Y. Both class identities and stored trajectory libraries are checked.

Steps followed in the test or experiment

Compare the classifier mapping with the stored profiles, then confirm that every identity occurs in the writer-validation records, physical pages and generated jobs.

Results or measurements

Eleven identities have classifier entries and stored profiles. All have physical samples in Figure 27 and correct pooled identities in QT 2, exceeding the required count by one.

Observations

The result demonstrates concurrent use of multiple stored styles; QT 1 measures their different reproduction quality.

Statistical analysis

This is an inventory count, so a sampling interval is inapplicable. R6 is met by the evaluated eleven-style set.

4.2.2 Qualification under the proposal's field conditions

Indoor captures and physical drawings demonstrate operation in the fixed test arrangement. The author reports successful operation across lighting conditions wider than the planned range, but no illuminance readings or surface-level/vibration measurements accompany the retained records. The evidence supports the observed bench conditions and capture examples in Section 3.4.7; it does not numerically establish the 400–1000 lux endpoints. Table 7 compares the retained evidence with the proposal's exact environmental conditions.

5. Discussion

5.1 Critical evaluation of the design

The project connects photographed handwriting, numerical recognition, persistent writer profiles and a working Cartesian gantry. The results support this central function across eleven writers, with quality depending on the handwriting and embedded classification time remaining the principal performance constraint. This evaluation relates those outcomes to the decisions explained in Design.

5.1.1 Interpretation of results

Useful reproduction is demonstrated, with a clear operating range. The 84.41% read-back character score is close to the 85% target, but its engineering meaning becomes clearer when the eleven writers are separated. Seven exceed 85% and four exceed 93%; those four include both personal contributors and a database writer. Thus the method is capable of substantially better than the pooled target on demonstrated styles. The lower scores on other writers show that the remaining improvement concerns consistency across styles, rather than an inability of the gantry to produce readable text at all. Figure 28 grounds this distinction in physical output.

The association between components per character and legibility is consistent with the limitations of the chosen representation. A glyph library stores observed character shapes and assembles them into new contexts. In more joined handwriting, the appearance of a character can depend strongly on its neighbours, so a cut glyph and a generic connection need not reproduce the same local shape. The observed rank association of 0.845 supports investigating this mechanism, but eleven writers do not isolate joining from glyph coverage, input quality or reader preference. The corresponding identification association is close to zero; it shows that a writer can remain identifiable even where individual characters are difficult for the reader.

The 86.41% line-identification score and correct attribution of all fourteen page pools are useful evidence that writer-specific structure survives the full physical process. They answer a different question from stylistic correlation. The new physical geometric study gives seven-feature shape correlation 0.697 and nine-feature correlation 0.713, both below 0.85. It also identifies useful retained structure: slant tracks at 0.991, while ascender extent and character width track at approximately 0.817 and 0.798. Word spacing, at 0.486, is a weaker channel. These values motivate improving layout and context while preserving the successful orientation and glyph information; they do not establish that layout is the sole source of loss.

The writer model also participates in candidate selection, so it cannot be treated as an independent aesthetic judge of that selection. The geometric study uses no neural classifier, although it still depends on its descriptor definitions and the different original and reproduced texts. Its separately standardised coefficient primarily describes cross-writer ordering, and the lower 0.611 absolute shape variant shows that agreement of feature magnitudes is more demanding. Automated read-back is likewise an instrument-dependent measure of legibility. It provides a repeatable comparison, while a blinded human reading and style assessment would test the human-facing

interpretation of the proposal more directly. Neither the physical overlay nor a human rating is inferred from model confidence.

The classifier results support the chosen architecture. The selected joint reader achieves 94.06% pooled character accuracy, 0.94 percentage points below the 95% target; the NumPy-trained scratch reader achieves 94.39% on the same evaluation. The scratch reader's public and photographed subsets score 94.39% and 94.42%, while the joint reader scores 94.12% and 93.33%. These results show that the line-recognition architecture works on both distributor-cut IAM lines and the project's photographed material. Similar averages do not prove that segmentation is harmless: different text, writers and image conditions may compensate in the mean, and the study does not measure the segmentation contribution separately.

The 98.08% writer-identification point estimate and correct pooling of all eleven writer groups are particularly strong outcomes for the multi-writer objective. The selected 0.17 stroke width makes this result relevant to a pen plotter that cannot reproduce each contributor's original ink thickness. The three errors also define the limits of the confidence output: one incorrect classification has an 83.8% posterior, so confidence can guide review but cannot guarantee an identity. The evaluation is closed set, and its photographed split shares acquisition conditions with the training material; a new session with the same writers would be a stronger deployment check.

The selected library reader is retained as the intended demonstration configuration, while the alternative reader result is reported alongside it. This is a configuration decision rather than evidence that the library route necessarily gives the higher text score. On these data the scratch reader is slightly higher; for writer identification the library-trained model is higher. Keeping both comparisons visible gives a more useful assessment of the implementation than a single claim that one training route is uniformly superior.

The physical timing result is a practical success. All 25 explicitly completed positive-character ODROID records satisfy 4 s per character, including generation and drawing. The worst request still has 5.39 s of allowance remaining. This demonstrates usable physical throughput across the recorded 43–378-character range, rather than relying on the G-code generator's nominal feed calculation. That distinction is important because pen transitions, path length, GPIO scheduling and generation time all contribute to the request.

The shortest block has the smallest worst-case margin. Generation work is incurred before the drawing begins, while the allowance grows directly with character count. The measured worst short job includes 26.5 s of synthesis and approximately 3.248 s per character of drawing. Treating those two terms as fixed overhead plus a constant drawing rate would put a break-even length near 35 characters, but this is only a local projection. Search effort, repair and geometry change with the text, so the result motivates a direct short-request test rather than a claim that all shorter requests fail. Coordinate clamping in some recorded jobs also prevents the timing pass being used as a substitute for the separate fidelity result.

The classification-time gap is measurable and actionable. The two ODROID classification requests exceed their allowances by 31.76 s and 28.94 s. Approximately two thirds of the combined time is spent in the networks, with the remaining third in image processing. This makes network execution the largest observed optimisation opportunity, while also showing that segmentation alone cannot explain the total overrun. A two-request profile identifies the large contributors; it does not establish a constant cost law for arbitrary pages.

The development-host experiment supplies a concrete alternative: equivalent library networks batched by page are 2.76 times faster in the recorded comparison, with the reader text unchanged. It is reasonable to investigate that route on the ODROID. It is not yet reasonable to report the transferred factor as an embedded timing measurement, because the processor, libraries, thread scheduling and memory behaviour affect it. Even the projection with unchanged image processing leaves the two complete requests above their allowances. The required engineering work is therefore a coordinated reduction of network and image-stage cost, while retaining the output checks.

Mechanical accuracy is promising, with a precise distinction in what was measured. Every measured square and triangle feature is within 0.5 mm of nominal, and the square's mean absolute side error is 0.175 mm. These are useful results for the calibrated belt-and-rail mechanism. The triangle's unequal sloped sides additionally suggest an alignment effect that a side-length-only square check cannot expose. The apparent 0.706° shear diagnostic offers a direction for adjustment, conditional on the assumed height and other geometric errors.

Neither equal side lengths nor small side-length errors fix the figure's absolute placement on the page. A translated figure can retain the same lengths, and repeated drawings can vary even when one specimen is accurate. The dimensional result should therefore be credited for what it demonstrates: a physically realised coordinate-to-motion path with sub-0.5 mm length differences on the measured features. A repeated-word overlay and a referenced bed-coordinate grid would extend that evidence to the proposal's positional and repetition objectives.

Relationship to published approaches. The sequence reader follows the CNN–BiLSTM–CTC architecture discussed by Kızılırmak and Yanıkoğlu [12, pp. 4–5]. The project's measured CER is 5.94% for the selected reader and 5.61% for the scratch reader, obtained directly from Equation 39. This is evidence of a functioning implementation in the same task family. It is not a controlled ranking against a published test result: the distributor's validation partition, training mixture, checkpoint selection and decoding protocol are not a matched published test protocol. In particular, the positional word-match score here is not directly comparable with published WER.

The reproduction strategy has a closer practical relationship to example-based approaches. Haines et al. demonstrate synthesis using a small collection of handwritten pages and explicit extracted forms [23, p. 4]. That supports the use of a small personal dataset as a design rationale without establishing an identical output quality. Graves's sequence generator uses ordered pen-coordinate observations [3, p. 18]; this project starts with offline photographs and does not observe those temporal paths. Modern image generators can model richer context, including One-DM's use of a single reference at generation time, but that reference count must be distinguished from the corpus and hardware used to train the generator [22, p. 9]. These comparisons explain the selected scope: an inspectable geometric system built from the available data, with context modelling as its principal reproduction trade-off.

5.1.2 Critical evaluation

Separating the implementation contribution from checkpoint selection. The project implements convolution, normalisation, recurrent operations, CTC and optimiser updates numerically, as documented in Section 3.5.6. It also has a framework route for training the corresponding

architecture. Choosing the library-trained writer for the demonstration does not remove the first-principles work; the numerical checks and trained-model comparison provide separate evidence for it. Conversely, the existence of a numerical trainer does not establish that every selected checkpoint came from that trainer.

Table 14 compares the saved models on common material. The numerical writer model exceeds the 95% target as an observed line-level result, reaching 149/156 or 95.51%, while the library-trained model reaches 153/156 or 98.08%. On the 185-crop physical-output comparison the difference is larger, 80.54% against 85.95%. Both numerical and library checkpoints therefore learn useful writer distinctions, with the selected model providing the stronger measured writer result on these two populations. The 185-crop comparison is kept distinct from the 184 paired lines of QT 1.

Common evaluation	Library trained	NumPy trained
Writer: original validation lines	153/156 (98.08%)	149/156 (95.51%)
Writer: physical-output crops	159/185 (85.95%)	149/185 (80.54%)
Writer: original groups correct	11/11	11/11
Writer: physical writer groups correct	11/11	10/11
Reader: pooled character accuracy	94.06%	94.39%

Table 14. Saved trained-model comparison. The reader row compares the joint and evaluated scratch checkpoints. The writer rows use matched crops and the same 0.17 preprocessing; these are comparisons of particular trained models, not an isolation of training precision or framework effects.

The supplied numerical-writer training log records 1 031 training and 155 validation lines, a best displayed validation accuracy of 0.968 by epoch 41, and approximately 19–23 minutes per reported epoch. The reader excerpt records approximately 111–122 minutes per reported epoch and a resume from an existing numerical checkpoint. The later writer re-evaluation has 156 lines because the extraction and transcript correspondence used in that evaluation differs from the training-era population. Its 95.51% result should therefore not be substituted for the 96.8% training-validation figure or vice versa.

The reported float32 training choice is consistent with the need to limit the cost of these numerical runs, but this experiment does not isolate precision as the cause of an accuracy difference. Higher arithmetic precision does not, by itself, establish a higher trained-model classification score. A controlled precision study would require the same initial weights, sample order, preprocessing, optimiser settings and stopping rule. The existing fixed-parameter numerical checks support the implementation's arithmetic separately from these trained-model comparisons.

Line recognition addresses a real weakness in character isolation. The proposal described separating characters or connected groups before classification. The implemented reader instead recognises lines, while known-transcript CTC alignment recovers character boundaries during profile construction. This was a defensible response to connected handwriting: uncertain character cuts are not made a prerequisite for reading the text, and CTC supports training from complete transcripts [14, pp. 2–3]. The cost is a larger sequence model and continued dependence on correct line ownership. The Results therefore qualify the final line-based design while retaining the functional contribution promised in the proposal.

The glyph library fits the data and exposes its own limitations. An explicit library permits size, slant, spacing and machine coordinates to be examined directly. It also allows the same stored writer to be used for different text without fitting a new generator for each request. These advantages are borne out by eleven functioning profiles and the different physical outputs. The price is a finite set of observed shapes, with limited information about how a writer changes a letter between neighbours. Multiple-character units or context-conditioned variants would address that loss more directly than increasing candidate count alone. They would also need reliable extraction and enough examples of the relevant combinations.

The first-principles and embedded-processing commitments of the proposal also constrain the interpretation of a library speed remedy. A framework benchmark is a useful engineering comparison; it does not by itself establish compliance with the original library restriction. A batched numerical implementation could retain that contribution, while compiled kernels for the identified hot loops offer a second route. Neither has an embedded compliance result in the present measurements. The practical workflow also separates off-device training from embedded operation. The supplied training records substantiate the numerical implementation and the cost of model preparation, while the proposal's intended embedded training mode is not demonstrated by those off-device runs. This distinction keeps the substantial implementation contribution and the final operating arrangement clear.

Economic feasibility. The author's current build-cost estimate is approximately R4 000. The ODROID was borrowed at no purchase cost and will be returned after the project. This reduces the prototype's cash expenditure, but a replacement board would be part of a reproducible deployment cost. The ledger and subsequent purchases require reconciliation before a precise total is claimed; the later listed clipboard, paper, pen and estimated training electricity amount to R750 and may overlap the approximate estimate. The electricity amount is an author estimate, not an energy-meter result.

Using the available board allowed the project to prioritise the complete mechanical and image-processing system. The cost choice is relevant to the available compute, although the measurements do not show what a different board would have achieved. The 2.76-fold development-host comparison is stronger evidence for a software optimisation opportunity than an unsupported assertion that a more expensive platform would necessarily meet R4. A future cost comparison should include hardware, development effort and the measured quality retained by each acceleration route.

5.1.3 Unsolved problems

The main reproduction problem is the variation between writers. Improvements should target the poorer cases without losing the high-performing printed and lightly joined styles. A useful next experiment would keep the reader, writer model and input text fixed while comparing current single-glyph paths with selected multi-character units. That would test the proposed context explanation directly. The new geometric study establishes a physical-output baseline; matching original and reproduced sentences and repeating capture sessions would better separate writer fidelity from content and acquisition effects.

The timing problem requires equivalent-output optimisation. The reduced-resolution experiment is valuable precisely because it measures the cost: it returns fewer lines on every comparable page and fails to segment two pages. It therefore cannot be accepted simply because its projected

total may fall below an allowance. The better sequence is to preserve the reference output, measure the ODROID's hot loops and memory use, then compare a batched numerical route or selected compiled routines against that reference. The stopping criterion should be the complete request allowance together with unchanged transcripts and an agreed writer-output check.

The remaining mechanical work is a stronger positional qualification rather than an unsupported redesign. Frame squaring, belt consistency and pen-holder stiffness are plausible sources of geometric error, but the present length measurements do not rank them conclusively. Measuring repeated grid points, both square diagonals and the proposal's repeated word would identify where calibration is sufficient and where a mechanical adjustment is needed. A more controlled pulse scheduler and acceleration profile are sensible extensions of the motion design, but their benefit must be assessed on completed drawings rather than inferred from commanded feed.

The model files, preprocessing width and stored writer mappings must also be carried into the demonstration as one identified configuration, so that a changed checkpoint does not silently change the meaning of the report's results.

5.1.4 Strong points of the design

The functioning chain is a principal strength: the system captures pages, recognises text, identifies writers, stores trajectories and produces real handwriting. Eleven styles are exercised in recognition and physical reproduction, seven exceed the reproduction character target, and all eleven writer-validation groups are identified correctly when pooled. These results substantiate the multi-writer objective.

An output can be followed through its source crop, normalised glyph, placement and emitted command. The independent G-code checks and physical measurements support this inspectable separation, allowing an improvement to be tested at a known interface while retaining the successful stages.

Operation-level checks and trained results support the numerical implementation: the numerical writer reaches 95.51% on the common evaluation, and the numerical reader is competitive with the joint reader. The framework route supplies a practical reference and the stronger selected writer model. Together they support the implementation contribution and the final model choice.

5.1.5 Expected failure conditions

Strong cursive connections, scarce character variants and text outside the available alphabet are the most relevant reproduction limits. A profile can preserve only the geometry it contains or the approximations selected for missing characters. Excessive layout extent can also cause clipping or coordinate clamping. A request should therefore be inspected in the preview and validated against the writing area before physical execution; a completed interpreter run alone is not proof of an unchanged trajectory.

For reading, unusually sparse pages, touching lines, page curvature and strong perspective changes can defeat the geometric preparation even when a cropped-line network is accurate. The Design's real capture examples illustrate these conditions. The writer classifier always chooses among its eleven represented identities, so an unfamiliar writer can receive a confident

known-writer label. Short or ambiguous lines provide less evidence, while the present confidence sample does not establish a universally safe rejection threshold. Page pooling improves the observed group result but cannot recover information absent from every crop.

The gantry uses open-loop step counts. A missed step changes the physical position without an encoder measurement of that displacement. Limit switches and software abort handling provide useful boundary and stop functions, but depend on the installed switches, electrical circuit and running controller. The installed pen-motor circuit's lack of a flyback diode is a hardware limitation already identified in Section 3.3.4; repeated reliable operation should not be treated as proof that the switching transient is harmless.

Finally, classification requests outside the tested sizes and very short writing requests have no demonstrated timing envelope. The measured results describe the recorded workload and platform, not all possible text, page sizes or concurrent users. This defines a practical laboratory prototype whose demonstrated successes can be repeated with an identified configuration, and whose extensions have specific measurements to target.

5.2 Considerations in the design

5.2.1 Ergonomics

The browser interface places text entry, captured-page review, writer selection and output preview on the operator's normal screen. Calibration and a writing-area indication reduce setup ambiguity, while the preview permits inspection before movement. Network operation is the selected arrangement; the ODROID also has a display interface [30].

The page and pen still require manual positioning, and the motion area must remain accessible for loading while being kept clear during execution. A more complete user flow would confirm the paper restraint, valid calibration, selected profile and preview bounds before a job, and clearly distinguish generation, running, stopped and completed states. These are consequences of combining a software interface with a moving mechanism, rather than cosmetic interface additions.

5.2.2 Health and safety

The moving carriage, pulleys and belts present pinch points, and a protruding pen or tool can contact a hand in the working area. The installed end stops, limits, abort handling and controlled setup procedure reduce exposure during normal laboratory operation. The software stop is useful operational control; independent removal of motor power would provide protection if the controller or network became unresponsive. The driver and transistor temperature, wiring restraint and supply enclosure also require attention because the motor circuits carry the actual load currents.

The 24 V motor rail and 3.3 V logic are separate from the mains inputs of the purchased supplies. Low internal voltage does not establish mains insulation or enclosure quality. The supply labels and installed circuit should therefore be part of the final hardware record. A flyback path for

the pen motor is a specific improvement supported by the switching analysis already given in Design [32]; it is not described as a fitted protection component.

5.2.3 Environmental impact

The borrowed ODROID will be returned after the project. Reusable motors, rails, drivers and replaceable printed parts allow components to be repaired or used in other work. The printed-part material is not identified in the retained record, so no material-specific recycling claim is made.

Paper, pens and training electricity are the main recurring consumables identified by the author. Previewing jobs and testing geometry digitally can reduce wasted paper, while real physical tests remain necessary to establish performance. The reported R500 training-electricity allowance is a cost estimate; no total energy or environmental saving is calculated from it without power and runtime measurements. Electronics, plastics and remaining consumables should be separated and handled through appropriate local collection routes at end of life.

5.2.4 Social and legal impact

The intended benefit is personalised writing from digital text, including possible use by someone who cannot comfortably produce their own handwriting. The present system demonstrates the technical mechanism; accessibility or clinical benefit has not been evaluated with users. Reproducing another person's style also creates a risk of impersonation. Samples and profiles should be used with permission, and generated material should be identified as machine-produced where its origin matters.

The author confirms that all five personal contributors signed consent forms. Those signatures should be retained securely as supporting records; publishing them in the report would unnecessarily expose identifying material. Identifiable handwriting and writer profiles require consideration under South Africa's Protection of Personal Information Act, including minimal collection, a lawful processing basis, retention limits and security safeguards [40, pp. 8–9, 11]. Consent is one lawful basis and does not itself establish every aspect of compliance. The applicable database terms must also accompany use of the IAM material. The local-network server and stored profiles should have access restricted to the intended operator; the prototype is not presented as a publicly secured handwriting service.

5.2.5 Ethics clearance

Signed consent was obtained from the five personal contributors. No institutional clearance or exemption reference is included in the retained records; participant consent is not presented as evidence of institutional approval.

6. Conclusion

6.1 Summary of the work completed

6.2 Summary of observations and findings

6.3 Contribution

6.4 Suggestions for future work

7. References

[1] A. Chen, “Handwritten text classification based on convolutional neural network,” *Highlights in Science, Engineering and Technology*, vol. 34, pp. 39–44, 2023.

[2] J. Ma, “A study on a hybrid CNN-RNN model for handwritten recognition based on deep learning,” in *Proc. 1st Int. Conf. Data Analysis and Machine Learning (DAML)*, 2023, pp. 268–273.

[3] A. Graves, “Generating sequences with recurrent neural networks,” arXiv preprint arXiv:1308.0850, 2013. [Online]. Available: https://arxiv.org/pdf/1308.0850

[4] U.-V. Marti and H. Bunke, “The IAM-database: an English sentence database for offline handwriting recognition,” *Int. J. Document Analysis and Recognition*, vol. 5, no. 1, pp. 39–46, 2002.

[5] G. R. Hemanth, M. Jayasree, S. K. Venii, P. Akshaya, and R. Saranya, “CNN-RNN based handwritten text recognition,” *ICTACT Journal on Soft Computing*, vol. 12, no. 1, pp. 2457–2463, Oct 2021.

[6] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[7] D. Bradley and G. Roth, “Adaptive thresholding using the integral image,” *Journal of Graphics Tools*, vol. 12, no. 2, pp. 13–21, 2007. [Online]. Available: https://people.scs.carleton.ca/~roth/iit-publications-iti/docs/gerh-50002.pdf

[8] W. Postl, “Detection of linear oblique structures and skew scan in digitized documents,” in *Proc. 8th Int. Conf. Pattern Recognition (ICPR)*, 1986.

[9] A. Vinciarelli and J. Luettin, “A new normalization technique for cursive handwritten words,” *Pattern Recognition Letters*, vol. 22, no. 9, 2001.

[10] S. Avidan and A. Shamir, “Seam carving for content-aware image resizing,” *ACM Transactions on Graphics*, vol. 26, no. 3, pp. 10:1–10:9, 2007. [Online]. Available: https://www.cs.jhu.edu/~misha/ReadingSeminar/Papers/Avidan07.pdf

[11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. [Online]. Available: https://leon.bottou.org/papers/lecun-98h

[12] F. Kizilirmak and B. Yanikoglu, “CNN-BiLSTM model for English handwriting recognition: comprehensive evaluation on the IAM dataset,” arXiv preprint arXiv:2307.00664, 2023. [Online]. Available: https://arxiv.org/pdf/2307.00664

[13] B. Shi, X. Bai, and C. Yao, “An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.

[14] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, 2006, pp. 369–376. [Online]. Available: https://www.cs.toronto.edu/~graves/icml_2006.pdf

[15] S. Ioffe and C. Szegedy, “Batch normalization: accelerating deep network training by reducing internal covariate shift,” in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456. [Online]. Available: https://proceedings.mlr.press/v37/ioffe15.pdf

[16] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: https://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf

[17] C. Wigington, S. Stewart, B. Davis, B. Barrett, B. Price, and S. Cohen, “Data augmentation for recognition of handwritten words and lines using a CNN-LSTM network,” in *Proc. 14th IAPR Int. Conf. Document Analysis and Recognition (ICDAR)*, vol. 1, 2017, pp. 639–645.

[18] M. Bulacu and L. Schomaker, “Text-independent writer identification and verification using textural and allographic features,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 4, pp. 701–717, 2007.

[19] S. He and L. Schomaker, “FragNet: writer identification using deep fragment networks,” *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 3013–3022, 2020.

[20] A. A. Brink, J. Smit, M. L. Bulacu, and L. R. B. Schomaker, “Writer identification using directional ink-trace width measurements,” *Pattern Recognition*, vol. 45, no. 1, pp. 162–171, 2012. [Online]. Available: https://www.ai.rug.nl/~mbulacu/brink-quill-pr2012.pdf

[21] L. Kang, P. Riba, Y. Wang, M. Rusiñol, A. Fornés, and M. Villegas, “GANwriting: content-conditioned generation of styled handwritten word images,” in *Computer Vision – ECCV 2020*. Springer, 2020, pp. 273–289.

[22] G. Dai, Y. Zhang, Q. Ke, Q. Guo, and S. Huang, “One-DM: one-shot diffusion mimicker for handwritten text generation,” in *Computer Vision – ECCV 2024*, ser. Lecture Notes in Computer Science. Springer, 2024. [Online]. Available: https://www.ecva.net/papers/eccv_2024/papers_ECCV/papers/07598.pdf

[23] T. S. F. Haines, O. Mac Aodha, and G. J. Brostow, “My text in your handwriting,” *ACM Transactions on Graphics*, vol. 35, no. 3, pp. 26:1–26:18, 2016. [Online]. Available: https://reality.cs.ucl.ac.uk/projects/haines/haines16handwriting.pdf

[24] O. Teikari, “Reproducing handwriting with position-controlled robots,” Master’s thesis, Tampere University of Technology, Tampere, Finland, 2016.

[25] D. H. Douglas and T. K. Peucker, “Algorithms for the reduction of the number of points required to represent a digitized line or its caricature,” *Cartographica*, vol. 10, no. 2, pp. 112–122, 1973.

[26] T. Y. Zhang and C. Y. Suen, “A fast parallel algorithm for thinning digital patterns,” *Communications of the ACM*, vol. 27, no. 3, pp. 236–239, 1984. [Online]. Available: https://agcggs680.pbworks.com/f/Zhan-Suen_algorithm.pdf

[27] IQS Directory, “Types of linear actuators,” IQS Directory Technical Library, 2024, [Online]. Available: https://www.iqsdirectory.com/articles/linear-actuator/types-of-linear-actuators.html.

[28] P. P. Acarnley, *Stepping Motors: A Guide to Theory and Practice*, 4th ed. London, U.K.: Institution of Electrical Engineers, 2002.

[29] Allegro MicroSystems, “A4988 DMOS microstepping driver with translator and overcurrent protection,” Datasheet, rev. 8, 2022, [Online]. Available: https://www.allegromicro.com/-/media/files/datasheets/a4988-datasheet.pdf.

[30] Hardkernel, “ODROID-N2+ single-board computer,” Hardware technical reference, [Online]. Available: https://www.hardkernel.com/shop/odroid-n2-with-4gbyte-ram-2/ [Accessed: Oct. 31, 2026].

[31] ON Semiconductor, “BC337/BC337-40 NPN general-purpose amplifier transistor,” Datasheet, [Online]. Available: https://www.onsemi.com/download/data-sheet/pdf/bc337-fsc-d.pdf.

[32] ROHM Semiconductor, “NPN transistor: Low-side switch fundamentals,” TechWeb, Transistors Basic, 2026, 27 February 2026; accessed 1 November 2026. [Online]. Available: https://techweb.rohm.com/product/transistors-diodes/transistors/27354/

[33] Joint Committee for Guides in Metrology, “Evaluation of measurement data—guide to the expression of uncertainty in measurement,” JCGM 100:2008, 2008.

[34] J. E. Bresenham, “Algorithm for computer control of a digital plotter,” *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965. [Online]. Available: https://janmr.com/files/papers/bresenham65.pdf

[35] R. M. Haralick, S. R. Sternberg, and X. Zhuang, “Image analysis using mathematical morphology,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 9, no. 4, pp. 532–550, 1987. [Online]. Available: https://www.haralick.org/journals/04767941.pdf

[36] A. Rosenfeld and J. L. Pfaltz, “Sequential operations in digital picture processing,” *Journal of the ACM*, vol. 13, no. 4, pp. 471–494, 1966. [Online]. Available: https://www.cs.virginia.edu/~jlp/66.sequential.op.pdf

[37] K. Chellapilla, S. Puri, and P. Simard, “High performance convolutional neural networks for document processing,” in *Proc. 10th Int. Workshop on Frontiers in Handwriting Recognition (ICFHR)*, 2006.

[38] PyTorch contributors, “CTCLoss: reduction and alignment-loss conventions,” PyTorch documentation, accessed 1 November 2026. [Online]. Available: https://docs.pytorch.org/docs/2.15/generated/torch.nn.CTCLoss.html

[39] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[40] Republic of South Africa, “Protection of personal information act 4 of 2013,” 2013, official consolidated text. [Online]. Available: https://www.justice.gov.za/legislation/acts/2013-004.pdf [Accessed: 1 November 2026].

[41] T. Tieleman and G. Hinton, “Lecture 6.5—RMSprop: divide the gradient by a running average of its recent magnitude,” COURSERA: Neural Networks for Machine Learning, 2012. [Online]. Available: https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf

[42] D. P. Kingma and J. Ba, “Adam: a method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/pdf/1412.6980

[43] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. 7th Int. Conf. Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/pdf/1711.05101

[44] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” in *Proc. 30th Int. Conf. Machine Learning (ICML)*, 2013, pp. 1310–1318.

A. Supporting implementation calculations

The main Design section explains the engineering decisions and representative operations. This appendix records the derivative conventions, image operations and parameter meanings needed to inspect the numerical implementation in more detail. Algorithms in the main text are explanatory pseudocode; the separately submitted source remains the executable implementation.

A.1 Numerical network derivatives

Applying the backpropagation chain rule [11, pp. 3–4] to a linear layer $Y = XW^T + b$ with incoming derivative G, the implemented derivatives are $\partial\mathcal{L}/\partial X = GW$, $\partial\mathcal{L}/\partial W = G^T X$ and the sum of G over samples for the bias. Convolution applies the corresponding matrix calculation to overlapping image patches as described in Equation 47. Contributions to the same input pixel are summed.

For each batch-normalisation channel, $N = BHW$ values contribute to the batch statistics. The training forward pass uses population variance, while the running variance update uses its $N/(N-1)$ corrected estimate. The running update momentum is 0.1. With $\widehat{x} = (x-\mu)/\sqrt{\sigma^2+\varepsilon}$ and $g = \partial\mathcal{L}/\partial y$, Equation 41 gives the input derivative, obtained by simplifying the batch-normalisation chain-rule expressions [15, p. 4].

$$\frac{\partial\mathcal{L}}{\partial x_i} = \frac{\gamma}{N\sqrt{\sigma^2+\varepsilon}}\left[Ng_i - \sum_j g_j - \widehat{x}_i \sum_j g_j \widehat{x}_j\right]. \tag{41}$$

The parameter derivatives accompanying Equation 41 are $\sum_i g_i \widehat{x}_i$ for γ and $\sum_i g_i$ for β. Inference uses stored statistics and requires no batch-dependent update.

ReLU passes an incoming derivative only where its cached input was positive. Spatial max pooling scatters it to the selected maximum in each window. Height-max pooling does the same along height; width averaging distributes the derivative equally over the pooled columns. The implementation uses inverted dropout: it multiplies by the cached Bernoulli mask divided by $1-p$ during training and is the identity at inference. This moves the expected-output scaling from test time to training time relative to the original presentation [16, pp. 1931, 1934]. Reusing the same mask in the backward calculation is essential.

The LSTM backward pass differentiates the implemented recurrence in Equation 25; full back-propagation through time for this cell form is described by Graves [3, pp. 4–5]. It carries both the hidden derivative from the next step and a separate cell derivative. Writing the combined hidden derivative as dh_t, Equation 42 shows the dependencies before gate-activation derivatives are applied.

$$\begin{aligned} dc_t &= dc_t^{\text{next}} + dh_t \odot o_t \odot [1 - \tanh^2(c_t)], \\ do_t &= dh_t \odot \tanh(c_t), & df_t &= dc_t \odot c_{t-1}, \\ di_t &= dc_t \odot g_t, & dg_t &= dc_t \odot i_t, \\ dc_{t-1} &= dc_t \odot f_t. \end{aligned} \tag{42}$$

Sigmoid gates multiply these derivatives by $a(1-a)$; the candidate multiplies by $1-g_t^2$. The concatenated gate derivatives accumulate outer products with x_t and h_{t-1} into the two weight matrices. Multiplication by W_h provides the next reverse-time hidden derivative in Equation 42.

Each bidirectional layer reverses the appropriate sequence and sums the two input derivatives; stacked layers are differentiated in reverse layer order.

The numerical CTC forward score uses the recurrence in Equation 43, written in log space from the CTC path recursion [14, p. 4].

$$\alpha_{t,s} = \log p_t(\ell'_s) + \mathrm{LSE}_{r\in\mathcal{P}(s)}\,\alpha_{t-1,r}. \tag{43}$$

For frame 11 and the V state in the real Owen line, the predecessor log-forward scores are −12.8882643, −8.9807719 and −0.1088294, and the log emission is −0.01988985. Equation 43 gives $\alpha = -0.01988985 + \log(e^{-12.8882643} + e^{-8.9807719} + e^{-0.1088294}) \simeq -0.12857619$. This sums the admissible predecessor mass, whereas the main-text forced alignment selects one predecessor.

The predecessor set in Equation 43 contains the current state, the preceding state and an admissible two-state skip. A skip is excluded when its destination is blank or repeats the label two states earlier. With log scores a, b, the stable pairwise reduction is $m + \log(\exp(a-m) + \exp(b-m))$, where $m = \max(a,b)$; the all-unreachable case remains $-\infty$. The backward recurrence uses successor emissions at $t+1$, yielding the occupancy convention in Equation 27.

A.2 Optimisers and gradient scaling

The reader uses an uncentred RMSprop accumulator, following the moving-square-gradient principle [41, p. 29]. The code selects its own averaging coefficient and adds the stated decay term. For parameter θ, raw gradient g, learning rate η and weight-decay coefficient λ_w, the implemented update is Equation 44.

$$g' = g + \lambda_w\theta, \quad v_t = 0.99v_{t-1} + 0.01(g')^2, \quad \theta_t = \theta_{t-1} - \eta\,\frac{g'}{\sqrt{v_t} + 10^{-8}}. \tag{44}$$

Decay in Equation 44 enters the gradient before accumulation. The writer's AdamW instead uses separate parameter decay, as shown in Equation 45 [42, p. 2], [43, p. 3].

$$\begin{aligned} m_t &= 0.9m_{t-1} + 0.1g, & v_t &= 0.999v_{t-1} + 0.001g^2, \\ \widehat{m}_t &= m_t/(1-0.9^t), & \widehat{v}_t &= v_t/(1-0.999^t), \\ \theta_t &= \theta_{t-1} - \eta\left(\frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + 10^{-8}} + \lambda_w\theta_{t-1}\right). \end{aligned} \tag{45}$$

Following global gradient-norm clipping [44], before the writer update all gradient tensors are scaled together by $\min(1, 5/\|g\|_2)$, with a numerical denominator guard. Clipping individual tensor norms independently would be a different operation. The numerical reader loop does not apply that clipping. Equations 44 and 45 describe code operations, not a claim that their default learning rates were used in every completed run.

A.3 Morphology, thinning and path simplification

For binary mask M and neighbourhood K, dilation is the maximum of translated mask values and erosion is their minimum. Closing applies dilation followed by erosion with the same footprint [35, pp. 533, 541]. For a binary mask and symmetric flat footprint, the set operations correspond to neighbourhood maximum and minimum. The implementation obtains rectangular neighbourhood counts through prefix sums; binary comparisons then implement these operations.

Connectivity analysis uses horizontal runs and union operations with diagonal overlap admitted, rather than a library component-labelling call.

The thinning conditions follow Zhang and Suen [26, p. 237]. The method inspects the eight neighbours $P_2, \ldots, P_9$ clockwise from north. A removable foreground pixel has two to six foreground neighbours and exactly one background-to-foreground transition around that ring. The two alternating subpasses additionally require, respectively,

$$\begin{aligned} P_2P_4P_6 = 0,\ P_4P_6P_8 = 0 \quad &\text{(first subpass)},\\ P_2P_4P_8 = 0,\ P_2P_6P_8 = 0 \quad &\text{(second subpass)}. \end{aligned} \tag{46}$$

During thinning of the actual Owen crop, the first recorded deletion in subpass one was pixel $(y,x) = (12,190)$ with neighbours $(P_2, \ldots, P_9) = (0,0,1,1,1,1,0,0)$. Its neighbour count is four and its circular transition count is one. Both first-subpass products are zero because $P_2 = 0$ and $P_8 = 0$, so the pixel is deleted when the subpass mask is applied. The deletion is applied with the other eligible pixels in that subpass.

Candidates in Equation 46 are deleted together in each subpass, and the pair of subpasses repeats until unchanged. This reduces thickness while limiting topology changes; it cannot restore ink already lost during thresholding. The same thinning primitive supports stroke normalisation, glyph tracing and the diagnostic retracing route.

The default tracer begins at graph nodes whose degree differs from two, follows unused pixel edges, and then recovers remaining closed loops. Short dangling spurs are pruned, and nearby chain ends are greedily rejoined when their continuation angle is below 75°. This remains a geometric estimate rather than a verified natural handwriting order. Polyline simplification compares interior points with the line through the endpoints, retaining a point and splitting recursively when the deviation exceeds tolerance. The code adapts the polyline reduction method [25]. Near-closed paths are first split at a distant point so that a zero-length endpoint chord does not collapse the loop. The extraction tolerance is 0.6 source pixels before normalisation; final path smoothing and machine decimation occur in their own coordinate units.

A.4 Profile and trajectory quantities

Table 15 records the fields most likely to be confused during inspection. Full arrays and every tuning constant remain in the source rather than expanding the main text into a variable dictionary.

Raw alignment confidence is a path score used for relative filtering, not a calibrated probability of a correct cut. The cross-writer prior uses a median occupancy descriptor, then unit normalisation; its minimum pooled sample count is twelve. Writer consensus retains approximately 75% of a sufficiently populated pool, with at least three variants. Compatible prototypes group equal stroke counts, resample strokes to 44 points, align them and take robust pointwise summaries. Admission to the variant pool requires prior distance below 0.32 and improvement of more than 0.02 over the median raw distance. These gates reduce malformed geometry but do not prove that a distinctive form is incorrect.

Machine conversion maintains image, glyph, trajectory and machine coordinate frames separately. The line-centre shifts in Equation 36 are zero when no respacing is requested. Workspace fitting and clipping are distinct: fitting scales the whole trajectory uniformly, while clipping changes individual points and can deform letters. Final file parsing provides a useful conversion check

Quantity	Meaning and unit
`xHeightPx`	Reference source x-height in pixels; not a physical pen height
`strokes`	Ordered point lists in glyph x-height units; *y* increases upwards
`width`, `lead`, `advance`	Ink width, lead-in and cell/placement extent; distinct x-height quantities
`top`, `bot`	Vertical extrema relative to the baseline, in x-heights
`connL`, `connR`	Observed boundary crossings during source extraction
`priorD`	Descriptor distance from the pooled character reference; not writer confidence
`idealMedoid`, `idealGlyphs`	Actual representative variant and compatible median-stroke prototype
`letterAdvance`	Per-character median source advance; sampled variants retain their own placement geometry
`lineStepMm`	Synthesised baseline spacing before optional machine respacing
`jointScore`	Internal harmonic selection score; not independent stylistic correlation
`clipped`	Count of points clamped by the motion-file conversion; not an acceptable layout target

Table 15. Selected implementation fields and their interpretation.

because it reconstructs movements from emitted commands instead of directly displaying the input trajectory again.

A.5 Detailed training procedures

The main chapter retains the model, loss, design choices and representative checks. The following pseudocode records the detailed recurrences and update sequence. Algorithm 5 adapts the CTC forward–backward method [14, pp. 4–6]; Algorithm 6 assembles the project's training operations.

Algorithm 5 Numerical CTC loss and logit gradient

Require: Logits $z \in \mathbb{R}^{T\times B\times C}$ and encoded targets.

log $p \leftarrow \mathrm{LogSoftmax}(z)$; $L \leftarrow 0$; $g \leftarrow 0$
for each batch item b **do**
  $\ell' \leftarrow$ target interleaved with blanks
  Initialise α, β to $-\infty$; admit first blank and first label
  for $t \leftarrow 1$ **to** $T-1$ **do**
    for each state s **do**
      $\alpha_{t,s} \leftarrow \log p_t(\ell'_s) + \mathrm{LSE}_{r\in\mathcal{P}(s)}\, \alpha_{t-1,r}$
    end for
  end for
  log $Z \leftarrow$ LSE of the admissible final states
  Initialise final β to zero; recurse backward with next-frame emissions
  $q_{t,k} \leftarrow \sum_{s:\ell'_s=k} \exp(\alpha_{t,s} + \beta_{t,s} - \log Z)$
  $L \leftarrow L - \log Z$; $g_{t,b,k} \leftarrow (p_t(k) - q_{t,k})/B$
end for
return $L/B, g$

A.6 Expanded numerical calculation checks

For one image, let X_c contain flattened input patches, so the matrix formulation of convolution gives $Y = WX_c + b$ [37]. Applying the chain rule to $Y_{oj} = \sum_k W_{ok}(X_c)_{kj} + b_o$ gives $\partial\mathcal{L}/\partial W_{ok} = \sum_j G_{oj}(X_c)_{kj}$ and $\partial\mathcal{L}/\partial(X_c)_{kj} = \sum_o W_{ok}G_{oj}$. Equation 47 collects these derived sums in matrix form.

$$\frac{\partial\mathcal{L}}{\partial W} = GX_c^T, \qquad \frac{\partial\mathcal{L}}{\partial X_c} = W^T G. \tag{47}$$

Weight and bias derivatives are summed over the batch and output positions. Overlapping patch derivatives in Equation 47 are added back into the padded input, not overwritten. Max-pool derivatives return to the recorded maxima, width-average derivatives are distributed uniformly, and the LSTM backward pass accumulates both hidden-state and cell-state contributions through time. Appendix A records the remaining derivative conventions and optimiser calculations without repeating the architecture here.

For the traced Owen crop, the first convolution uses one 64×640 input and a 3×3 kernel. Its patch matrix contains $(1)(1)(3)(3)(64)(640) = 368640$ values, requiring $368640 \times 4/2^{20} = 1.40625$ MiB in float32. This determines one intermediate array's storage, not peak process memory. Later-layer arrays and any training gradients add storage; training and forward-only inference footprints must be assessed separately.

Algorithm 6 One numerical training batch and validation checkpoint decision

Require: Training batch, task, parameters θ and optimiser state.

$(X, y) \leftarrow$ task-specific preparation and training augmentation
$(F, \mathcal{K}) \leftarrow \text{BackboneForward}(X, \theta)$; retain caches
if task is reader **then**
    $(z, \mathcal{K}_h) \leftarrow \text{BiLSTMHead}(F)$; $(L, g) \leftarrow \text{CTC}(z, y)$
else
    $(z, \mathcal{K}_h) \leftarrow \text{WriterHead}(\text{MeanWidth}(F))$
    $(L, g) \leftarrow \text{CrossEntropy}(z, y)$
end if
$g_F \leftarrow \text{HeadBackward}(g, \mathcal{K}_h)$
$\nabla\theta \leftarrow \text{BackboneBackward}(g_F, \mathcal{K})$
if task is writer **then** clip total gradient norm to 5
$(\theta, \text{state}) \leftarrow \text{OptimiserStep}(\theta, \nabla\theta, \text{state})$
if epoch is complete **then**
    Validate with fixed preparation, stored normalisation and dropout disabled
    Update schedule; save progress; replace best only if its criterion improves
end if
return L, θ, state

Numerical perturbation provides a finite-difference sensitivity check [33, pp. 18–19]. For the loss function here, the central-difference expression is derived by subtracting the two Taylor expansions $\mathcal{L}(\theta \pm \varepsilon e_i) = \mathcal{L}(\theta) \pm \varepsilon \partial_i \mathcal{L} + \frac{1}{2}\varepsilon^2 \partial_{ii} \mathcal{L} + O(\varepsilon^3)$. Division by 2ε cancels the constant and even terms, giving Equation 48 with truncation error $O(\varepsilon^2)$ where the loss is sufficiently smooth. One logit or parameter is perturbed at a time.

$$g_i^{\text{num}} = \frac{\mathcal{L}(\theta + \varepsilon e_i) - \mathcal{L}(\theta - \varepsilon e_i)}{2\varepsilon}. \tag{48}$$

For the real Owen line's ▽ logit at frame 11, the two losses with $\varepsilon = 10^{-4}$ are $\mathcal{L}_+ = 0.4703364611173033$ and $\mathcal{L}_- = 0.4703403997759956$. Equation 48 gives $g^{\text{num}} = -0.0196932934615$, compared with the analytic CTC derivative -0.0196932934309; the absolute difference is 3.06×10^{-11}. This verifies that logit derivative on a real input without claiming a complete network-gradient or training-run test.

Here θ is a parameter vector, e_i selects one component and ε is a small perturbation. Equation 48 requires care at ReLU or max-pool boundaries and with stochastic dropout. The input or mask must be controlled so that the two loss evaluations test the same local operation.

A.7 Implementation and interface reference tables

These tables preserve supporting implementation distinctions and representation contracts. The main chapter explains the corresponding choices and transitions.

Aspect	First principles route	Framework route
Layer calculations	Explicit forward and backward operations	Framework modules and automatic differentiation
Architecture	Project's reader and writer definitions	Corresponding layer pattern and parameter names
Data preparation	Project crop, label and split functions	Uses the same preparation functions
Weights for inference	Named array archive	Export to the same named array layout
Verification role	Implementation being checked	Reference for matched numerical comparisons

Table 16. Roles of the numerical and framework implementations.

Quantity	Source	Use in synthesis
Slant	Projection concentration under trial shears	Reapplied to stored upright glyphs
X-height	Height of the estimated line body band	Normalises stored paths; physical x-height sets drawing size
Ascender/descender extent	Relative glyph heights	Vertical proportions and line spacing
Letter advance	Character-cell and ink geometry	Placement of following characters
Word spacing	Measured gaps in real line ink	Separation of words
Connectedness	Observed skeleton crossings between adjacent characters	Probability of eligible in-word joins
Glyph variants	Filtered normalised trajectories	Writer-specific letterforms and variation

Table 17. Principal stored style quantities and their roles.

Stage	Required input	Output and governing convention
Segmentation	Photographed or saved page	Ordered raw crops and full-page boxes; image y points down
Recognition	Line crops and model arrays	Text plus writer probabilities interpreted by the checkpoint mapping
Profile construction	Original lines, known transcripts and reader	Writer-keyed glyph paths in x-height units, baseline origin and y up
Synthesis	Text, stored profile, scale and search settings	Millimetre trajectory, source tags and internal selection scores
File conversion	Trajectory and workspace configuration	Absolute millimetre G-code, pen events, placement and timing budget
Physical execution	G-code, connected GPIO and completed calibration	Step-counter updates and job outcome; physical accuracy requires observation

Table 18. Representation boundaries that connect the three main processing stages to the gantry.

A.8 Motion-file conversion procedure

Algorithm 7 Trajectory placement and motion-file preparation

Require: Selected trajectory T and calibrated workspace configuration.

Measure bounds; determine available width and height after margins and offset
$a \leftarrow$ uniform fit scale; apply robust height cap when ruled fitting is enabled
Group paths by text row; determine ruled or free baseline shifts
$\mathcal{P} \leftarrow$ machine coordinates from Equation 36
Decimate short segments; optimise path order within rows
Clamp remaining out-of-bounds points; record their count
Emit absolute millimetre mode and initial pen state
for each path $p \in \mathcal{P}$ **do**
    Close eligible near-ring; join previous path if configured gap permits
    if a new pen-down path is needed **then**
        Raise pen; travel to start; lower pen; emit configured dwells
    end if
    Emit drawing moves with modal drawing feed; accumulate ink and travel budgets
end for
Raise pen; save file, placement and separate timing budgets
return file path and job metadata

A.9 Controller pin and supply planning

Header pin / GPIO line	Connection	Direction and nominal level	Notes / verification
USB host	ODROID USB host → IMX179 camera	USB data and bus power	Camera current draw and captured resolution are not established by the sensor's advertised maximum.
Ethernet / network	ODROID RJ45 or Wi-Fi ↔ laptop/browser	Network data; no GPIO voltage	Operator request and response path.
ODROID DC jack	Supplied adapter → board power input	Actual adapter output unknown	The manufacturer's 12 V/2 A recommendation is not confirmation of the adapter in use [30].
External supply +	24 V supply → VMOT on both A4988 carriers	24 V DC; supply limit set to 2.11 A	A separate 0.35 A supply indication was recorded under one operating condition; neither value is the motor phase-current setting.
3.3 V header rail	ODROID dedicated 3.3 V output → both A4988 VDD inputs and pen-motor positive rail	3.3 V supply	Physical header pin and total rail loading are not recorded.
Ground return	ODROID GND, driver logic/motor returns, switch returns and BC337 emitter	0 V reference	A common signal reference is required; exact return routing and physical GND pin are not documented.

Table 19. Controller pin, signal-level and supply map for the installed system.

Header pin / GPIO line	Connection	Direction and nominal level	Notes / verification
7 / 64	`DIR1_PIN` → X-driver DIR	Output, nominal 0/3.3 V	Direction state; software inversion is configured for X.
11 / 68	`STEP1_PIN` → X-driver STEP	Output pulses, nominal 0/3.3 V	A4988 advances on STEP pulses; this is a logic input, not the motor supply.
12 / 81	`DIR2_PIN` → Y-driver DIR	Output, nominal 0/3.3 V	Direction state; software inversion is configured for Y.
13 / 69	`STEP2_PIN` → Y-driver STEP	Output pulses, nominal 0/3.3 V	A4988 advances on STEP pulses; this is a logic input, not the motor supply.
15 / 72	X-min switch → `X_MIN_PIN`	Input, active high, nominal 0/3.3 V	Reported 10 kΩ pull-down.
16 / 65	X-max switch → `X_MAX_PIN`	Input, active high, nominal 0/3.3 V	Reported 10 kΩ pull-down.
18 / 66	Y-min switch → `Y_MIN_PIN`	Input, active high, nominal 0/3.3 V	Reported 10 kΩ pull-down.
22 / 67	Y-max switch → `Y_MAX_PIN`	Input, active high, nominal 0/3.3 V	Reported 10 kΩ pull-down.
33 / 70	Pen-up switch → `PEN_SWITCH_PIN`	Input, active high, nominal 0/3.3 V	High indicates the raised state. Switch model and pull-down location are not recorded.
35 / 71	`PEN_MOTOR_PIN` → 1 kΩ base resistor → BC337	Output, nominal 0/3.3 V	GPIO controls the transistor only; motor current is supplied by the dedicated 3.3 V rail. A 10 kΩ base pull-down was reported; its connection is not documented. No flyback diode is fitted.
A4988 outputs	X/Y carrier 1A/1B and 2A/2B → motor coil pairs	Switched/chopped winding drive from VMOT	The coil voltage is not a fixed 24 V STEP signal. Phase-current limit depends on VREF and sense resistance; actual VREF remains unmeasured [29, pp. 4–5, 9].

Table 19. Controller pin, signal-level and supply map (continued).

B. Subsystem visual evidence and independent software checks

This appendix records newly run offline checks and illustrations made from existing project data. It supplements the Design explanations; the integrated qualification tests against the proposal remain in Results. The evidence scripts use the current processing modules, record source and input SHA-256 hashes, and write their outputs into this revision's `Evidence` directory. No training or physical motion was performed. Diagnostic settings are stated explicitly rather than inherited silently from server defaults. The results in this appendix are project experiments. Their inputs, settings and numerical outputs are recorded in `Evidence/bench.json`, `Evidence/camera_bench.json` and `Evidence/coefficient_sensitivity.json`; these manifests and the evidence scripts accompany the report source.

B.1 Actual capture variation and input resolution

The three camera-roll inputs in Figure 16 are `WIN_20261028_23_23_30_Pro.jpg`, `WIN_20261028_23_28_34_Pro.jpg` and `WIN_20261028_23_32_40_Pro.jpg`. Each stored image is 3264×2448 pixels. The page visibly contains three handwritten text rows. Automatic axis decisions were retained, including the failure in the third capture. The observed crop counts were 4, 3 and 28, respectively. Shadows are visually different, but neither lux levels nor physical angles were measured. These observations therefore explain a design limitation rather than qualifying operation across a specified lighting range.

The native Owen01 page (2448×3264) produced 20 crops for 20 labelled lines. A separate exploratory bench first fitted each of three original pages inside 1600×1600 pixels. From each reduced page it produced an unchanged view, a synthetic left-to-right multiplier from 0.42 to 1.00, a synthetic +3° rotation using bicubic interpolation on an expanded white canvas, and a quarter turn. Each was passed through the same automatic axis and segmentation functions. Table 20 retains every case, including unsuccessful ones.

Page	Label lines	Reduced	Gradient	+3°	90°
Owen	20	19	20	11	19
Dylan	24	24	24	25	24
Yeukita	26	26	26	26	26

Table 20. Exploratory crop counts after extra input reduction and digital perturbation. Equal counts do not establish correct ink ownership.

The reduced Owen case produced 19 crops and lost the usable first-line crop, whereas the native page produced 20. Its rotated reduced image produced only 11. The gradient image happened to restore the count to 20, which is not evidence that adding a shadow improves segmentation: thresholds, interpolation and component geometry changed together. These cases show why nominally scale-relative thresholds do not guarantee resolution invariance. A suitable follow-up would compare matched stroke ownership at controlled resolutions and improve capture guidance; the present bench does not supply pixel-level annotations or attribute every error to one operation.

B.2 Independent component and seam references

Algorithm 8 states the two independent reference comparisons. The run-based eight-connected component labeller was compared with an independently written flood-fill reference on 100 binary masks of size 9×11, using NumPy seed 2026 and foreground probabilities spanning 0.1–0.8. The test compared the full same-component relation between foreground pixel pairs, not arbitrary numerical label values. All 100 partitions agreed. Small random masks exercise diagonal contact and label equivalences, but do not demonstrate large-image speed or prove every possible topology.

For the seam solver, 20 random 4×6 cost arrays (seed 83) were checked against exhaustive enumeration of all row sequences with adjacent row changes of at most one. Both objectives included the implementation's $0.02|\Delta y|$ penalty. The maximum objective discrepancy was 0. The exhaustive reference tests optimal cost on those arrays; it does not establish that the chosen image-derived cost always follows the correct handwritten line boundary.

Algorithm 8 Independent partition and seam verification

Require: Fixed random seeds, mask cases and small seam-cost arrays.

for each binary mask M **do**
  $A \leftarrow \mathrm{LabelComponents}(M)$; $B \leftarrow \mathrm{FloodFill8}(M)$
  assert $[A_p = A_q] = [B_p = B_q]$ for every foreground pair (p, q)
end for
for each small cost array C **do**
  $p^* \leftarrow \mathrm{LeastCostSeam}(C, 1)$
  $\mathcal{P} \leftarrow$ all paths with adjacent row differences at most one
  $E(p) \leftarrow \sum_x C(p_x, x) + 0.02 \sum_x |p_x - p_{x-1}|$
  Record $|E(p^*) - \min_{p \in \mathcal{P}} E(p)|$
end for
return partition agreements and seam-cost discrepancies

B.3 CTC probability and derivative check

Algorithm 9 gives the exhaustive probability and derivative procedure. The loss check used float64 logits of shape $5 \times 1 \times 4$, blank index 0 and target $[1, 1]$. All 1024 frame paths were enumerated. Consecutive repeats were collapsed before blanks were removed, and only paths yielding the target contributed to its probability. This independently exercises the repeated-label restriction that a single uninterrupted run of label 1 cannot encode two labels. The path sum gave negative log likelihood 3.875974036731708; the dynamic programme gave 3.875974036731709. The absolute difference was 4.44×10^{-16}.

All 20 logits were perturbed individually with central-difference step 10^{-5}. The maximum absolute discrepancy from `train.py::CtcLoss` was 3.63×10^{-11}. Logits were drawn after the fixed-seed component-mask draws in the bench; the script preserves that order. These checks concern the implemented CTC loss and derivative on a finite feasible target. They do not test impossible alignments, every transcript length, trained recognition accuracy or the full CNN–LSTM backward pass.

Algorithm 9 Exhaustive CTC likelihood and finite-difference check

Require: Small logits z, target ℓ, blank index and step ε.

$(L, g) \leftarrow \mathrm{CtcLoss}(z, \ell)$; $p \leftarrow \mathrm{Softmax}(z)$; $Z \leftarrow 0$
for each frame path $\pi \in \{0, \ldots, C-1\}^T$ **do**
  if $\mathcal{B}(\pi) = \ell$ **then** $Z \leftarrow Z + \prod_t p_t(\pi_t)$
end for
Record $|L + \log Z|$
for each logit index i **do**
  $g_i^{\mathrm{ref}} \leftarrow [L(z + \varepsilon e_i) - L(z - \varepsilon e_i)]/(2\varepsilon)$
  Record $|g_i - g_i^{\mathrm{ref}}|$
end for
return loss discrepancy and maximum derivative discrepancy

B.4 Stored profiles, emitted files and physical page

Figure 30 uses the final Owen, Dylan, Yeukita and IAM 150 profiles with the same text, seed 2026, requested x-height 4 mm and jitter 0.15. The left column shows generated pen paths. The right column reconstructs pen-down paths from each emitted G-code file with the independent parser; each panel is fitted separately for legibility. The normal converter was used, including its path ordering, joins and near-ring closure. All four generated jobs had coordinates within the configured bed bounds. The figures illustrate profile-dependent geometry and file conversion, rather than measuring similarity to a human writer.

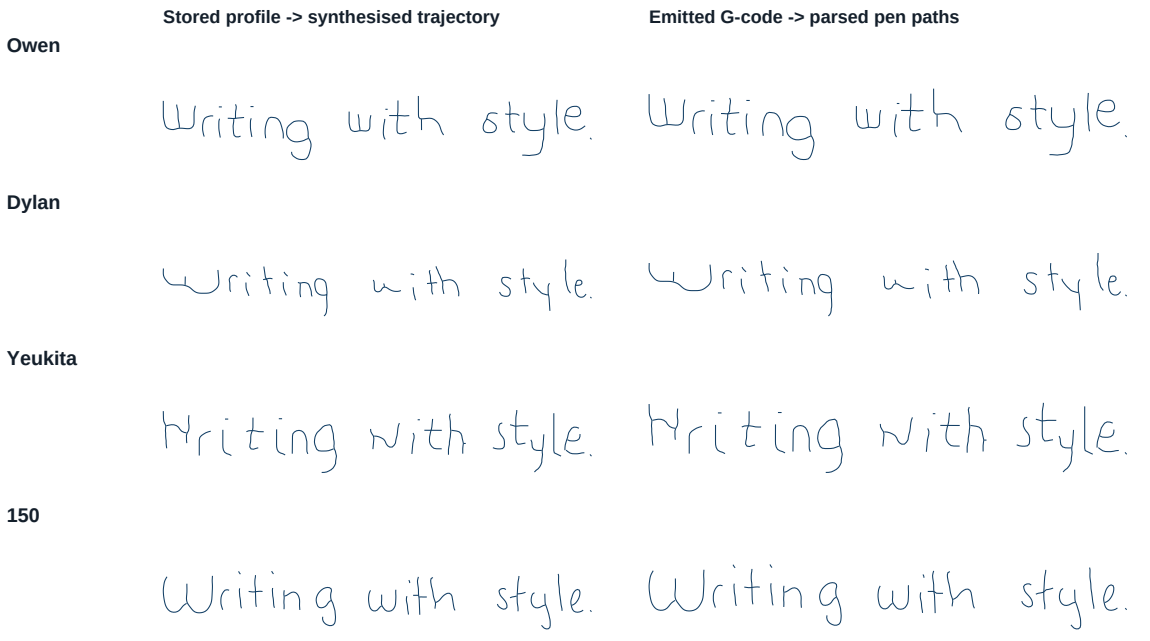


Figure 30. Same text generated from four final profiles and reconstructed from emitted G-code. These are software path drawings.

Algorithm 10 describes the serialization check. Nine controlled fixtures combined scale factors 1, 3 and 30 with line offsets 0, 8.4 and 25.2 mm. Ring closure and joining were disabled, minimum-segment filtering was set to zero, and the independently parsed paths were paired with `ToMachine`'s output. Each fixture retained path and vertex counts, with no clamped points. Maximum coordinate error was 0.0004 mm, within the 0.0005 mm per-coordinate rounding bound from three-decimal emission. This comparison checks serialization after placement; it does not independently prove that the placement equation itself is correct or that a motor reached a coordinate.

Algorithm 10 Controlled G-code serialization correspondence

Require: Known multi-path trajectory and configurations with joins and closure disabled.

1: **for each** fixture configuration c **do**
2: $P \leftarrow \mathrm{ToMachine}(T, c)$
3: $f \leftarrow \mathrm{WriteGcode}(T, c)$; $Q \leftarrow \mathrm{ParseGcode}(f)$
4: Compare path counts and corresponding vertex counts
5: Record $\max_i \|P_i - Q_i\|_\infty$ and clamped-point count
6: **end for**
7: **return** count agreements, rounding discrepancies and clipping records

Figure 31 is separately identified as a photograph of actual robot output, from the existing Owen reproduction sheet. The line crops are newly obtained from that stored photograph. This closes the visual explanation from a physical page to the segmentation interface, without presenting the earlier drawing as a newly executed hardware experiment. No exact same-job correspondence with Figure 30 is claimed.

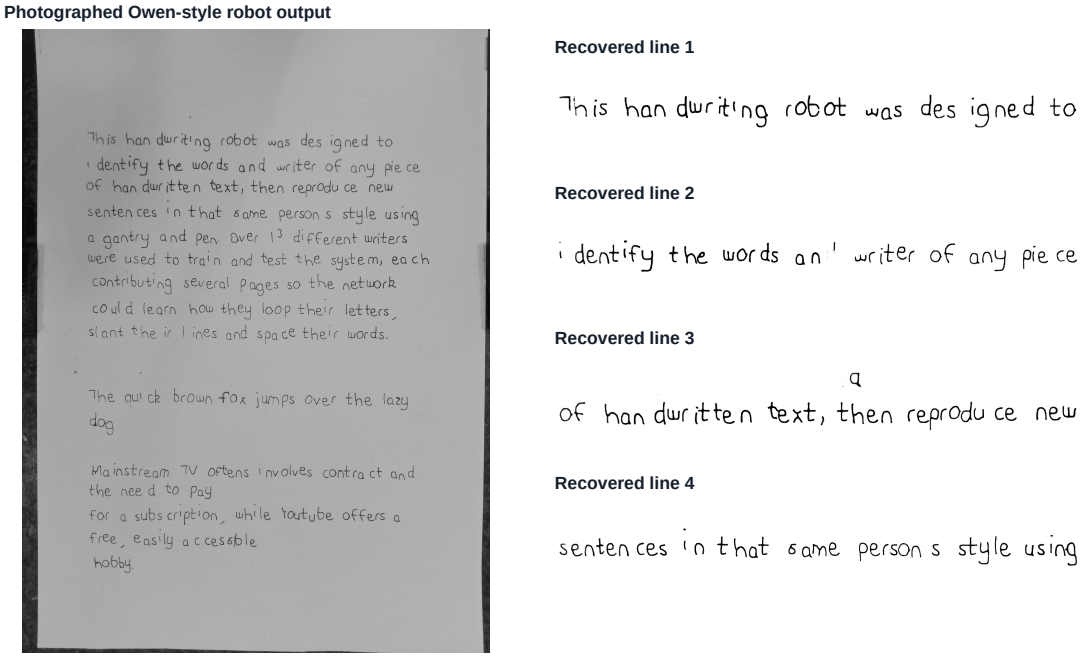


Figure 31. Existing photographed robot output and newly recovered line crops. The photograph is distinct from the software renders above.

B.5 Candidate-scoring configuration and scope

The Owen trials use the joint reader and eleven-writer `pw170` checkpoint, with explicit 0.17 normalisation and uniform-ink rendering. They score character accuracy and requested-writer posterior before repair, independently of inherited server defaults. Exact inputs, readings and full-precision scores are retained in `Evidence/bench.json`.

B.6 Post-design seam-coefficient sensitivity

A new one-at-a-time check examines the existing seam weights. Owen01 and Yeukita01 are reduced to a maximum dimension of 1600 pixels, then passed through orientation, conditioning and line-centre estimation. The first, middle and last gaps on each page are selected by index before inspecting outputs. Eight settings share these six gaps, step limit seven and ink weight four: midpoint weight varies at step penalty 0.02, then step penalty varies at midpoint weight 0.012. This is post-design sensitivity, not a joint search or recovered historical tuning.

The parameterised reference matches all six unmodified default paths. Table 21 reports raw-ink intersections, total row movement and mean midpoint distance in working pixels; the midpoint is a preference, not a labelled correct boundary.

Midpoint weight	Step penalty	Ink intersections	Row movement	Mean offset / px
0.000	0.020	2	35	4.0553
0.006	0.020	2	38	0.5517
0.012	0.020	2	42	0.3625
0.024	0.020	2	44	0.3261
0.048	0.020	2	47	0.2622
0.012	0.000	2	48	0.2622
0.012	0.010	2	44	0.3261
0.012	0.040	2	38	0.5517

Table 21. New coefficient-sensitivity check across six fixed real-page gaps; the implemented setting is 0.012/0.020.

Increasing midpoint weight from zero to 0.012 reduces mean offset from 4.055 to 0.363 pixels, while row movement rises from 35 to 42 pixels. Doubling the weight gives 0.326 pixels offset. Increasing step penalty has the opposite trade-off. Every setting retains two raw-mask and three closed-mask intersections. These cases therefore show competing effects without establishing a unique optimum, eliminating intersections or measuring recognition accuracy. Input/code hashes and all 48 runs are supplied; no pipeline coefficient was changed.

B.7 Real-input calculation traces

The numerical substitutions in Design were captured from offline processing of existing project inputs. They are execution traces of the stated calculations, not reconstructed historical training decisions or measurements of the mechanism. The complete settings, input/model hashes and full-precision values are stored in `Evidence/real_calculations.json`;

`trace_real_calculations.py` reproduces them without changing model weights or final profiles.

The page-conditioning trace uses Owen01 reduced to 1600 rows by 1200 columns, giving an 800-row by 600-column working image before deskew. The network and extraction trace instead uses the separately identified archived first original Owen01 crop (156×2170), its known transcript and the joint reader. The latter produces $160 \times 1 \times 75$ log probabilities: the checkpoint head has 75 rows, consistent with 74 alphabet symbols and blank. The selected eleven-writer model uses explicit 0.17-width preparation for the two-crop vote. These inputs must not be mistaken for one continuous new whole-system test.

For the first convolution calculation, the actual input patch and learned kernel, listed in the same row-major order, are

$$X = \begin{bmatrix} -1 & -1 & -1 \\ -0.552941 & -0.380392 & -0.568627 \\ 0.866667 & 0.960784 & 0.866667 \end{bmatrix}, \qquad W = \begin{bmatrix} 0.213027 & -0.215063 & 0.234474 \\ 0.068398 & 0.083297 & 0.187499 \\ -0.312354 & -0.291350 & -0.221197 \end{bmatrix}.$$

Using the full-precision values with bias -0.000435125 gives -1.15133214 at output channel 0, row 34, column 462. Rounded values above explain the sum; the manifest retains the precise arrays needed to reproduce it. The subsequent batch-normalisation and LSTM substitutions follow this same crop through the reader.

The path and timing trace generates “Writing with style.” using the final Owen profile, seed 2026, requested x-height 4 mm and jitter 0.15. The generated file is `Evidence/real_trace_owen.gcode`. Its default configuration supplies drawing/travel feeds 900/2400 mm/min, nominal conversion 80 microsteps/mm and the 190.6 by 251.3 mm workspace. No physical command was sent. The pulse-delay calculation uses one emitted segment at that nominal scale; actual homing counts remain a separate hardware measurement. The trace also records the pre-file coordinate mapping, distance accumulators, 56 pen transitions and the resulting 32.8 s nominal budget.

The existing random-mask, exhaustive seam and small-lattice checks remain independent numerical tests. Their synthetic inputs are intentional test fixtures. They are separate from these real-input traces and from the qualification results.

B.8 Visual checks of numerical training operations

The retained visual-test record contains 249 scalar perturbation comparisons: 73 sampled convolution-weight entries, 73 bias comparisons, 73 input entries and all 30 logits of a small CTC lattice. The sampled entries need not be distinct parameters. Central differences use a step of 10^{-6} for the convolution and 10^{-5} for CTC, following the numerical-check principle in Equation 48. Figure 32 compares the resulting numerical derivatives with the implemented backward operations. Relative difference is the absolute discrepancy divided by the sum of the two derivative magnitudes, with a 10^{-12} denominator floor.

The largest relative difference is 7.50×10^{-7}, among the sampled convolution inputs; the largest absolute input discrepancy is 7.26×10^{-9}. The CTC relative difference is at most 1.69×10^{-9}. These checks support the tested local derivatives under the fixed fixtures. They do not exercise every layer, parameter, floating-point regime or complete training run.

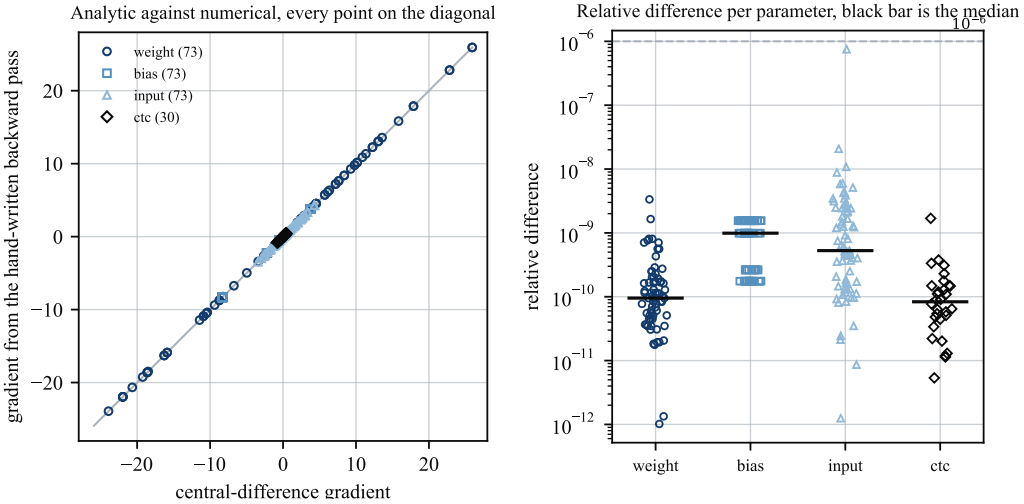


Figure 32. Analytic and central-difference gradients for sampled convolution quantities and a complete small CTC logit array.

B.9 Stroke-width quantisation in the validation sweep

Figure 33 shows four real crops processed at the four sweep fractions. The image transformation depends on the resulting odd integer dilation kernel as well as the fraction. For Owen and Abhinav, 0.06 and 0.09 both select $k = 3$ on the illustrated crop; for Dylan, 0.09 and 0.11 both select $k = 5$. Identical kernels give identical normalised images for that same crop. This does not imply that two settings produce identical training datasets, since x-height varies across crops. The validation bars reproduce the supplied training sweep, not a new accuracy measurement. Its best observed setting remains 0.17, while a single run at each setting does not isolate width effects from training variability.

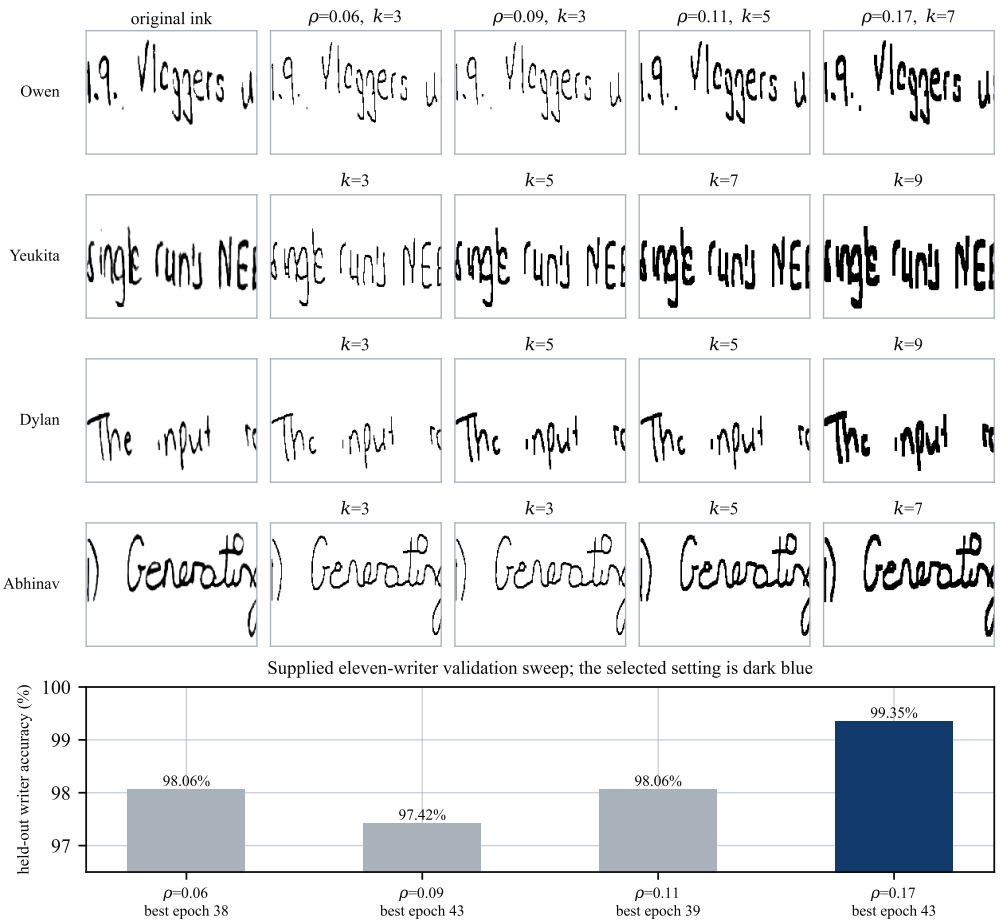


Figure 33. Real-crop normalisation kernels alongside the previously reported eleven-writer validation sweep.

B.10 Glyph coordinates and measured connectedness

Figure 34 shows the actual Owen V used in the coordinate calculation in Design: left boundary 146 pixels, baseline 107 pixels, x-height 38 pixels and slant −4.5°. The stored path uses the baseline as its origin, measures height upwards and expresses lengths in x-heights. The lead is stored separately from the path so that source-cell spacing is not confused with the first surviving ink point.

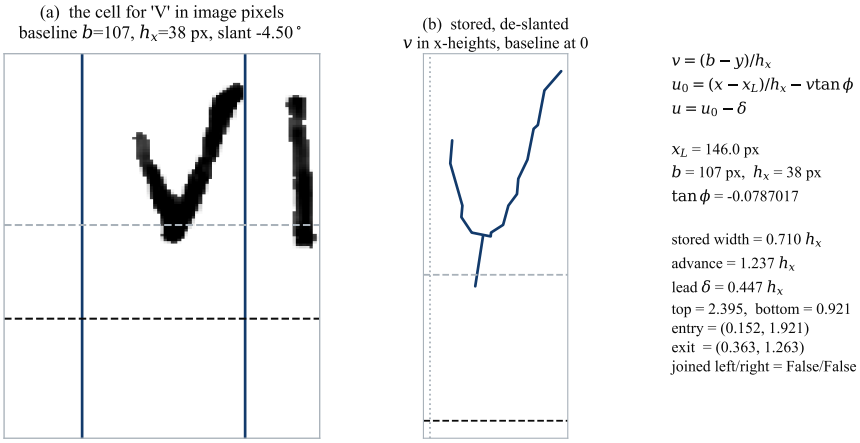


Figure 34. The same glyph in image coordinates and in its stored, de-slanted coordinate frame.

Figure 35 makes the connectedness definition inspectable on the same line. Three of its 39 eligible adjacent non-space boundaries are crossed by a traced centreline, giving $3/39 = 0.076923$. This is the line-level example already used in Design. A crossing records connected source ink under the extraction convention; it does not reveal the writer's natural pen-lift sequence or establish a connection fraction for every page by itself.

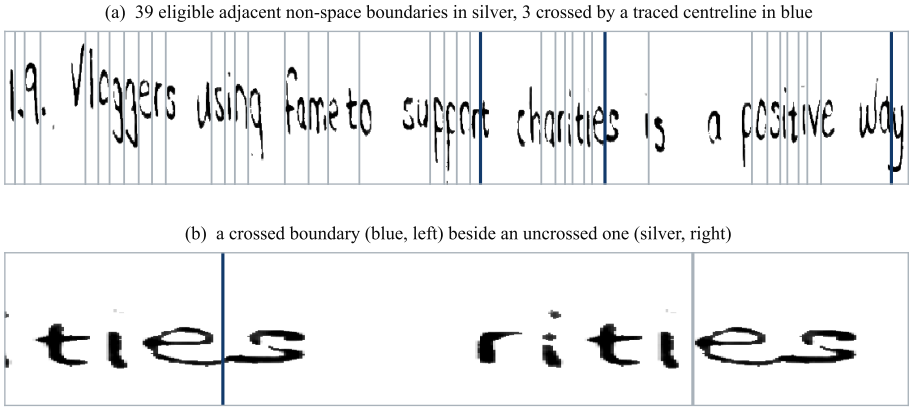


Figure 35. Eligible character boundaries and the three observed centreline crossings in one original line.

C. Supporting qualification evidence

This appendix preserves the detail needed to interpret the main qualification tests without repeating the Design's subsystem derivations. The source package contains the original v6 measurement archive, the reviewed aggregate records and the scripts that generate the revised tables and figures. Newly computed analyses are identified separately from the original measurements. No physical measurement is generated by a plotting script.

C.1 Evaluated checkpoints and populations

The selected demonstration reader is the library-trained joint reader, as confirmed by the author. The numerical scratch reader is a comparison model. The selected writer model is the library-trained eleven-writer 0.17 model. Table 22 records hash prefixes; the evidence manifest retains the full SHA-256 values. A filename alone is insufficient because the archived reader study identified two different files named `scratch_best`.

Role	Checkpoint identifier	SHA-256 prefix
Selected reader	joint_best	`336802d535992323`
Comparison reader	evaluated scratch_best	`2add315ded078de5`
Selected writer	11_pw170	`1884fb86467538eb`
Comparison writer	11w_pw170_scratch	`b23a709a30144f67`

Table 22. Identities of the measured model files.

The numerical-writer training excerpt supplied by the author records 1 031 training lines and 155 validation lines, 0.17 normalisation, a 50-epoch budget and a best displayed validation accuracy of 0.968 by epoch 41. The retained excerpt continues through epoch 47 and does not document a completed epoch 50. Its later common-set score is measured on 156 re-extracted validation lines; the different population is retained rather than silently equated with training validation. The numerical-reader excerpt records 6 481 public training lines, 569 personal training lines, 976 public validation lines and 100 personal validation lines, and a resume from an existing checkpoint. These are training-history records, while QT 2 evaluates saved weights directly.

An additional inventory reads each selected stored profile and verifies nonempty glyph mappings. The numbers of distinct character keys are 42, 38, 37, 36, 40 and 41 for the six database writers, and 56, 42, 43, 42 and 58 for A, B, O, T and Y. All eleven names agree with the selected writer checkpoint. The full source paths and content hashes are preserved in `ResultsEvidence/profile_inventory.json`. These counts establish stored mappings; they do not imply equal coverage of every character in the reader alphabet.

C.2 Physical reproduction by writer

The paired read-back study contains 184 transcript-bearing lines. A separate model comparison contains 185 writer-labelled crops, including one crop with no corresponding transcript line. It can support a writer-prediction comparison, but that extra crop is not inserted into a character

Writer	Lines	Character / %	Writer / %	Word match / %
Y	13	97.01	92.31	74.16
150	13	96.75	53.85	76.40
O	16	94.25	100.00	58.51
B	16	93.11	93.75	68.24
384	16	86.93	87.50	43.18
552	15	86.17	80.00	48.39
152	15	85.74	80.00	49.44
A	27	77.29	100.00	18.09
153	19	75.72	57.89	31.46
588	18	73.19	94.44	22.99
T	16	60.97	100.00	13.41

Table 23. Full per-writer reproduction results on 184 labelled lines. Word match uses equal word positions and is not one minus word edit error rate.

score. The model-comparison pooled groups are writers, not a claim that only eleven physical sheets were drawn.

C.3 New geometric style comparison on physical pages

The new analysis applies the archived descriptor definitions with the current segmentation primitives to 156 original reference lines and 184 physical-output lines. The original population consists of the same six held-out database pages and 100 seeded photographed reference lines used for writer evaluation. The physical population comprises all fourteen robot pages after the recorded orientation convention. Each page's crop count agrees with its transcript-bearing records, and every line returns usable descriptors. All eleven writers are retained.

Each line is binarised, its body band estimated and its image resized toward a 28-pixel body height before the descriptors are evaluated. The same procedure is applied to both domains. Writer means give equal weight to available lines for each feature, and the pooled correlation gives equal weight to writers and selected features. Word-gap values are absent where no qualifying gap exists: 155/156 original and 180/184 physical lines have a value; the other descriptors are available for every line. All eleven writer means remain usable. The within-domain variant separately standardises each feature in each domain; the absolute variant uses original-domain means and standard deviations for both. Source images, the personal-crop cache, scripts and full measurements are recorded by hash. The raw outputs are in `ResultsEvidence/physical_style/results.json`.

The per-feature correlations are slant 0.9911; stroke width 0.7986; ink density 0.7392; word gap 0.4861; components per word 0.6688; character width 0.7978; ascender extent 0.8173; descender extent 0.5394; and extent ratio 0.5762. Figure 36 shows the actual writer means. The 10 000-repeat writer-resampling range for the seven-feature within-domain coefficient is 0.5555–0.8272, using seed 23539195. It is a sensitivity range over this writer set. This retrospective comparison does not use identical sentences between domains and should not be interpreted as a controlled perceptual-equivalence experiment.

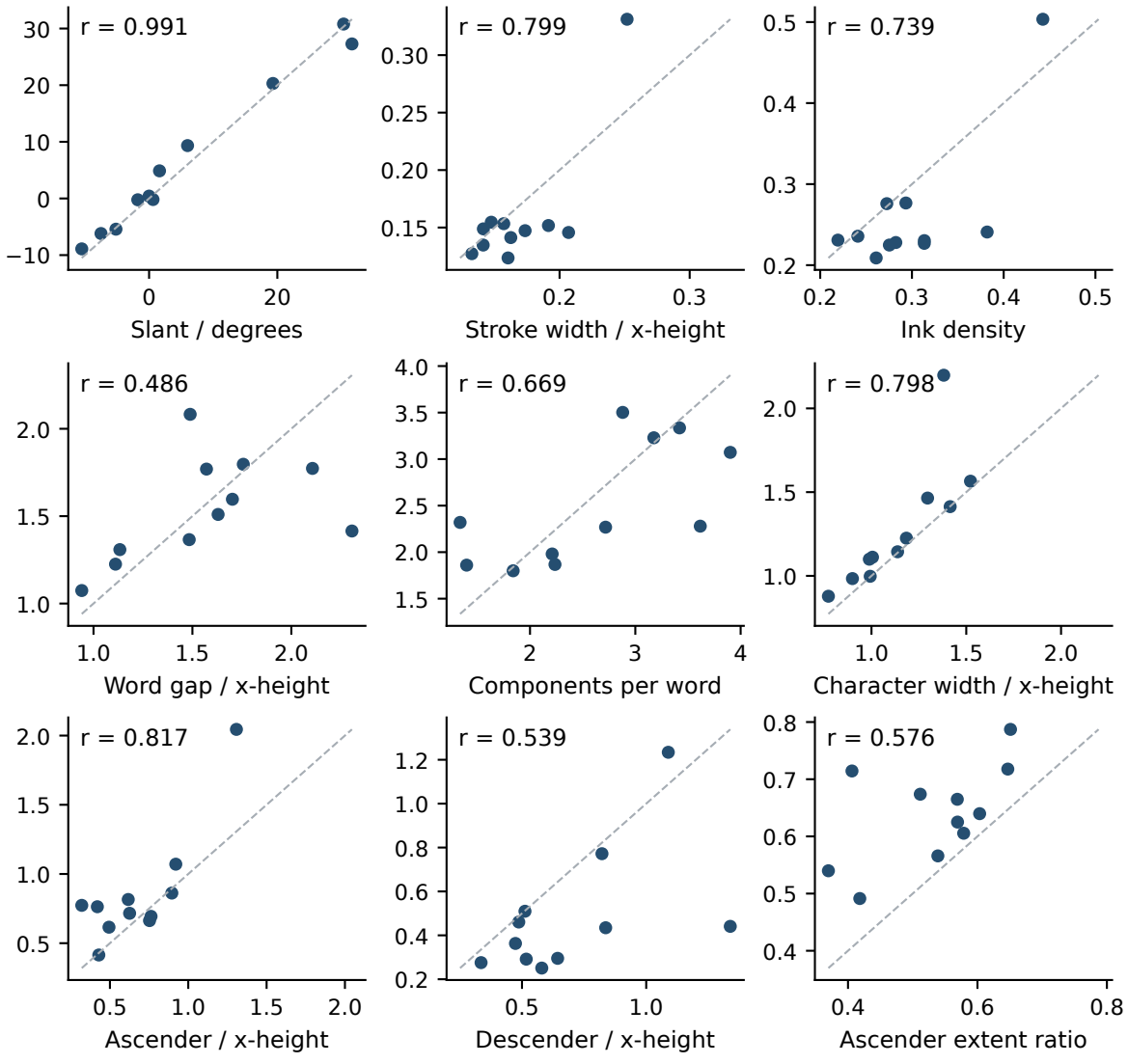


Figure 36. New physical-to-original style descriptors across eleven writers. Each point is one writer mean. Dashed lines indicate equal values; correlation can remain high even when magnitudes differ.

C.4 Archived geometric style study

The archived study covers writers 150, 151, 152, 153, 384, 551, 552, 588, B and Y. There are six synthetic lines per writer, sixty in total, compared with 123 original reference lines marked as excluded from profile building. This earlier ten-writer set differs from the final eleven-writer evaluation. The study is retained to explain geometric behaviour of the method; it is not relabelled as a measurement of final physical pages.

Nine deterministic descriptors measure slant, normalised stroke width, ink density, word gap, components per word, character width, ascender extent, descender extent and the ascender-to-total-extent ratio. Seven shape descriptors omit stroke width and ink density. The within-domain pooled correlation first standardises each feature across writers separately in the real and synthetic domains, then correlates the flattened arrays. It is therefore insensitive to a uniform offset or scale difference between those domains. The alternative absolute variant standardises both arrays using the real-domain mean and standard deviation. This distinction matters when interpreting preservation of magnitude rather than writer ordering.

Feature set	Separate-domain standardisation	Real-domain standardisation
Nine features	0.7013	0.5994
Seven shape features	0.7142	0.6711

Table 24. Archived synthetic-to-original pooled correlations against the profile-held-out reference. The 0.714 value is the seven-shape-feature variant.

The per-feature coefficients are 0.9964 for slant, 0.6315 for stroke width, 0.6802 for ink density, 0.6050 for word gap, 0.8323 for components per word, 0.9358 for character width, 0.9735 for ascender extent, 0.1480 for descender extent and 0.5085 for the extent ratio. Correlation describes variation across writers; it does not prove pointwise agreement of all letter shapes. The recorded 2 000 writer-resample interval for the seven-feature within-domain result is 0.6118–0.8586.

The archived overlay experiment uses thirty source texts across those ten writers. With x-height normalisation, mean matched chamfer distance is 0.6302, mean mismatched distance is 1.3868, their ratio is 2.2007 and the recorded AUC is 0.7074. When the archive also applies its width-normalisation variant, AUC is 0.4630. Thus the favourable separation depends on the chosen normalisation; an AUC above 0.5 in one variant must not be presented as universal writer fidelity. These remain software comparisons, without a new independent human panel.

C.5 Timing selection and optimisation scope

QT 3 uses a deliberately checkable completion boundary. The earlier archive retained 36 records after deduplication and a drawing-rate filter. The revised primary count uses 25 positive-character console records with explicit program-end evidence. The original 36-record aggregation remains in the preserved provenance document. A rate threshold alone does not establish completion: a job stopped after substantial drawing can exceed that threshold. This change does not alter the observed worst 3.875 s per character. A terminal marker confirms the interpreter finished, while geometry and content remain separate qualification questions.

The 500- and 982-character counts for QT 4 are the counts printed in the timer record. A final repeat should retain independent transcript lengths and an acquisition-to-output timing boundary. The two current overruns are large, so no inferred sub-second camera transfer time is added to create a favourable pass.

Comparison	Measured outcome and scope
Sequential NumPy vs batched library networks	359.8 s vs 130.3 s over fourteen stored pages on the development host; factor 2.76. Both backend and batching change. Reader text is identical; writer-label equivalence is not inferred from that fact.
Full vs half linear image resolution	4.37-fold recorded image-stage factor on comparable successful page pairs, with 247 vs 182 detected lines. Zero of twelve comparable pages retain line count; two of fourteen reduced pages fail segmentation.
Projected network-only replacement	Applying 2.76 to ODROID network stages, with image stages unchanged, gives approximately 27.05 s and 30.82 s. Neither is an observed ODROID result or a total-time pass.
Projected combined replacement	Applying both host factors gives approximately 14.61 s and 17.86 s. The second is numerically below its allowance, but the image reduction changes the output and the timing is transferred between hosts. It is not accepted as R4 compliance.

Table 25. Acceleration evidence and the boundary between a measured comparison and a projected compliant configuration.

The G-code study of the same 378-character paragraph yields pen-down path lengths from 2 015.4 to 3 942.8 mm and recorded pen-state counts from 544 to 1 188 across eleven profiles. These are arithmetic properties of generated paths. They demonstrate profile-dependent geometry and help explain timing variation; they are not independent proof of similarity to genuine handwriting, and their values can still be affected by the candidate-selection process.

C.6 Cost, consent and final records

The photographed ledger supplied by the author is retained as evidence. The present build estimate is approximately R4 000, with a borrowed ODROID entered at zero purchase cost. The later listed clipboard (R90), paper (R130), gel pen (R30) and estimated training electricity (R500) total R750. Their overlap with the approximate build estimate is not established, so they are not added to produce an unsupported exact total. Replacement value and estimated operating costs should be distinguished from actual purchased parts.

Signed consent forms are held by the author for the five personal contributors. The report records that fact without reproducing signatures. Participant consent is recorded separately from institutional clearance or exemption.

C.7 Supporting timing and dimensional figures

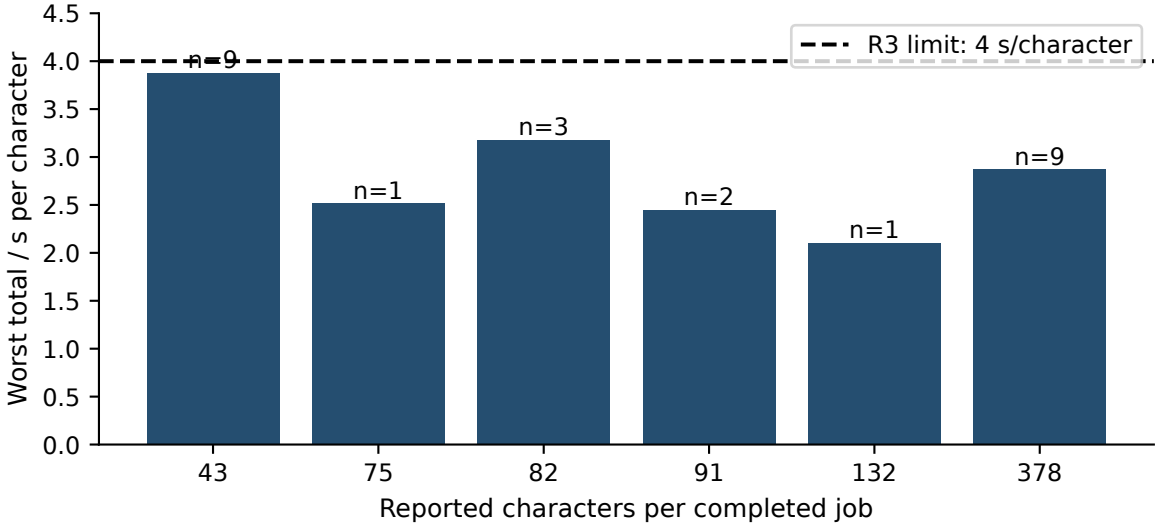


Figure 37. Worst recorded generation-plus-drawing time at each tested length, using only explicitly completed jobs. Labels state the number of records. The timing result applies to these recorded jobs and does not imply identical quality for every writer.

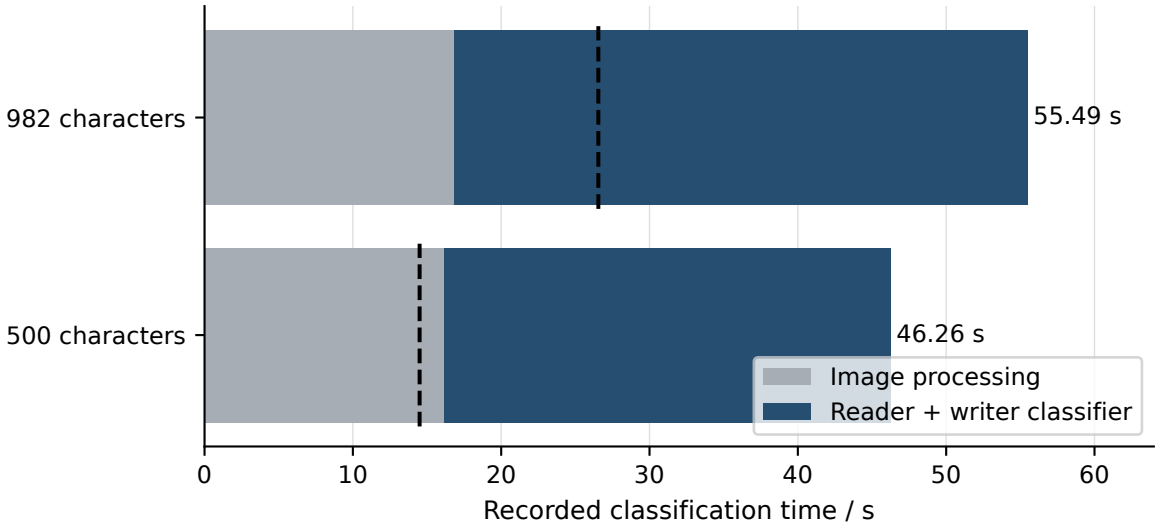


Figure 38. Recorded ODROID classification time split into image and network stages. Dashed markers show the complete R4 allowance at each logged character count. Both measured totals exceed their allowance.

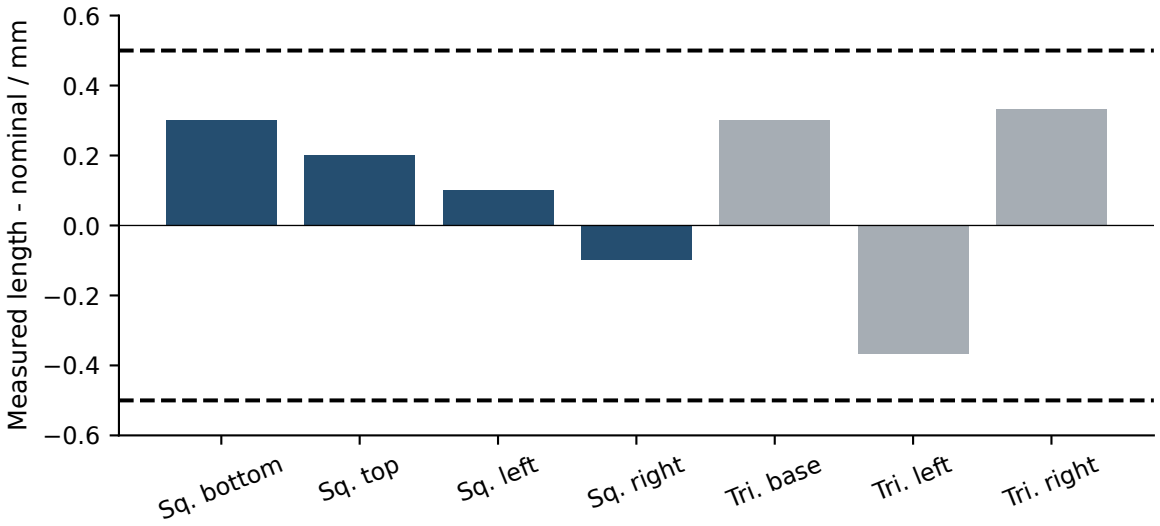


Figure 39. Signed dimensional differences for one square and one triangle. Dashed lines are the 0.5 mm reference. Being inside those lines is a feature-length observation, not an absolute coordinate-error bound over the bed.